

NATIONAL TECHNICAL UNIVERSITY OF ATHENS
DEPARTMENT OF MECHANICAL ENGINEERING
INTERDEPARTMENTAL POSTGRADUATE PROGRAM "AUTOMATION SYSTEMS"

ON THE USE OF NEURAL NETWORKS IN THE ANALYSIS AND PREDICTION OF HUMAN GAIT

MASTER THESIS

of

EVANGELOS KARAVAS

Supervisor: Ioannis Poulakakis
Associate Professor

Athens, September 2026



NATIONAL TECHNICAL UNIVERSITY OF ATHENS
DEPARTMENT OF MECHANICAL ENGINEERING
INTERDEPARTMENTAL POSTGRADUATE PROGRAM "AUTOMATION SYSTEMS"

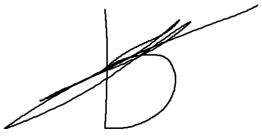
Copyright © – All rights reserved.
Evangelos Karavas 2026.

The copying, storage and distribution of this Master thesis, exall or part of it, is prohibited for commercial purposes. Reprinting, storage and distribution for non - profit, educational or of a research nature is allowed, provided that the source is indicated and that this message is retained.

The content of this thesis does not necessarily reflect the views of the Department, the Supervisor, or the committee that approved it.

DISCLAIMER ON ACADEMIC ETHICS AND INTELLECTUAL PROPERTY RIGHTS

Being fully aware of the implications of copyright laws, I expressly state that this Master thesis, as well as the electronic files and source codes developed or modified in the course of this thesis, are solely the product of my personal work and do not infringe any rights of intellectual property, personality and personal data of third parties, do not contain work / contributions of third parties for which the permission of the authors / beneficiaries is required and are not a product of partial or complete plagiarism, while the sources used are limited to the bibliographic references only and meet the rules of scientific citing. The points where I have used ideas, text, files and / or sources of other authors are clearly mentioned in the text with the appropriate citation and the relevant complete reference is included in the bibliographic references section. I fully, individually and personally undertake all legal and administrative consequences that may arise in the event that it is proven, in the course of time, that this thesis or part of it does not belong to me because it is a product of plagiarism.



.....
Evangelos Karavas

Abstract

Human gait prediction is a challenging problem due to the nonlinear, hybrid, and highly variable nature of human walking. Accurate and stable gait modeling is very important for applications in assistive robotics, where predicted joint trajectories must remain consistent when executed within a closed-loop control framework. This thesis investigates the use of deep learning techniques for predicting lower-limb joint trajectories, with a particular focus on how different representations influence long-horizon prediction stability and generalization to previously unseen patients.

Multiple neural network architectures were developed, trained, and evaluated, including Long Short-Term Memory (LSTM) and Convolutional Neural Network (CNN) models using both timestamp and phase-variable inputs. The models were trained and evaluated on motion capture data provided by the ELEPAP rehabilitation center, using a split at the level of whole patients, so that every stride belonging to a test patient was entirely unseen during training and the reported performance reflects generalization to previously unseen children with cerebral palsy. While all architectures achieved accurate short-term prediction on a single fixed test segment, substantial differences emerged once they were evaluated across sixteen held-out patients and all three anatomical planes. The decisive factor proved to be the architecture rather than the temporal representation: both recurrent models held their mean absolute error near two degrees, whereas both convolutional models exceeded six degrees and failed to explain the measured motion on the majority of unseen patients. Phase-variable conditioning contributed a distinct property. Its effect on one-step-ahead accuracy was small, but once the models were required to run recursively on their own output, both phase-variable models preserved stride-to-stride alignment over extended execution while both timestamp models accumulated phase drift until their trajectories diverged. This held for both architectures, and was most pronounced at joints characterized by fast velocity transitions such as the knees and ankles. Recurrence and phase-variable conditioning therefore govern two different and equally necessary properties, accuracy on unseen patients and temporal stability under execution, and the phase-variable LSTM (PV-LSTM) is identified as the model recommended by this study as the only architecture that provides both. To assess model behavior beyond offline prediction, a ROS2 and Gazebo simulation framework was developed to execute the generated trajectories on a simulated lower-limb robotic system. This framework enabled direct visualization and comparison of executed joint trajectories using a position controller, revealing execution-level behaviors and temporal inconsistencies that are not fully captured by standard error metrics alone. The visualization results provided important insight into the stability and robustness of each model when deployed

in a realistic control environment.

Overall, this work demonstrates that phase-variable conditioning, particularly when paired with a recurrent architecture, provides a robust and biomechanically meaningful foundation for gait trajectory generation using neural networks. The findings highlight the importance of temporal representation when transitioning from offline gait prediction to stable execution and contribute toward the development of reliable learning-based control strategies for assistive locomotion systems.

Keywords

GAIT, Neural Networks, LSTM, CNN, Cerebral Palsy, Phase Variable, Gait Cycle, Lower-Limb Robotic Devices, ROS2, Gazebo

Acknowledgments

I want to express my sincere gratitude to my supervisor, **Ioannis Poulakakis**, for the continuous guidance, constructive feedback, and encouragement throughout this work, and most importantly, to my family, who dedicated their lives to us, I dedicate this to you.

Evangelos Karavas

Table of Contents

Abstract 1

Acknowledgments 3

1 Introduction 11
- 1.1 Rehabilitation Robotics and Machine Learning . . . 11
- 1.2 Problem Statement . . . 13
- 1.3 Thesis Objectives and Contributions . . . 13
- 1.4 Thesis Outline . . . 14

2 Literature Review 16
- 2.1 GAIT Analysis and Biomechanics . . . 16
  - 2.1.1 Gait Events and Gait Phases . . . 16
  - 2.1.2 Biomechanical Differences Between Typical and Pathological Gait . . . 19
- 2.2 Neural Networks for GAIT Prediction . . . 19
  - 2.2.1 Comparative Analysis of Neural Network Architectures for Gait Prediction . . . 21
- 2.3 Phase Variable Signal . . . 21
- 2.4 Control for Wearable Robotics . . . 23
- 2.5 Limitations of Existing Literature . . . 24
- 2.6 Chapter Summary . . . 24

3 Neural Network Approach to Predict GAIT 26
- 3.1 Lower-Limb Data . . . 26
  - 3.1.1 Data Provider . . . 26
  - 3.1.2 Differences between Typically Developed and Cerebral Palsy Data . . . 31
- 3.2 Data Preprocessing . . . 32
  - 3.2.1 Stride Segmentation and Dataset Augmentation . . . 32
  - 3.2.2 Signal Smoothing and Noise Reduction . . . 33
  - 3.2.3 Normalization and Data Cleaning . . . 33
  - 3.2.4 Formatting and Dataset Splitting . . . 33
- 3.3 Phase Variable Approach . . . 34
  - 3.3.1 Role of the Phase Variable in Neural Network Prediction . . . 38
- 3.4 Neural Network Analysis and Architectures . . . 38
  - 3.4.1 Tools for Neural Network Models . . . 39
  - 3.4.2 Training Setup and Hyperparameter Selection . . . 40

3.4.3 Long Short-Term Memory (LSTM) Architectures . . . 41
3.4.4 Convolutional Neural Network (CNN) Architectures . . . 42
3.4.5 Extension to All Anatomical Planes . . . 44
3.5 Chapter Summary . . . 44

4 Comparison of Neural Network Models **46**
4.1 Comparison Using Error Metrics . . . 46
4.1.1 Interpretation of RMSE and MAE . . . 47
4.2 Comparison of Prediction Behavior . . . 48
4.2.1 One-step-ahead predictions . . . 48
4.2.2 Multi-stride predictions on CP data . . . 50
4.2.3 Rolling one-step-ahead prediction and error accumulation . . . 51
4.2.4 Absolute error trajectories and failure cases . . . 53
4.2.5 Residual distribution (bias and variance) . . . 54
4.3 Generalization Across Held-Out Subjects and Gait Phases . . . 55
4.3.1 Results Across All Held-Out Patients . . . 56
4.3.2 From One-Step-Ahead Predictions to Stride- and Phase-Averaged Trajectories . . . 58
4.3.3 Stride-Averaged Predictions Over the Gait Cycle . . . 59
4.3.4 Error Distribution Across Gait Phases . . . 61
4.3.5 Subject-Specific Failure Modes . . . 62
4.3.6 The Median Case . . . 63
4.3.7 The Worst Case . . . 64
4.3.8 Comparison with a Typical-Gait Subject . . . 66
4.3.9 Clinical Weighting of the Three Anatomical Planes . . . 67
4.4 Summary of Findings . . . 68

5 Visualization of Neural Network Gait Predictions using ROS2 **71**
5.1 ROS2 Framework . . . 71
5.1.1 Motivation for Using ROS2 . . . 73
5.2 Gazebo Simulation . . . 74
5.2.1 Simulation Setup . . . 75
5.2.2 Kinematic Constraints and Joint Modeling . . . 75
5.3 Control Architecture . . . 77
5.3.1 Integration with the `ROS2 Control` Framework . . . 78
5.3.2 Joint Publisher Node . . . 79
5.4 ROS2 Data Visualization and Results . . . 81
5.5 Chapter Summary . . . 86

6 Epilogue **88**
6.1 Conclusions . . . 88
6.2 Future Work . . . 89

Bibliography **91**

A Source Code **95**
A.1 Simulation Launch Instructions 95
A.1.1 Workspace Build 95
A.1.2 Environment Setup 95
A.1.3 Launching the Simulation 95
A.2 Neural Network Models Scripts 96
A.2.1 CNN Model 96
A.2.2 LSTM Model 97
A.2.3 Phase Variable Computation 97
A.3 ROS2 - Gazebo Scripts 98
A.3.1 Joint Publisher 98
A.3.2 ROS2 Launch file 100
A.3.3 ROS2-Gazebo PD controller gains and constraints 101

List of Figures

1.1 Typical Gait Cycle. . . . 12

2.1 Human stride showing all the gait phases of the right leg. . . . 18

3.1 Joint rotation angles from a typically developed child. . . . 29
3.2 Joint rotation angles from a child with CP. . . . 30
3.3 Anatomical planes of human motion used in gait analysis: sagittal, frontal, and transverse planes. . . . 31
3.4 TD GAIT cycle over 40 steps on the sagittal plane . . . 32
3.5 CP GAIT cycle over 40 steps on the sagittal plane . . . 32
3.6 Sagittal hip (thigh) angle over one normalized stride, together with the four states of the Finite State Machine. . . . 35
3.7 Phase variable s obtained from the hip trajectory of Figure 3.6. The stance and pushoff branch (Equation 3.1) is active up to s_m, after which the preswing and swing branch (Equation 3.2) drives s to 1 at the next touchdown. The two branches are joined at the minimum hip angle, producing a continuous and monotonic phase signal. . . . 36
3.8 Correspondence between the phase variable s and the conventional gait events of the stance and swing phases. . . . 37
3.9 Phase Variable Signal on three individuals over 1 gait cycle (The plots visualize the left and right leg respectively). . . . 37
3.10 Phase Variable Signal on one individual over multiple steps . . . 38
3.11 Neural Network block diagram code. . . . 40
3.12 Architecture of the LSTM model. The timestamp-based and phase variable based variants share this structure and differ only in the number of network inputs. . . . 42
3.13 Architecture of the CNN model. The timestamp-based and phase variable based variants share this structure and differ only in the number of network inputs. . . . 43

4.1 TS-LSTM: one-stride input and one step ahead prediction. . . . 48
4.2 PV-LSTM: one-stride input and one step ahead prediction. . . . 49
4.3 TS-CNN: one-stride input and one step ahead prediction. . . . 49
4.4 PV-CNN: one-stride input and one step ahead prediction. . . . 50
4.5 Ground truth vs. all four models on the same CP segment. . . . 50
4.6 TS-LSTM rolling one-step-ahead rollout over multiple strides. . . . 51

4.7 PV-LSTM rolling one-step-ahead rollout over multiple strides. 52
4.8 TS-CNN rolling one-step-ahead rollout over multiple strides. 52
4.9 PV-CNN rolling one-step-ahead rollout over multiple strides. 53
4.10 Absolute error $|e(t)|$ on the CP segment for all four models. 54
4.11 Residual histograms $(\hat{y} - y)$ for each joint angle and model. 54
4.12 Stride-averaged predicted vs. ground-truth trajectories over the gait cycle, left side, for a held-out CP test subject. Background shading indicates the seven gait phases, anchored to the subject's own measured toe-off. 60
4.13 Stride-averaged predicted vs. ground-truth trajectories over the gait cycle, right side, for the same held-out CP test subject as in Figure 4.12. 60
4.14 Mean absolute error per gait phase, left side, for the held-out CP subject of Figure 4.12. 61
4.15 Mean absolute error per gait phase, right side, for the same held-out CP subject. 62
4.16 Left knee, sagittal plane, stride-averaged prediction vs. ground truth for the held-out CP subject. The ground truth shows a markedly reduced knee flexion/extension range during stance, characteristic of a stiff-knee or crouch-type gait pattern. 62
4.17 Stride-averaged predicted vs. ground-truth trajectories over the gait cycle, left side, for held-out CP subject 52880, the median performer of the 16 test patients. 63
4.18 Mean absolute error per gait phase, left side, for the median CP subject 52880. 64
4.19 Stride-averaged predicted vs. ground-truth trajectories over the gait cycle, left side, for held-out CP subject 42043, the worst case of the 16 test patients. 65
4.20 Mean absolute error per gait phase, left side, for the worst-case CP subject 42043. Even here the recurrent models remain below roughly 2° in every phase. 65
4.21 Stride-averaged predicted vs. ground-truth trajectories over the gait cycle, left side, for a typical-gait subject (part of the training distribution, not held out). 66

5.1 The overall simulation architecture. 78
5.2 Detailed diagram of ROS2 Node used for gait prediction and joint command publishing. 80
5.3 joint_states vs commanded_joint_positions for TS LSTM Model 84
5.4 joint_states vs commanded_joint_positions for TS CNN Model 84
5.5 joint_states vs commanded_joint_positions for PV LSTM Model 85
5.6 joint_states vs commanded_joint_positions for PV CNN Model 85

List of Images

3.1 Motion capture interface and pressure data. 27
3.2 Gait analysis room with embedded force plates. 27
3.3 Control and subject preparation area. 28
3.4 Full view of the motion capture walkway. 28

5.1 ROS2 workspace. 72
5.2 The core concepts of ROS2 Framework. 73
5.3 Simulated model of the exoskeleton in gazebo with view of transformation frames.. 75
5.4 Screenshot showing feet of the Gazebo model and foot contacts on Entity Tree. 77
5.5 Gazebo - PlotJuggler Hips Visualization 82
5.6 Gazebo - PlotJuggler Knees Visualization 82
5.7 Gazebo - PlotJuggler Ankles Visualization 83

List of Tables

2.1 The two gait events that delimit the gait cycle and separate stance from swing. 17
2.2 The seven phases of the gait cycle, grouped by stance and swing period. . . 17
2.3 Representative gait-prediction and phase-variable studies compared with the present work. 20

3.1 Hyperparameters of the four gait prediction models. Within each architecture the timestamp-based and phase-variable-based variants use identical settings, differing only in the width of the input layer, which gains two channels for the phase-variable models. The Adam optimizer's initial learning rate differs between backbones, as selected by the Optuna search described above. 41

4.1 Per-joint prediction error (degrees) for all models evaluated on CP test data. 47
4.2 Performance across all 16 held-out CP patients, averaged over the 18 joint–plane channels. Values are mean ± standard deviation across subjects. . . 56
4.3 Coefficient of determination per held-out CP patient, averaged over all 18 joint–plane channels. 57
4.4 Mean absolute error in degrees by anatomical plane, averaged over the 16 held-out CP patients, with the mean range of motion of the channels in each plane for reference. Relative to the available range of motion, the transverse plane is the least accurately predicted for every model. 68
4.5 The two properties are driven by different design choices. Recurrence (LSTM) is decisive for accuracy on unseen patients; phase-variable (PV) conditioning is decisive for stability under recursive execution, for either backbone. PV-LSTM is the only model strong on both. 69

5.1 Specifications of the simulated exoskeleton, including the base backpack unit and lower-limb links from the *March 6 Exoskeleton Simulated Model*. The Ixx, Iyy , Izz, are the moment of inertia tensor's diagonal values. 74
5.2 PD Controller Gains Used in ROS2-Gazebo Controller. For the multi-degree-of-freedom hip and ankle joints (Section 5.2.2), the same gains are applied independently to each of the sagittal, frontal, and transverse revolute axes. 79

Chapter 1

Introduction

1.1 Rehabilitation Robotics and Machine Learning

Rehabilitation robotics is a field focused on employing robotic technologies to improve the healing process of an individual. This discipline involves the design and development of systems that assist motor and sensory functions, including the movement of arms, hands, legs, and ankles. Major research areas cover the creation of robotic devices for therapeutic training, individualized assistance protocols, and objective assessment of patient capabilities. In clinical practice, robots are used mainly as aids in rehabilitation therapy, not as general assistive technologies. Clinical studies show that robotic-assisted rehabilitation is well-tolerated and can substantially enhance treatment outcomes, particularly for those recovering from motor deficits caused by conditions such as stroke [1].

This rehabilitation process represents an emerging approach for supporting individuals with joint impairments, particularly those with neurological disorders. Recently, flexible exoskeletons have been developed that are particularly suitable for children with cerebral palsy. These designs emphasize comfort and versatility to address the specific anatomical and therapeutic requirements of pediatric patients. To address gait anomalies in children with cerebral palsy, such as knee valgus and foot inversion, lower limb exoskeletons have been proposed [2]. By adapting mechanical support to the user's natural movement patterns, these advancements aim to facilitate more effective rehabilitation [3].

The integration of machine learning has enhanced the responsiveness and adaptability of exoskeleton control systems. Machine learning algorithms enable robotic devices to anticipate and adjust to users' intended actions by processing complex data in real time. This predictive capability provides a more natural and coordinated interaction between the user and the machine, which is particularly advantageous for managing the unpredictable and delayed gait patterns often associated with neurological disorders. However, ensuring effective utilization of these systems across diverse patient populations and real-world scenarios still remains a significant challenge. Ongoing research aims to develop rehabilitation programs that are more reliable, flexible, and user-friendly.

Figure 1.1 shows movements of the lower limbs during walking as a sequence. The cycle is delimited by two instantaneous events and subdivided into seven intervals, or phases. It begins with the event of initial contact, when the right foot first touches the

ground, and ends at the next initial contact of the same foot. The interval up to the toe-off event forms the stance period, roughly 60% of the cycle, during which the limb supports body weight and provides stability and forward progression; it comprises the loading response, mid stance, terminal stance, and pre-swing phases. From toe-off onward the limb advances through the air in preparation for the next step, forming the swing period, which accounts for the remaining 40% and comprises the initial swing, mid swing, and terminal swing phases. The timeline at the bottom of the figure highlights these events and the percentage of the gait cycle completed. This demonstrates the smooth, coordinated nature of normal walking. The distinction between events and phases, together with a full description of each phase, is given in Section 2.1.1.

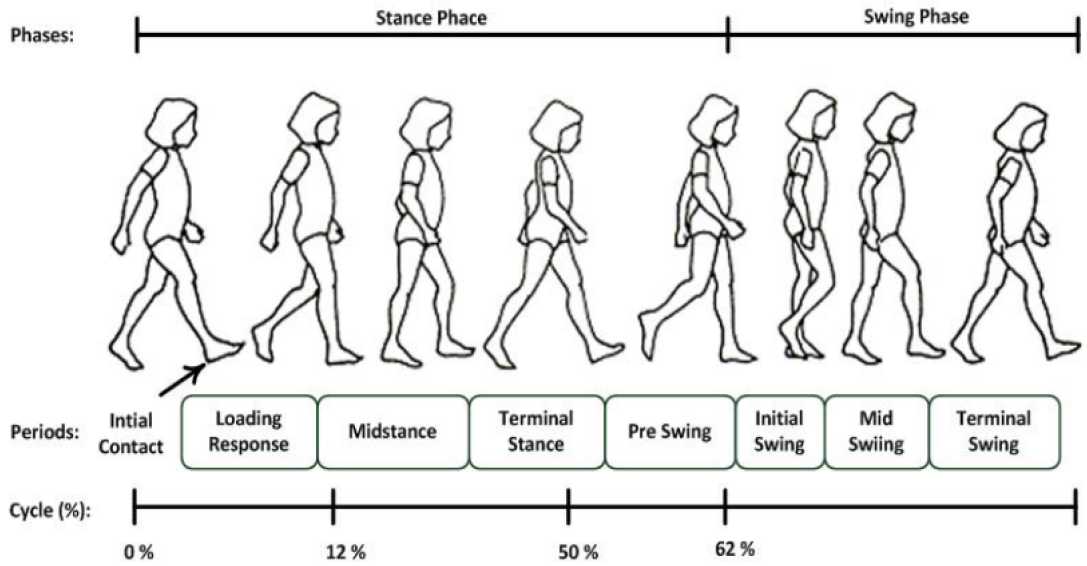


Figure 1.1. *Typical Gait Cycle.*

Trajectory prediction is one of the main uses of machine learning in this area. Neural networks can forecast future lower limb positions or velocities by learning from time series sensor data, such as joint angles and EMG signals. Because of its predictive capacity, a robotic device may move in unison with the operator, anticipating motion instead of just responding to it. For those with neurological conditions like cerebral palsy, whose gait may be erratic, delayed, or challenging to simulate with conventional techniques, this kind of synchronization is extremely important. Additionally, machine learning makes it possible for control systems to reconfigure over time. As users improve or change their gait due to growth, rehabilitation progress, or fatigue, an ML-based model can continually update its understanding of the user's movement intentions. Recurrent Neural Networks (RNNs) and Long-Short-Term Memory (LSTM) networks, for example, are particularly well-suited for handling sequential data and modeling the temporal dependencies in human gait.

An additional advantage of ML is its capacity for personalisation. When machine learning models are trained on user-specific data, controllers can be tailored to an individual's distinct biomechanical characteristics instead of relying on a generic, one-size-

fits-all control strategy. This not only improves the quality of assistance but also boosts user comfort and adherence to therapy. Beyond prediction, machine learning can also support classification tasks, such as grading the severity of motor impairment, identifying gait phases, or estimating levels of user fatigue. These outputs can then guide higher-level control decisions, including when to assist, how much torque to deliver, or which gait pattern better suits the user.

Machine learning turns rehabilitation robots from immobile, homogeneous devices into dynamic, adaptable therapeutic partners by integrating intelligence into robotic control systems. This method bridges the gap between the capabilities of robotic technology and present therapeutic needs, opening the door to more efficient, interesting, and customized rehabilitation.

1.2 Problem Statement

Despite considerable progress in both gait prediction and exoskeleton design, three obstacles continue to limit the clinical usefulness of learning-based assistive control, and together they define the problem addressed by this thesis.

The first is population mismatch. Most predictive gait models are trained on data from typically developed individuals and degrade when applied to people with neurological impairments, whose gait exhibits reduced ranges of motion, asymmetry, and irregular phase timing. The second is the reactive nature of conventional controllers: rather than anticipating the user's intended motion, they respond to measured joint angles or interaction forces only after movement has occurred, so assistance may arrive late or poorly coordinated, and fixed reference trajectories overlook the natural variability that occurs both within and between gait cycles [4]. This is especially limiting in pediatric rehabilitation, where walking characteristics change continuously with growth. The third is that predictive models are usually evaluated offline, in isolation from the control loop they are intended to serve, which leaves open whether a model that scores well on an error metric also behaves stably when its own predictions are fed back over time.

These constraints point toward data-driven, user-adaptive control architectures built on representations that decouple gait progression from elapsed time, and toward evaluating such models not only by prediction error but by their behaviour during execution. Chapter 2 reviews the relevant literature in detail and identifies the specific gaps that this thesis addresses.

1.3 Thesis Objectives and Contributions

This work aims to support clinical decision-making and individualized rehabilitation by developing and comparing neural network models for predicting lower-limb joint angles, with a particular focus on cerebral palsy (CP). The specific contributions are:

1. **A subject-level evaluation protocol on clinical pediatric data.** Models are trained and evaluated on motion capture recordings of both typically developed and CP

children, partitioned at the level of whole subjects rather than individual strides, so that reported performance reflects generalization to previously unseen patients rather than memorization of subject-specific patterns.

2. **A controlled comparison of architecture against temporal representation.** Two architectures, LSTM and CNN, are each studied in a timestamp-based and a phase-variable-based formulation, giving four models that differ along two independent axes and allow the contribution of each to be assessed separately.

3. **Extension from sagittal-plane to all-plane prediction.** All four models are extended from six sagittal joint angles to eighteen channels spanning the sagittal, frontal, and transverse planes of both legs, capturing the out-of-plane deviations that characterize CP gait but that sagittal-only models cannot represent.

4. **A ROS2 and Gazebo framework for executing predictions on a simulated exoskeleton.** Predicted trajectories are published to a position-controlled lower-limb model, allowing execution-level behaviour to be observed directly rather than inferred from offline error metrics alone.

5. **The finding that recurrence and phase-variable conditioning contribute different properties.** Recurrence is shown to drive generalization accuracy to unseen subjects, while phase-variable conditioning drives temporal stability during extended execution, with the phase-variable LSTM the only architecture benefiting from both.

1.4 Thesis Outline

This thesis is structured into the following chapters:

- **Chapter 1** introduces the motivation behind the study, focusing on the challenges in robotic rehabilitation, especially for individuals with cerebral palsy. It also highlights the critical role of accurate gait prediction in this context.
- **Chapter 2** presents essential background concepts, including human gait biomechanics, the distinction between gait events and gait phases, phase variable theory, and the role of machine learning algorithms in motion prediction.
- **Chapter 3** describes the approach for gait prediction, including the data collection process, preprocessing techniques, the construction of the phase variable, and the architecture of the neural network models.
- **Chapter 4** presents the comparative analysis of the four models: error metrics, prediction behaviour, stability under recursive rollout, and generalization to held-out subjects across all three anatomical planes.
- **Chapter 5** describes the ROS2 and Gazebo simulation framework used to execute the predicted trajectories on a simulated lower-limb exoskeleton, giving a view of execution-level behaviour that offline metrics do not capture.

- **Chapter 6** draws conclusions and outlines directions for future work.
- **Appendix A** lists the source code repositories, the simulation launch procedure, and the principal model and control listings.

Chapter 2

Literature Review

The current body of research in biomechanics, the analysis and forecasting of human gait, and the development of lower-limb assistive devices for rehabilitation will be the main topics of this chapter. The current control strategies employed in earlier exoskeleton research are then covered in further detail. These are divided into two categories: those that employ learning-based techniques, such as deep learning (DL), deep reinforcement learning (DRL), and the usage of the Phase Variable (PV), and those that are based on conventional model-based adaptive control approaches.

2.1 GAIT Analysis and Biomechanics

Understanding human mobility is crucial, and clinical gait analysis (CGA) influences assistive technology design as well as diagnostic choices. One study cites a variety of sensing technologies used in contemporary gait analysis, including vision-based markerless systems, pressure-sensitive insoles, and inertial measurement units (IMUs) [5]. Classifying the gait cycle is made easier by these techniques, which allow for continuous and real-time motion recording in clinical and mobile contexts.

2.1.1 Gait Events and Gait Phases

Because the terms are frequently used interchangeably in the literature, the distinction adopted throughout this thesis is stated here explicitly and is used consistently in all subsequent chapters.

A gait *event* is an instantaneous occurrence that can be assigned a single point in time, whereas a gait *phase* is the interval of the cycle bounded by two consecutive events. The gait cycle of one limb is delimited by two events, initial contact and toe-off, which are summarized in Table 2.1. These two events partition the cycle into a stance period, during which the foot is in contact with the ground, and a swing period, during which the limb is advanced through the air. In typical adult gait the stance period occupies approximately the first 60% of the cycle and the swing period the remaining 40%, although this ratio varies substantially in pathological gait and is therefore measured per subject rather than assumed in this work (Section 4.3).

Event	Cycle position	Description
Initial contact (IC), also called heel strike	0% (and 100%)	The instant the foot first touches the ground. Begins the stance period and, by convention, defines the start and end of the gait cycle.
Toe-off (TO), also called foot-off	≈60%	The instant the foot leaves the ground. Ends the stance period and begins the swing period.

Table 2.1. *The two gait events that delimit the gait cycle and separate stance from swing.*

The intervals between these events are conventionally subdivided into seven phases, four within stance and three within swing, following the standard clinical decomposition [5]. These seven phases are listed in Table 2.2 and are the phases used for the phase-resolved error analysis of Section 4.3. It should be noted that some clinical texts additionally count initial contact itself as a brief eighth phase rather than as an instantaneous event; the present work treats it as an event, giving seven phases in total.

Abbr.	Phase	Description
Stance period (initial contact to toe-off)		
LR	Loading Response	The initial double-support interval, during which body weight is transferred onto the leading limb and the knee flexes to absorb impact.
MSt	Mid Stance	Begins when the contralateral limb leaves the ground and the body progresses over the single supporting foot.
TSt	Terminal Stance	The heel rises and the body advances ahead of the supporting foot, until the contralateral limb makes contact.
PSw	Pre-Swing	The second double-support interval, during which the limb is unloaded and prepared for swing. Ends at toe-off.
Swing period (toe-off to next initial contact)		
ISw	Initial Swing	The foot leaves the ground and the limb accelerates forward through knee flexion.
MSw	Mid Swing	The swinging limb passes the stance limb, with the two feet adjacent to one another.
TSw	Terminal Swing	The shank extends and the limb decelerates, positioning the foot for the next initial contact.

Table 2.2. *The seven phases of the gait cycle, grouped by stance and swing period.*

Additionally, Figure 2.1, which graphically depicts human locomotion, displays a typical stride with gait phases. The foot touches the ground during the stance phase, and the leg and foot are raised off the ground and start to swing forward during the swing phase. Humans move normally by sequentially repeating the gait cycle. Although gait analysis is typically the focus of kinematic analysis of the lower extremities in the literature, a wide range of motions can be analyzed, particularly with the development of out-of-laboratory systems [6], [7].

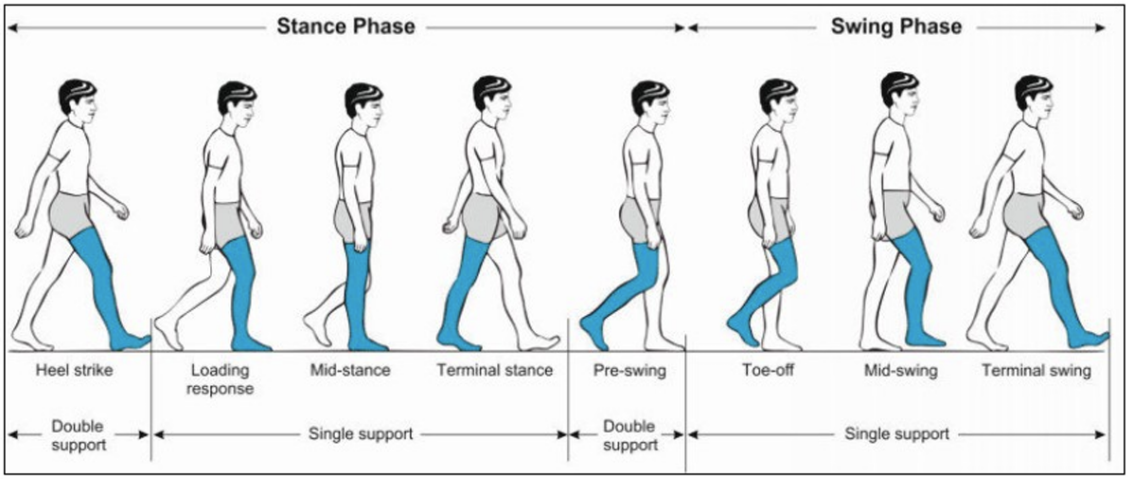


Figure 2.1. *Human stride showing all the gait phases of the right leg.*

Recent advancements in CGA include the use of deep learning for automatic gait phase recognition and the incorporation of robotic gait trainers for neuromotor recovery. These trends align closely with the goals of this thesis, which combines neural network models with real-world joint trajectory data to predict and control human gait. By integrating CGA principles into robotic platforms, researchers improve the ability to assess, replicate, and support human movement with precision. The study reinforces the importance of biomechanics and sensor-driven data in designing intelligent devices that adapt to individual users, especially in populations with neurological disorders.

It should be mentioned that different systems can be used to perform kinematic analysis. Among these systems, opto-electronic systems are classified as the gold standard. One of the most widely used brand is VICON [8]. In such systems, active or passive markers are attached to specific anatomical landmarks on the subject's body, allowing motion to be tracked with precision. However, opto-electronic systems have several limitations: they require controlled lighting conditions, are restricted to indoor laboratory environments, and demand high sampling rates to capture rapid movements accurately.

Recent developments at the intersection of biomechanics and data science have significantly expanded the scope of gait analysis, particularly in clinical applications. Studies argue that although traditional biomechanical methods, like motion capture and musculoskeletal modeling, have improved our understanding of human movement, they fall short in scaling to large, diverse datasets and addressing the variability seen in pathological gait [9]. Complex, noisy, and high-dimensional datasets, which are common in gait biomechanics, can be analyzed through the integration of data science approaches, such as statistical modeling. Crucially, research from institutions like Stanford and Calgary shows how treatment planning can be guided by predictive models derived from clinical and biomechanical data. For instance, they used a model that achieved up to 88% accuracy to successfully predict the results of rectus femoris (quadriceps) transfer surgeries in people with cerebral palsy. These developments demonstrate how data-driven methods can be used to identify hidden patterns in gait data, categorize different kinds of gait abnormalities, and assist in individualized clinical decision-making. There are still issues,

nevertheless, such as the requirement for interdisciplinary cooperation, strong validation methods to avoid overfitting, and shared data infrastructures. Addressing these issues will be essential for translating data science insights into meaningful improvements in patient care.

2.1.2 Biomechanical Differences Between Typical and Pathological Gait

While typically developed gait exhibits quasi-periodic and symmetric joint trajectories, pathological gait (particularly in children with cerebral palsy) is characterized by increased variability, asymmetry, and phase inconsistency. These deviations arise from neuromuscular impairments such as spasticity, muscle weakness, and impaired motor control, resulting in altered joint coordination and timing.

From a machine learning perspective, these characteristics pose significant challenges. Models trained exclusively on typically developed gait data often fail to generalize to pathological populations due to distribution mismatch. Irregular phase progression, reduced range of motion, and abrupt transitions violate the smooth temporal assumptions implicitly learned by many predictive models.

These biomechanical differences underscore the necessity of incorporating pathological gait data during training and motivate the use of phase-variable-based representations, which decouple gait progression from absolute time and enable more robust synchronization under non-ideal walking conditions.

2.2 Neural Networks for GAIT Prediction

Recently, machine learning (ML) has become widespread in research and has been incorporated into a variety of applications, including text mining, image classification, and time-series prediction tasks such as gait trajectory prediction and phase estimation. Two studies are particularly relevant to the present work, and are examined here in some detail because their experimental design directly informs the methodology of Chapter 3.

The first introduces a framework for applying deep learning to predict gait trajectories in children with cerebral palsy, with the goal of improving the control of rehabilitative exoskeletons [10]. Recognizing the limitations of the static, healthy-population reference trajectories commonly used in pediatric exoskeletons, the study trains four architectures, LSTM, Fully Connected Networks (FCN), CNN, and Transformers, on one-step-ahead prediction of hip, knee, and ankle angles for both legs from a 100-step input window, corresponding to approximately one second of motion at 100 Hz. Data were collected using an inertial motion capture system from 10 typically developing and 11 CP children. Notably, the authors employ dynamic time warping (DTW) distance as an early-stopping criterion, prioritizing consistency in multi-step recursive forecasts over minimization of one-step error alone.

Two findings from that study are directly relevant here. First, the LSTM achieved the lowest short-term error (MAE = 0.87°) but degraded most under injected Gaussian noise, whereas the convolutional and attention-based models proved more noise-robust, a

trade-off between short-term accuracy and stability that recurs in the results of Chapter 4. Second, because only TD data were used for training, the models tended to output TD-like characteristics when given CP input, effectively "correcting" pathological patterns toward typical ones. The authors frame this as adaptive reference trajectory generation, but it also illustrates the population-mismatch problem: a model trained only on typical gait reproduces typical gait. The present work instead includes CP subjects in training, precisely so that predictions track each patient's own kinematics rather than regressing toward a typical pattern, a distinction that becomes visible in the subject-specific failure modes examined in Section 4.3.

The second study predicts lower-limb trajectories and discrete gait phases simultaneously using an LSTM, targeting the compensation of system delays in exoskeleton control [11]. The model forecasts angular velocities of the hip, shank, and foot segments and classifies the corresponding gait phase up to 200 ms ahead, using IMU data from 12 healthy adults walking on a treadmill at various speeds, with foot switches providing ground-truth phase labels. A weighted discount loss emphasizes short-horizon accuracy while still accounting for the full prediction window, mitigating error accumulation in recursive prediction. Evaluated both intra-subject and inter-subject via leave-one-subject-out, it achieved correlation coefficients above 0.98 and 0.91 respectively, with swing-phase classification reaching 97% accuracy intra-subject and lower accuracy on transitional phases such as loading response and terminal stance.

The inter-subject protocol of that study is the closest precedent for the subject-level split adopted in Section 3.2.4, and the drop from 0.98 to 0.91 between intra- and inter-subject evaluation illustrates why splitting at the level of samples rather than subjects overstates performance. Its principal limitation for present purposes is population: the study was confined to healthy young adults on a treadmill, leaving open how the approach transfers to pathological pediatric gait.

Table 2.3 summarizes these and other representative studies alongside the present work.

Study	**Architectures**	**Population**	**Planes**	**Evaluation**	**Execution**
[10]	LSTM, FCN, CNN, Transformer	10 TD, 11 CP children	Sagittal	Train on TD only, test on TD + CP	Offline
[11]	LSTM	12 healthy adults	Sagittal	Leave-one-subject-out	Offline
[12], [13]	Phase variable, no learning	Healthy adults, amputees	Sagittal	Hardware trials	Powered prosthesis
[14]	Phase variable (PHI, SFH)	Healthy adults	Sagittal + frontal	Perturbed walking trials	Offline analysis
This work	LSTM, CNN × timestamp, phase variable	TD + CP children (ELEPAP)	Sagittal, frontal, transverse	Subject-level held-out split	ROS2 / Gazebo simulation

Table 2.3. *Representative gait-prediction and phase-variable studies compared with the present work.*

Two gaps are apparent from this comparison. Learning-based studies predict the

sagittal plane only and evaluate offline, while phase-variable studies achieve real-time execution but do not use learned models. The present work occupies the intersection: learned multi-plane prediction, conditioned on a phase variable, evaluated on held-out pathological subjects and executed in simulation.

2.2.1 Comparative Analysis of Neural Network Architectures for Gait Prediction

While a variety of neural network architectures have been proposed for gait prediction, their suitability differs significantly depending on the application context, prediction horizon, and target population. In particular, trade-offs emerge between temporal consistency, computational efficiency, and robustness to pathological gait patterns.

Recurrent architectures such as LSTM networks explicitly model temporal dependencies, making them well-suited for capturing long-term gait dynamics and inter-joint coordination. This characteristic is especially beneficial when modeling irregular or non-periodic gait, as observed in children with cerebral palsy. However, this temporal expressiveness comes at the cost of increased computational complexity and sensitivity to noisy inputs.

In contrast, CNN-based approaches rely on local receptive fields to extract short-term temporal features. Although they lack explicit memory mechanisms, CNNs demonstrate greater robustness to sensor noise and faster inference times, which are desirable properties for embedded real-time systems. Transformer-based architectures further extend modeling capacity by capturing long-range dependencies through attention mechanisms, though their computational requirements currently limit real-time deployment in wearable assistive devices.

These trade-offs motivate the selection of LSTM and CNN models in this thesis, as they represent complementary approaches balancing prediction fidelity and real-time feasibility.

2.3 Phase Variable Signal

As noted earlier, conventional control techniques for powered prosthetic limbs typically divide the gait cycle into distinct phases, including heel strike, mid-stance, and toe-off, with each phase controlled by an individual strategy. Although effective in organized situations, these techniques may turn unreliable when encountering non-rhythmic or dynamically changing human movements. The idea of a Phase Variable (PV) offers a more refined and resilient solution by depicting the development throughout the gait cycle with a smooth, monotonic signal, frequently based on the movement of the residual limb.

A study presented the application of piecewise continuous phase variables for rhythmic and non-rhythmic activities [12]. This method removes the necessity for intricate state machines by specifying target joint trajectories (virtual constraints) as functions of a phase variable inspired by human motion. In contrast to earlier models that frequently depended on time-dependent or manually adjusted transitions, this approach obtains the

phase variable from the remaining hip angle, which inherently captures the user's intent and advancement during a stride. The authors showed that employing the hip angle as a continuous phase variable enhances the synchronization between the prosthetic leg and the human user during steady-state walking and during transitions, such as stopping, starting, or changing direction

Another researcher implemented and tested a fully unified control architecture [13] that continuously parameterized the entire gait cycle by using the hip phase angle, derived from the combination of hip angular position and its integral, to define the phase velocity. This approach allows the control system to maintain synchronization without requiring external gait event detection or state transitions. Importantly, the controller is capable of generalizing across different walking speeds (0.67–1.21 m/s) and inclines (−2.5 to +9 degrees), using the same set of PD control gains for all participants. The powered prosthesis, with its embedded sensors and actuators, utilizes the PV to enforce virtual constraints across both knee and ankle joints.

Key findings from these experiments revealed that the continuous phase variable enabled real-time adaptation of joint trajectories to varying walking speeds and slopes, maintaining stable and biomechanically sound gait patterns. The use of periodic virtual constraints, parameterized using the Discrete Fourier Transform (DFT) of able-bodied gait data, ensured smooth, rhythmic joint trajectories that closely mimicked natural locomotion [13]. Moreover, this method minimized configuration time since it did not require individualized tuning for each user or activity mode.

Another study investigates a significant drawback of current phase variable approaches for gait control: their dependence on velocity data, which hinders real-time control applications [14]. Previous studies, like the hip phase angle method, effectively characterized human gait through phase portraits (e.g., angle against velocity), but these formulations led to relative degree-one systems. These pose difficulties for control as they restrict the application of derivative feedback, increasing system sensitivity to noise and decreasing control robustness. Villarreal and Gregg tackle this by suggesting different phase variables of relative degree two, based only on positional states (such as joint angles and their integrals). These consist of the Phase Hip Integral (PHI) and the Sagittal/Frontal Hip (SFH) phase parameters. The PHI approach substitutes velocity with the time integral of the hip angle to generate a smooth and monotonic phase trajectory that facilitates proportional-derivative (PD) control, while encompassing the entire gait cycle, covering both stance and swing phases. The SFH approach integrates sagittal and frontal hip movements by employing principal component analysis (PCA) to obtain orthogonal components, though this results in increased dimensional sensing and possible inconsistencies specific to individuals.

Experimental assessments with altered walking trials indicated that both PHI and SFH surpass time-based parameterizations in maintaining joint kinematic consistency amid disturbances. Significantly, although the original Phase Hip Velocity (PHV) approach yielded the highest correlation between perturbed and unperturbed movements, it was also more vulnerable to noise and not compatible with derivative control. This study highlights the significance of selecting phase variables, as it affects both precise gait rep-

resentation and the practical control usability in powered prosthetics and exoskeletons, where minimizing sensor noise and simplifying the controller are essential.

2.4 Control for Wearable Robotics

Several studies have made substantial contributions to feedback control strategies in powered lower-limb devices. One foundational research effort [15], [16] developed a hybrid zero dynamics (HZD) framework for stabilizing dynamic walking in paraplegic users using exoskeletons with minimal sensory feedback and no external supports. This methodology leverages the theory of virtual constraints, where joint trajectories are represented as holonomic constraints parameterized by a phase variable that progresses monotonically through the gait cycle.

The desired output trajectories $y(t)$ are encoded as functions of a phase variable s, often selected as a monotonic function of hip position. The virtual constraints take the general form:

$$y(q) = h_0(q) - h_d(s) \tag{2.1}$$

where $h_0(q)$ is the actual output and $h_d(s)$ is the desired output trajectory parameterized by phase s. A feedback control law then regulates this output such that $y(q) \to 0$, driving the system toward stable periodic walking.

These approaches are further supported by Lyapunov-based stability analysis and hybrid event-triggered control mechanisms, which reset the phase variable at each step based on discrete foot contact events. The use of hybrid systems theory is essential due to the discontinuous nature of walking dynamics, where foot-ground interactions introduce sudden transitions in system dynamics. These feedback-driven designs have been shown to enable robust, hands-free walking for paraplegic subjects using powered exoskeletons [15], [16].

Another critical research advancement involves real-time human-in-the-loop optimization. Rather than relying exclusively on pre-programmed trajectories, these methods iteratively adjust assistive torque parameters during gait based on measurable improvements in user comfort or energy efficiency. The core idea is to model the relationship between assistance parameters ∂ and a human response cost function $J(\partial)$, such as metabolic energy consumption.

This function is optimized iteratively via:

$$\partial_{k+1} = \partial_k - a\nabla J(\partial_k) \tag{2.2}$$

where a is a step size and ∇J is estimated through experiments, such as through Bayesian optimization or gradient descent based on feedback from sensors.

Studies employing this method have demonstrated significant reductions in metabolic cost (up to 24%) for exoskeleton-assisted walking, while preserving natural gait patterns. This paradigm is particularly effective in situations where user-specific variability renders open-loop strategies suboptimal [17].

A comprehensive review of control strategies has categorized exoskeleton control into hierarchical layers: high-level intention estimation, mid-level gait-phase-dependent modulation, and low-level actuation control. High-level controllers often rely on gait phase estimation or user intention prediction, using sensor inputs like joint angles, ground reaction forces, or electromyography (EMG). These inputs can feed into classifiers, rule-based detectors, or probabilistic estimators such as Hidden Markov Models (HMMs) to determine phase transitions.

The mid-level control layer is typically modeled using impedance-based or trajectory-based methods. Impedance control laws are defined as:

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) \tag{2.3}$$

where K_p and K_d are proportional and derivative gains, and q_d and $\dot{q}_d$ are desired joint positions and velocities. These controllers allow for compliant interaction between the user and the device [18].

In contrast, trajectory tracking controllers replay pre-recorded joint profiles using feedforward and feedback torque controllers, often segmented by finite state machines (FSMs). These are simple and effective but lack adaptability. At the low level, torque control or position control loops are implemented, often using PID or PD controllers tuned for stability and responsiveness.

2.5 Limitations of Existing Literature

Despite substantial progress in gait prediction and exoskeleton control, several limitations persist in the existing body of literature. First, many predictive models are trained and validated primarily on typically developed adult populations, limiting their applicability to pediatric or pathological populations. Second, a large portion of proposed methods are evaluated offline, without integration into real-time control architectures. This omission obscures practical concerns such as latency, stability, and safety, which are critical in wearable assistive systems. Furthermore, few studies provide open-source implementations or reproducible pipelines, hindering direct comparison and real-world adoption. Safety mechanisms, fallback strategies, and constraint handling are often underreported, despite their importance in clinical applications.

These gaps motivate the integrated learning-and-control framework proposed in this thesis, which emphasizes pathological gait modeling, real-time deployment, and safety aware control design.

2.6 Chapter Summary

This chapter reviewed key literature in the domains of biomechanics, human gait prediction, and exoskeleton control systems. It began by highlighting the principles of clinical gait analysis (CGA), its phases, and advancements in sensing technologies and data-driven models that support kinematic analysis, especially for rehabilitation. The

integration of machine learning, particularly deep neural networks like LSTM and CNN, has enabled the prediction of joint trajectories and phase detection from gait data. Studies demonstrate that such models not only offer robust predictions but can also adapt pathological gait patterns toward more natural trajectories, potentially transforming rehabilitation practices.

The concept of phase variables was introduced as a continuous alternative to traditional discrete state machines, enabling more fluid and responsive control of prostheses and exoskeletons. Various formulations, such as hip angle-based phase variables and their integral-based counterparts, offer different trade-offs in terms of control robustness and sensor requirements. Finally, the chapter examined different exoskeleton control models, from predefined trajectory tracking to adaptive, intention-driven systems. Studies in this domain underline the importance of real-time feedback, biomechanical synergy, and user-specific adaptability.

While recent advances in gait biomechanics, predictive modeling, and exoskeleton control have yielded promising results, several challenges remain. Most predictive models focus on typical gait patterns instead of pathological patients, such as those from children with cerebral palsy. Furthermore, few studies fully integrate neural network-based prediction with real-time control architectures using open-source platforms like ROS2. This thesis addresses these gaps by developing and comparing LSTM and CNN architectures for multi-joint gait prediction using real-world pediatric datasets. It further contributes a control framework that uses phase variable tuning and publishes joint trajectories to a lower-limb exoskeleton visualized in Gazebo-ROS2. The approach aims to enhance real-time adaptability and smooth motion control in the presence of both typical and pathological gait patterns.

Chapter 3

Neural Network Approach to Predict GAIT

This chapter presents the methodology and implementation of a deep learning approach to predict human gait patterns. The aim is to develop a predictive model capable of anticipating joint trajectories and gait phases, which can be applied in real-time to support the control of assistive devices such as prosthetics and exoskeletons.

3.1 Lower-Limb Data

3.1.1 Data Provider

An essential aspect of this task is the dataset employed for training and assessment. The gait data utilized was provided by ELEPAP [19], the Hellenic Society for the Protection and Rehabilitation of Disabled Individuals. ELEPAP is Greece's first charitable organization focused on rehabilitating children with physical disabilities and developmental disorders. With its extensive experience and dedication to enhancing the quality of life for children with mobility challenges, ELEPAP upholds a top-notch motion capture system and clinical gait analysis laboratory, from which practical data have been collected. This dataset consists of kinematics and kinetics of lower limb joints, along with ground force sensors (GFS) from pediatric patients, rendering it especially useful for developing machine learning models to help children with gait disorders. The gait and movement information utilized in this research was recorded at the Gait & Motion Analysis Lab of ELEPAP, depicted in images 3.1 – 3.4. The laboratory features a precise optical motion capture (Mocap) system, force plates built into the floor, and coordinated data collection software. The existence of several calibrated cameras facilitates precise three dimensional reconstruction of joint paths during walking experiments. Force plates offer additional measurements of ground reaction forces, crucial for biomechanical assessment

This state-of-the-art setup ensures the collection of high-fidelity, temporally aligned kinematic and kinetic data under the supervision of clinical experts. The well-lit, unobstructed walking space, combined with real-time feedback systems, facilitates reliable and repeatable trials for both typically developing children and those with gait impairments. The professionalism and controlled environment at ELEPAP's lab greatly contributed to the reliability and clinical relevance of the dataset used to train and validate the predictive models in this thesis.



Image 3.1: *Motion capture interface and pressure data.*



Image 3.2: *Gait analysis room with embedded force plates.*



Image 3.3: *Control and subject preparation area.*



Image 3.4: *Full view of the motion capture walkway.*

The gait data consisted of multiple individual files, each representing a single gait cycle (stride) recorded from a different patient. Each file included hip, knee, and ankle joint

angles (sagittal, frontal and transverse planes) for typically developed children as well as for children with cerebral palsy, as shown in Figure 3.1 and Figure 3.2 respectively, where the red line represents the left leg and the blue dotted line the right leg (both legs are standardized to be on the same GAIT phases to compare the asymmetries of the stride). A clinical motion capture device was used to record these joint rotation angles, and each step was standardized to ensure uniformity between samples. Each file was initially imported using a data pipeline, which parsed the joint angle values and organized them into fixed-length sequences suitable for model input in order to get the data ready for machine learning training.

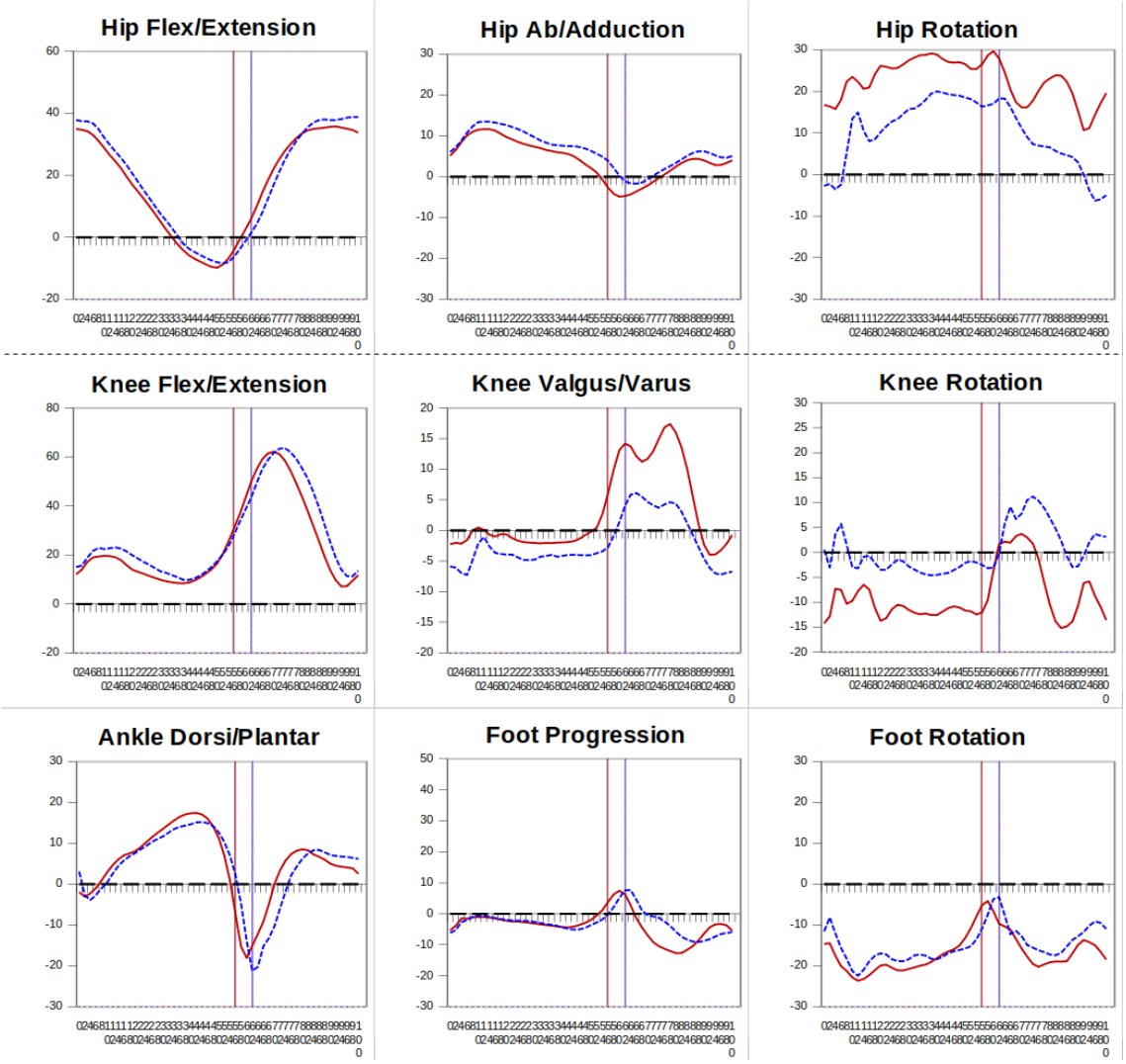


Figure 3.1. *Joint rotation angles from a typically developed child.*
Red line represents left leg angles and blue line right leg angles (The vertical lines show each leg's start of swing phase)

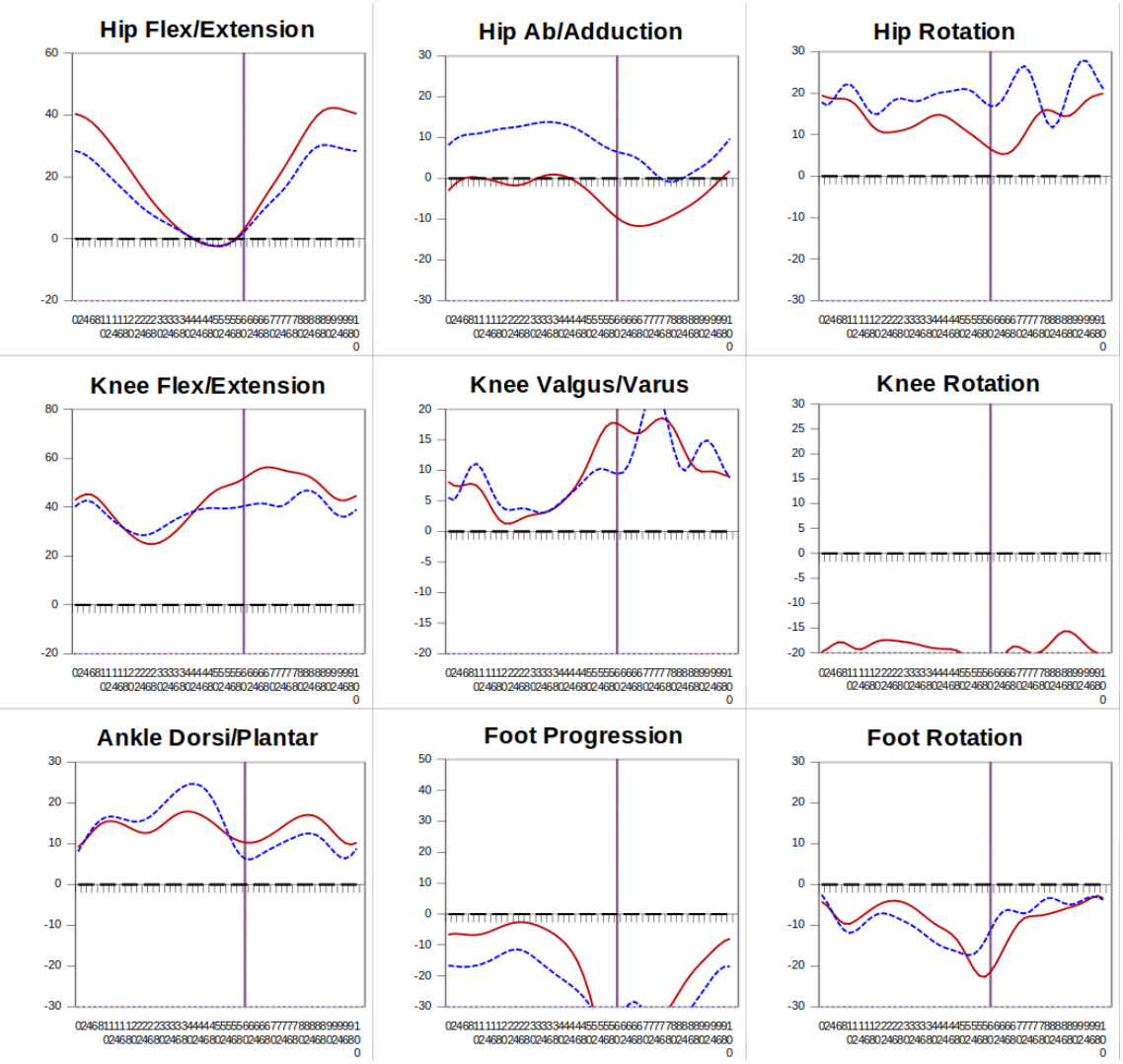


Figure 3.2. *Joint rotation angles from a child with CP.*
Red line represents left leg angles and blue line right leg angles (The vertical lines show each leg's start of swing phase)

GAIT data included the joint angles for the sagittal, frontal, and transverse planes, as illustrated in Figure 3.3. The sagittal plane describes forward and backward motion and includes hip flexion–extension, knee flexion–extension, ankle plantar–dorsiflexion, and pelvic tilt. The frontal plane captures side-to-side motion such as hip abduction–adduction and pelvic obliquity, while the transverse plane represents rotational movements including hip and knee rotation and foot progression angle.

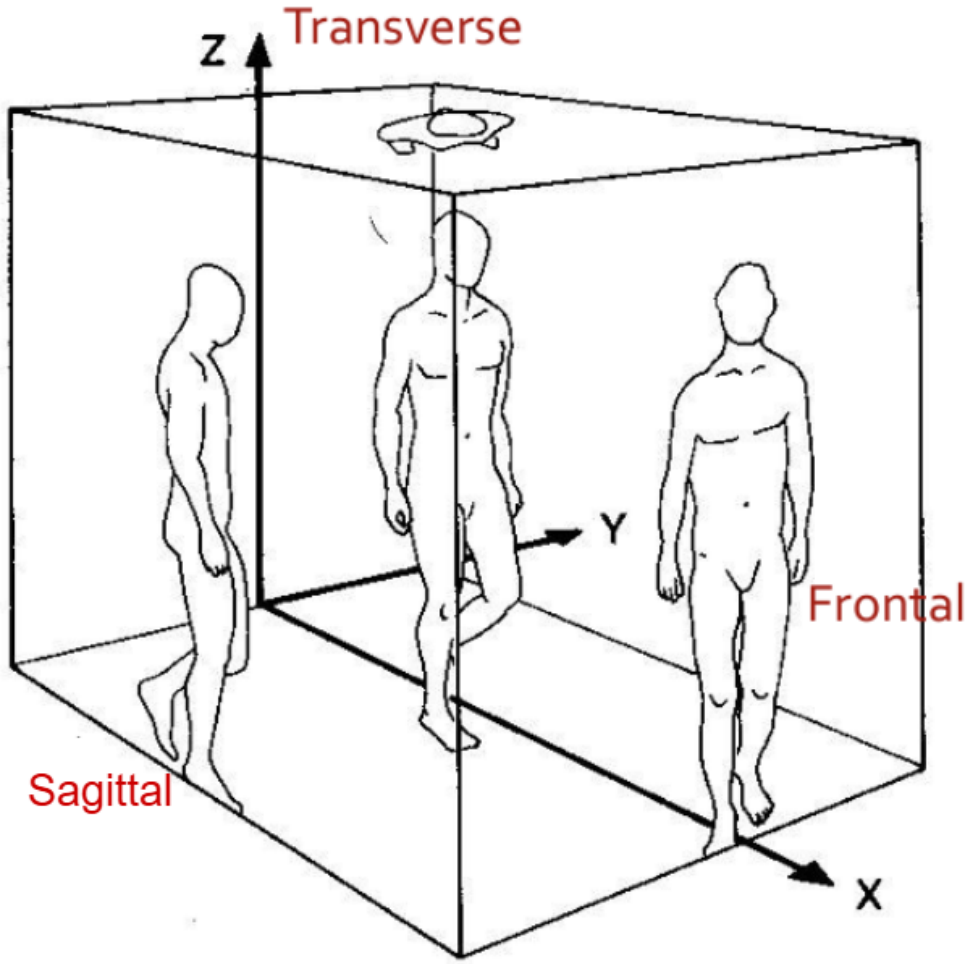


Figure 3.3. *Anatomical planes of human motion used in gait analysis: sagittal, frontal, and transverse planes.*

3.1.2 Differences between Typically Developed and Cerebral Palsy Data

Cerebral palsy (CP) is a group of permanent movement and posture disorders caused by damage to the developing brain, typically occurring before or during birth. It often results in abnormal muscle tone, poor coordination, spasticity, and involuntary movements, all of which significantly affect gait. In contrast, typically developed (TD) children exhibit regular, symmetric, and cyclic gait patterns with coordinated joint trajectories and smooth transitions between phases of the gait cycle, as shown in Figure 3.4.

The gait data obtained from children with CP display substantial deviations from typical patterns. These include reduced joint ranges of motion, especially at the knee and ankle, delayed or irregular gait phases (e.g., toe-walking or crouch gait), and asymmetries between the left and right limbs, as shown in Figure 3.5. For instance, children with spastic diplegia often exhibit excessive hip and knee flexion, as well as decreased dorsiflexion at the ankle. These deviations are clearly visible in sagittal plots, which show more erratic and compressed curves compared to the smoother, sinusoidal trajectories observed in TD subjects.

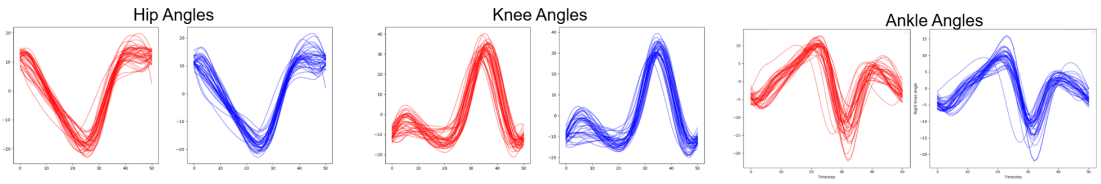


Figure 3.4. *TD GAIT cycle over 40 steps on the sagittal plane*
(Red: left joint angles and Blue: right joint angles)

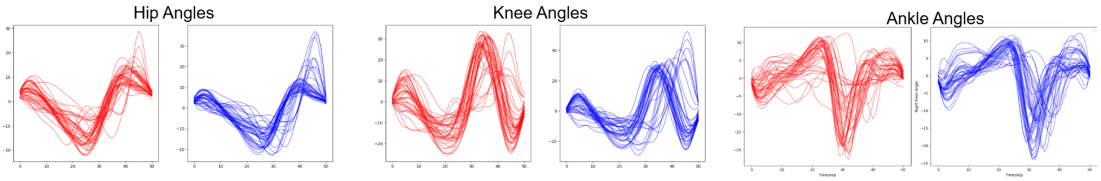


Figure 3.5. *CP GAIT cycle over 40 steps on the sagittal plane*
(Red: left joint angles and Blue: right joint angles)

Understanding these biomechanical differences is crucial for developing robust machine learning models. A model trained only on TD data may fail to generalize to pathological gait unless it is specifically designed or adapted to account for the variability and asymmetry seen in CP. By incorporating both datasets, the model can learn to distinguish between typical and atypical patterns, allowing for more personalized and clinically useful gait predictions.

3.2 Data Preprocessing

Before the gait data could be used for training the neural network models, several preprocessing steps were required to ensure data consistency, numerical stability, and biomechanical validity. Since the available datasets included both typically developed (TD) and cerebral palsy (CP) gait, special care was taken to handle differences in sample size, noise levels, and gait variability.

3.2.1 Stride Segmentation and Dataset Augmentation

The raw gait data provided by ELEPAP consisted of joint angle measurements for the hip, knee, and ankle joints of both the left and right legs, along with foot contact information. Each data file represented a complete gait cycle. A stride-based segmentation approach was therefore used, where each sample corresponded to one full stride.

To ensure uniform input size across all samples, each stride was resampled to a fixed length of 51 time steps. This allowed all gait cycles to be represented in a consistent format, which is required for sequence-based neural networks such as LSTMs and CNNs. Representing gait at the stride level also made it easier to compare typical and pathological gait patterns, since each sample corresponded to a complete biomechanical unit of walking.

A major challenge was the limited size of the typically developed dataset. Only 60 TD strides were available, compared to more than 400 strides from children with cerebral

palsy. Since deep learning models generally require large amounts of data to generalize well, data augmentation was applied to the TD dataset.

Augmentation was performed by introducing small, controlled variations to the original joint angle trajectories. These included noise injection to simulate sensor noise, amplitude scaling to represent inter-subject differences in joint range of motion, and mild temporal warping to account for small variations in gait timing. These transformations were kept within realistic limits to preserve the biomechanical characteristics of normal gait.

Using these techniques, the TD dataset was expanded from 60 to 200 strides. Augmentation was not applied to the CP dataset, as preserving the original pathological characteristics of the gait patterns was considered more important than increasing sample size. Augmentation was applied only to the training portion of the data, after the subject-level split described in Section 3.2.4, so that no augmented copy of a held-out subject's stride could appear in the validation or test sets.

3.2.2 Signal Smoothing and Noise Reduction

The raw joint angle signals contained measurement noise originating from the motion capture system and marker movement. To reduce this noise, a filtering technique was applied to all joint angle trajectories. A low-pass Butterworth filter was used to remove high-frequency noise while preserving the main gait dynamics.

3.2.3 Normalization and Data Cleaning

To improve numerical stability during training, all input features were normalized using z-score normalization. A `StandardScaler` [20] was fitted on the training data and then applied to the validation and test sets to avoid data leakage.

Missing values in the dataset were filled where necessary, and samples showing large divergence between the first and last points of a stride were corrected by averaging the endpoint values. This step helped ensure smooth transitions within each stride and prevented artificial discontinuities from negatively affecting the learning process.

3.2.4 Formatting and Dataset Splitting

Finally, the processed data were reshaped into three-dimensional tensors with dimensions corresponding to the number of strides, time steps, and features. This format is required for temporal neural networks, allowing the models to learn both temporal dependencies within a stride and correlations between different joint angles.

The dataset was then divided into training, validation, and test sets at the *subject* level rather than the sample level. Each patient's several trial files were grouped under a common subject identifier parsed from the filename, and whole subjects (not individual strides) were partitioned into training, validation, and test sets using a 70%/15%/15% split with a fixed random seed. This ensured that every stride belonging to a held-out subject was entirely unseen during training, so that the reported test performance reflects

generalization to previously unseen patients rather than memorization of subject-specific patterns. Both typically developed and cerebral palsy subjects were present in all three splits, so that the models were trained on, and evaluated against, both populations rather than being trained on typical gait alone. Input-output pairs were then constructed within each subject's own trials for supervised learning, so that no rolling window ever crossed a subject boundary, enabling both short-term and multi-step gait prediction tasks. This subject-level protocol is used consistently throughout the thesis; the extended evaluation of Section 4.3 applies the same split to the multi-plane models introduced there.

Overall, this preprocessing pipeline ensured that the gait data were suitable for deep learning while preserving the essential biomechanical characteristics required for meaningful gait prediction.

3.3 Phase Variable Approach

The phase variable is a key feature which allows robust and natural synchronization between a powered robotic device and the user's stride [13]. Unlike traditional controllers that rely on time-based cycles or EMG signals, this method derives the phase variable directly from sensors, specifically the hip angle on the sagittal plane and foot contact, which are more reliable across a range of activities and users.

The phase variable s is constructed as a function of the hip angle q_h, segmented across four intervals governed by a Finite State Machine (FSM):

- **S1: Stance**
- **S2: Pushoff**
- **S3: Preswing**
- **S4: Swing**

These four states are *controller* states, defined by the shape of the hip trajectory, and should not be confused with the seven clinical gait phases of Section 2.1.1. They are coarser, and their boundaries are determined by hip-angle landmarks rather than by the clinical event definitions. The clinical phases are used in this thesis only for reporting error as a function of position in the gait cycle (Section 4.3); the controller states are used only to construct s.

Within each state, s is computed such that it increases continuously from 0 to 1 over the entire gait cycle. Since the hip angle is not globally monotonic, each segment is defined to ensure local monotonicity.

The state transitions are driven by the hip angle itself together with foot contact. Figure 3.6 shows the sagittal hip (thigh) angle over a single normalized stride, with the four FSM states shaded and the transition instants marked. The stride begins at touchdown with the hip angle at q_h^0, after which the thigh extends monotonically through stance (S1) until it crosses the pushoff threshold q_{po}, at which point the controller switches to pushoff (S2). The hip reaches its minimum $q_h^{\min}$ at the S2→S3 transition, marking the reversal of

thigh motion, and the leg then progresses through preswing (S3) and swing (S4) while the hip flexes back towards q_h^0 at the next touchdown. This decomposition is precisely what makes the piecewise definition of s necessary: the hip angle traverses the same range twice per stride, once decreasing and once increasing, so a single global mapping from q_h to s would be ambiguous.

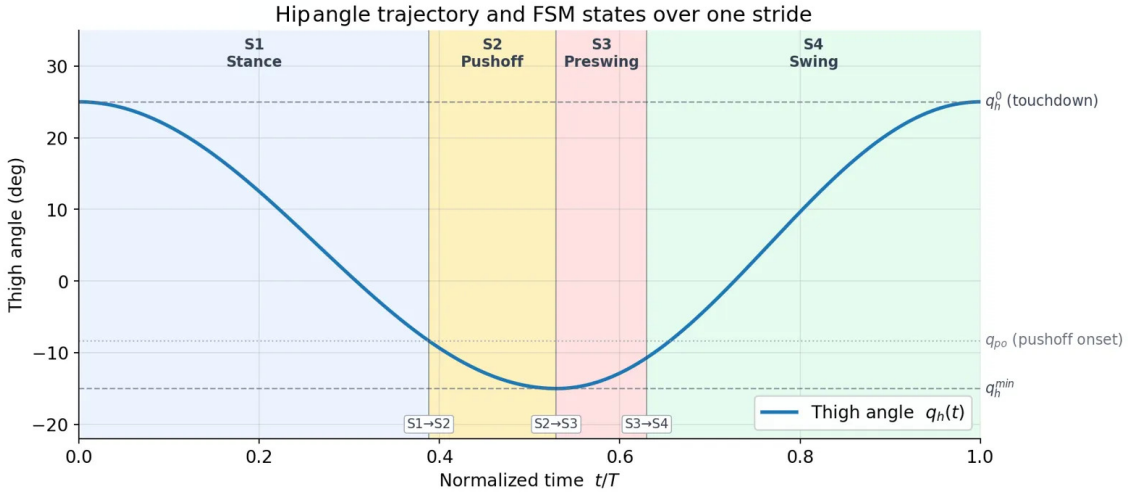


Figure 3.6. *Sagittal hip (thigh) angle over one normalized stride, together with the four states of the Finite State Machine.*

During the initial stance and pushoff phases (States S1 and S2), the phase variable is defined as shown in Equation 3.1.

$$s = \frac{q_h^0 - q_h}{q_h^0 - q_h^{\min}} \cdot c \tag{3.1}$$

Where q_h^0 is the hip angle at touchdown (start of the stride), $q_h^{\min}$ is the minimum hip angle during stance, q_h is the hip angle at that time, and c is a fixed scaling constant. Following [13], c is chosen empirically as $c = 0.53$ and is held constant: the same value is applied to every stride and every subject, so that the phase variable is constructed identically for all patients regardless of their individual stance timing. The value 0.53 reflects the fact that the hip angle reaches its minimum slightly after the midpoint of a typical stride.

During preswing and swing (States S3 and S4), the phase variable is updated as shown in Equation 3.2:

$$s = 1 + \frac{1 - s_m}{q_h^0 - q_{h,m}} \cdot (q_h - q_h^0) \tag{3.2}$$

Where s_m is the phase value at the transition from S2 to S3, and $q_{h,m}$ is the hip angle at the same transition point.

The effect of applying Equations 3.1 and 3.2 to the hip trajectory of Figure 3.6 is shown in Figure 3.7. The first branch maps the descending portion of the hip angle onto the interval $[0, s_m]$, where $s_m = c = 0.53$ is reached at the instant of minimum hip angle, since

substituting $q_h = q_h^{\min}$ into Equation 3.1 gives exactly c. The second branch takes over from that point and maps the ascending portion onto $[s_m, 1]$, so that s returns exactly to 1 at the next touchdown. The two branches meet with matching values at s_m, and the resulting signal is continuous and monotonically increasing across the whole stride even though the underlying hip angle is not.

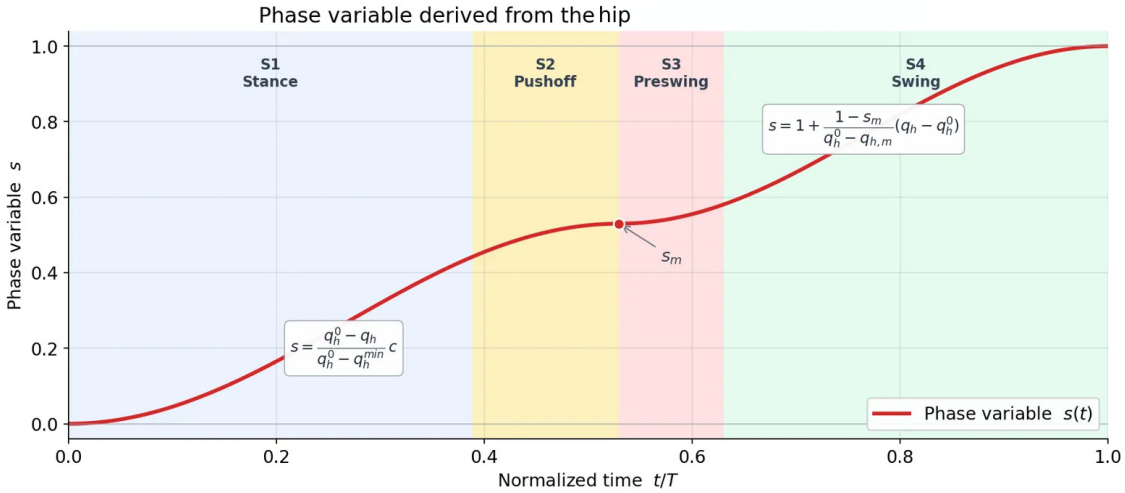


Figure 3.7. *Phase variable s obtained from the hip trajectory of Figure 3.6. The stance and pushoff branch (Equation 3.1) is active up to s_m, after which the preswing and swing branch (Equation 3.2) drives s to 1 at the next touchdown. The two branches are joined at the minimum hip angle, producing a continuous and monotonic phase signal.*

Because s is monotonic, every value in $[0, 1]$ corresponds to a unique instant of the stride, which allows the phase variable to be interpreted directly as a normalized gait timeline. Figure 3.8 maps characteristic values of s onto the gait events and phases defined in Section 2.1.1. The two bounding *events* occur at fixed values of s: initial contact at $s = 0$, and toe-off at approximately $s = 0.57$, with the next initial contact closing the cycle at $s = 1$. The intervening *phases* occupy the intervals between them: loading response and mid stance span the first half of the cycle, terminal stance begins at approximately $s = 0.46$, and pre-swing begins at $s = s_m = 0.53$ and runs until toe-off. The swing phases follow, with mid swing beginning near $s = 0.74$ and terminal swing closing the cycle. This correspondence is what makes the phase variable a biomechanically meaningful input feature: a given value of s carries the same physical interpretation regardless of stride duration or walking speed.

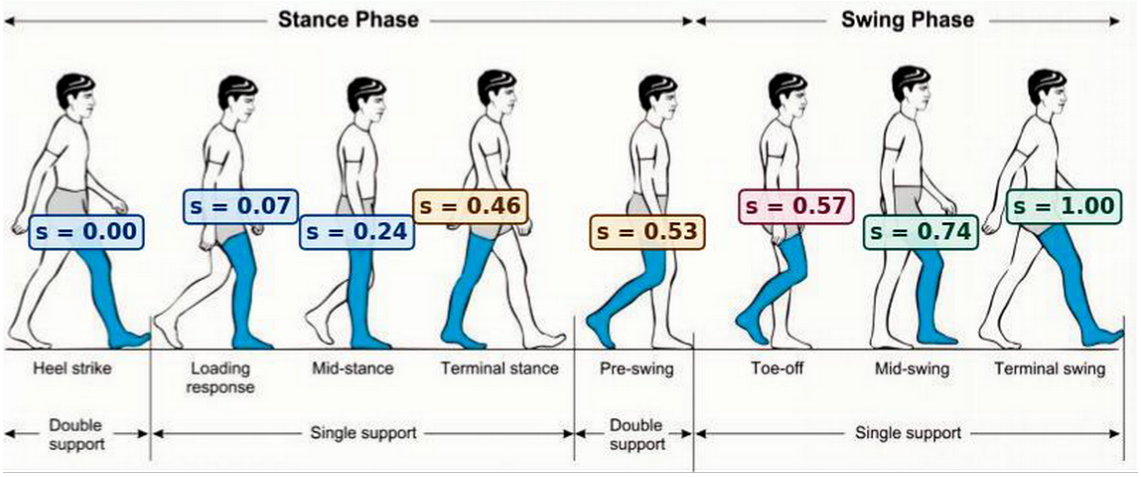


Figure 3.8. *Correspondence between the phase variable s and the conventional gait events of the stance and swing phases.*

The same behaviour is observed when the formulation is applied to the measured data of this work, as shown in Figure 3.9 for a single stride and in Figure 3.10 over consecutive strides. In both cases the phase variable increases smoothly through the stride, despite the hip angle reversing direction mid-gait. The phase variable provides a continuous, monotonic signal that directly reflects the progression of the gait cycle. The *virtual constraints* (predefined joint trajectories) are generated using this signal as a parameter.

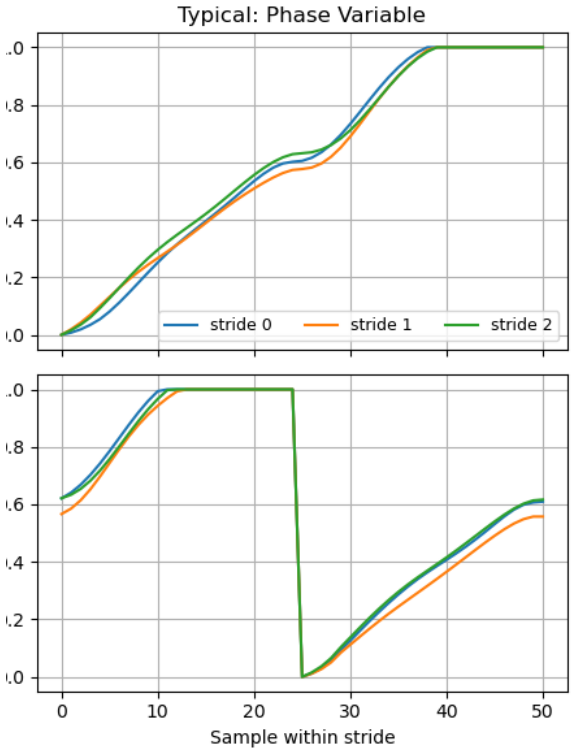


Figure 3.9. *Phase Variable Signal on three individuals over 1 gait cycle (The plots visualize the left and right leg respectively).*

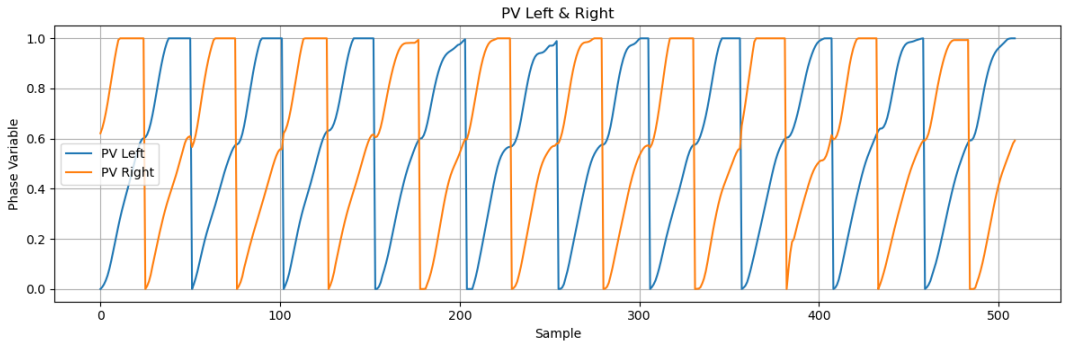


Figure 3.10. *Phase Variable Signal on one individual over multiple steps*

The proposed phase variable method offers several key advantages. First, it enables real-time gait synchronization, allowing joint trajectories to naturally align with the user's intent without relying on external triggers or timing mechanisms. This continuous coordination enhances the intuitiveness and responsiveness of the system. Second, the approach is highly adaptable to non-rhythmic behaviors, making it suitable for scenarios involving irregular gait patterns and different walking speeds. Moreover, by closely tracking biomechanical cues, the system reduces the occurrence of compensatory motions such as vaulting or hip hiking, which are common in less responsive control strategies. Finally, because the phase variable is derived solely from kinematic measurements, specifically the hip angle and foot contact, it maintains robustness in the presence of sensor noise and inter-user variability, making it well-suited for real-world deployment.

3.3.1 Role of the Phase Variable in Neural Network Prediction

Incorporating the phase variable as an explicit input feature fundamentally changes how the neural networks interpret gait data. Rather than learning joint trajectories solely as functions of time-indexed kinematic signals, the models are provided with a biomechanically meaningful representation of gait progression.

The phase variable acts as a normalized temporal reference, enabling the networks to associate joint configurations with specific gait phases such as stance, push-off, and swing. This is particularly important for pathological gait, where stride duration and phase timing vary significantly across subjects and trials. Without a phase-aware representation, neural networks may struggle to distinguish whether similar joint angles correspond to early stance or late swing, leading to phase ambiguity and timing errors. By conditioning predictions on the phase variable, both the CNN and LSTM models are used to learn mappings between joint kinematics and gait progression. This results in improved temporal alignment, reduced phase drift during multi-step prediction, and smoother transitions between gait phases.

3.4 Neural Network Analysis and Architectures

Over the course of this thesis, experimentation was conducted with different neural network architectures and hyperparameter configurations. Models were trained using

kinematic data, phase-variable representations, and kinetic signals extracted from the available datasets. However, the kinetic data were excluded from the final analysis due to a large number of missing values and, in some cases, entirely empty columns within the CP datasets. In addition to the deep learning models presented in this work, simpler machine learning approaches, including decision trees [21] and feed-forward neural networks [22], were also implemented and evaluated during the writing of this thesis. While these models provided a useful baseline, their performance was limited in capturing the temporal dependencies in human gait data.

Based on these observations, two neural network architectures were selected for detailed investigation: Long Short-Term Memory (LSTM) networks and Convolutional Neural Networks (CNNs). These architectures were chosen due to their proven effectiveness in modeling sequential and time-series data, particularly in applications involving complex temporal dynamics such as human locomotion. Each architecture was implemented using two different modeling formulations: a timestamp-based representation and a phase-variable-based representation. The primary difference between these formulations is that the phase-variable-based models incorporate two additional input features corresponding to the phase variables of the left and right legs. This resulted in a total of four neural network models, enabling a systematic investigation of how gait representation influences prediction performance.

3.4.1 Tools for Neural Network Models

All neural network models in this thesis were implemented in *Python 3.10* [23], using the TensorFlow–Keras deep learning framework for model definition, training, and inference. Data handling and preprocessing were performed using NumPy [24] and pandas [25], including loading and concatenating gait files, reshaping sequences into learning windows, and organizing predicted outputs. Feature normalization was implemented using scikit-learn (specifically StandardScaler), ensuring consistent scaling across training, validation, and test splits. Model artifacts such as trained weights and scalers were stored using Keras saving utilities and joblib [26], while plots for qualitative evaluation were generated using Matplotlib [27]. In addition, the neural network models training block diagram code is shown in Figure 3.11 for better understanding the sequence from data preprocessing to exporting the nn models.

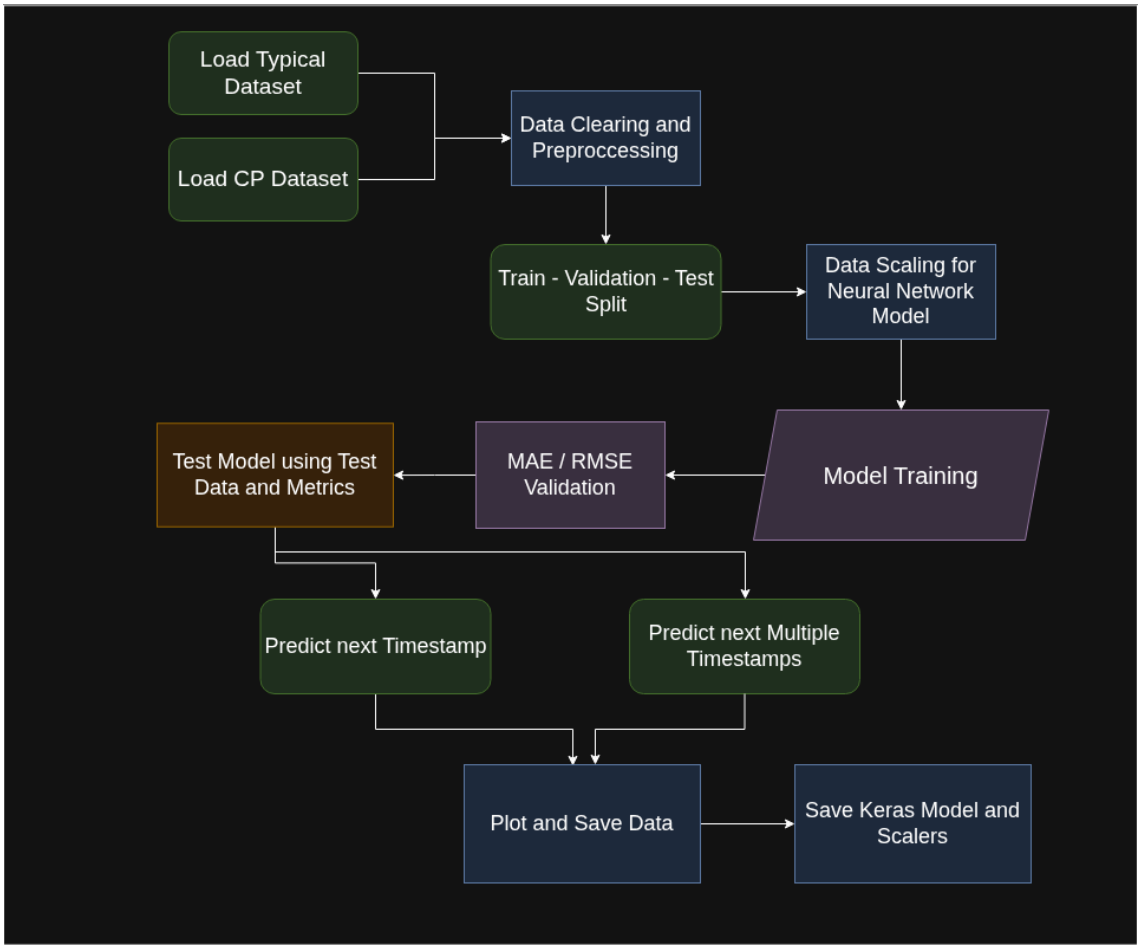


Figure 3.11. *Neural Network block diagram code.*

3.4.2 Training Setup and Hyperparameter Selection

The objective of the neural network models is to learn a mapping from input gait trajectories to predicted joint trajectories, minimizing the difference between predicted outputs $\hat{y}$ and ground truth values y. All models were trained using supervised learning, where each training sample consisted of a gait sequence of fixed length (51 time steps) and a set of joint-angle features. Depending on the modeling formulation, the input features included either joint angles only (timestamp-based models) or joint angles augmented with the phase variable (phase-variable-based models).

The dataset was split into training, validation, and testing sets, as described in the preprocessing steps. Input features were normalized using z-score normalization (`StandardScaler`), and the same scaler fitted on training data was applied to validation and test data to avoid data leakage. All four models were trained to minimize the mean squared error between predicted and measured joint angles, using the Adam optimizer [28] with learning rates selected using Optuna [29]. An *Nvidia Geforce RTX 5060 Ti* GPU was used for computation.

Rather than fixing the learning rate for the whole run, a `ReduceLROnPlateau` schedule was used, halving the rate whenever the validation loss stopped improving for ten consecutive epochs, down to a floor of 10^{-5}. The parameters explored during selection were

the learning rate, the number of layers, the number of units or filters per layer, the kernel size, and the dropout rate. The final values are summarized in Table 3.1, reported for both formulations: the sagittal-plane models predicting six joint angles and the all-planes models predicting eighteen channels (Section 3.4.5).

Hyperparameter	**Sagittal (6 ch)**	**All planes (18 ch)**
Common to all four models		
Loss function	Mean squared error	
Learning-rate schedule	`ReduceLROnPlateau`, factor 0.5, patience 10	
Batch size	512	
Epochs	150	
Input window	51 time steps	
Dropout rate	0.2	
LSTM models		
Optimizer	Adam, initial learning rate 10^{-3}	
Recurrent layers	2	2
Units per layer	128	192
Dense layer width	128	256
Trainable parameters	≈218k	≈512k
CNN models		
Optimizer	Adam, initial learning rate 10^{-4}	
Convolutional filters	(32, 48, 256, 256)	(32, 48, 384, 384)
Kernel size	3	3
Stride	2	2
Pooling size	2	2
Dense layer width	128	256
Trainable parameters	≈306k	≈608k

Table 3.1. *Hyperparameters of the four gait prediction models. Within each architecture the timestamp-based and phase-variable-based variants use identical settings, differing only in the width of the input layer, which gains two channels for the phase-variable models. The Adam optimizer's initial learning rate differs between backbones, as selected by the Optuna search described above.*

3.4.3 Long Short-Term Memory (LSTM) Architectures

Long Short-Term Memory (LSTM) networks are a special type of recurrent neural network designed to capture long-term temporal dependencies in sequential data. Unlike standard recurrent neural networks, LSTMs use internal gating mechanisms to regulate information flow and mitigate the vanishing and exploding gradient problems that arise when processing long sequences. Each LSTM unit consists of a memory cell, an input gate, a forget gate, and an output gate. These components allow the network to selectively retain or discard information across time steps, making LSTMs well suited for modeling gait dynamics, where current joint motion depends on previously observed states [30]. The LSTM models were implemented using the TensorFlow-Keras framework [31]. The networks received input sequences of 51 time steps. Depending on the modeling formulation, the input features consisted of either joint angles only (timestamp-based models) or joint angles augmented with phase variables (phase variable based models).

The LSTM architecture consists of two stacked LSTM layers with tanh activation, each followed by a dropout layer with rate 0.2, then a fully connected layer with ReLU activation, and finally a linear output layer producing one joint-angle vector for the next time step. The first recurrent layer returns the full sequence and the second returns only its final state, so that the network compresses the 51-step input window into a single fixed-length representation before regression. In the sagittal-plane formulation each recurrent layer has 128 units and the dense layer 128 units, giving approximately 218,000 trainable parameters; in the all-planes formulation these widen to 192 and 256 units respectively, giving approximately 512,000 parameters.

The overall structure of the LSTM architecture is shown in Figure 3.12 for the timestamp-based model. The phase variable based model shares exactly the same architecture and differs only in the number of inputs fed to the network, since the joint angles are augmented with the two phase variable channels; its diagram is therefore identical to Figure 3.12 apart from the width of the input layer and is not repeated here.

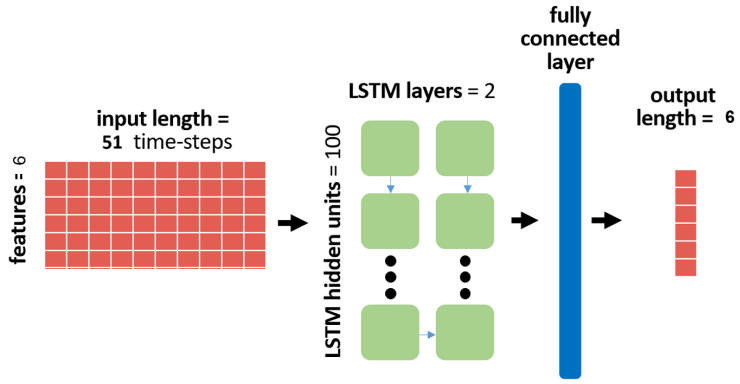


Figure 3.12. *Architecture of the LSTM model. The timestamp-based and phase variable based variants share this structure and differ only in the number of network inputs.*

3.4.4 Convolutional Neural Network (CNN) Architectures

Convolutional Neural Networks (CNNs) were also investigated as an alternative architecture for gait prediction. Although CNNs were originally developed for image processing, one-dimensional convolutional neural networks have demonstrated strong performance on time-series data by extracting local temporal features. The CNN models processed gait sequences using one-dimensional convolutional layers applied along the temporal axis. The input format was identical to that of the LSTM models, with sequences of 51 time steps and either joint-angle-only inputs (timestamp-based models) or joint angles combined with phase variables (phase variable based models).

As illustrated in Figure 3.13, the CNN architecture is organized into two convolutional blocks. The first block applies two Conv1D layers with 32 and 48 filters respectively, each using a kernel of size 3, a stride of 2 and ReLU activation, followed by a max-pooling layer of size 2 and a dropout layer. These layers extract short-term temporal patterns while progressively reducing the temporal resolution. The second block repeats this pattern with two wider Conv1D layers, again followed by max-pooling and dropout.

The resulting feature map is flattened and passed through a fully connected ReLU layer and a linear output layer producing the predicted joint angles. Zero-padding layers are inserted before the convolutions so that the strided kernels do not discard samples at the window boundaries.

In the sagittal-plane formulation the second block uses 256 filters per layer and the dense layer 128 units, giving approximately 306,000 trainable parameters. In the all-planes formulation these widen to 384 filters and 256 dense units, giving approximately 608,000 parameters.

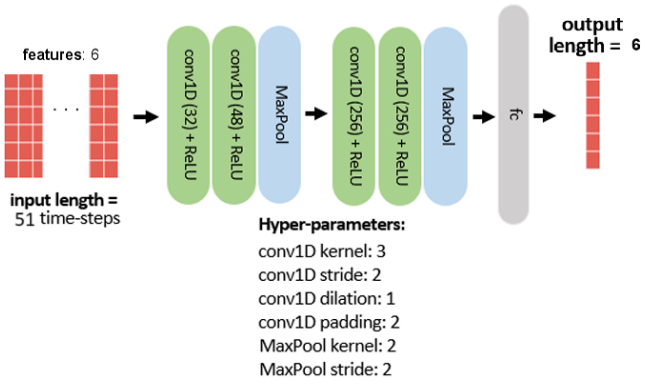


Figure 3.13. *Architecture of the CNN model. The timestamp-based and phase variable based variants share this structure and differ only in the number of network inputs.*

In the sagittal-plane formulation the phase variable based CNN uses exactly the same architecture as the one shown in Figure 3.13, differing only in the number of inputs to the network, as the joint angles are augmented with the two phase variable channels. A separate diagram is therefore not included.

Two differences arise in the all-planes formulation and are noted here for completeness. The timestamp-based CNN retains the zero-padding layers described above, whereas the phase-variable CNN omits them and relies on the `same` padding of the convolutions alone. More significantly, the timestamp-based CNN was given a multi-step output head, producing ten consecutive future samples in a single forward pass rather than one, of which only the first is used at inference time; the remaining nine act purely as an auxiliary training signal encouraging the network to model motion slightly further ahead. The phase-variable CNN and both LSTM models predict a single step directly. This asymmetry is a consequence of the two convolutional models having been developed separately, and it should be borne in mind when interpreting the comparison in Chapter 4, since the timestamp CNN divides its capacity across a ten-step objective while being evaluated on the first step alone.

3.4.5 Extension to All Anatomical Planes

The four architectures described above were initially developed for the sagittal plane only, predicting hip, knee, and ankle flexion–extension for both legs, which gives six output channels. This is consistent with the scope of most of the gait-prediction literature reviewed in Section 2.2.

However, cerebral palsy gait pathology frequently manifests outside the sagittal plane. The knee valgus and foot inversion patterns noted in Section 1.1 are frontal- and transverse-plane phenomena, and the ELEPAP dataset records hip ab/adduction, hip rotation, knee valgus/varus, knee rotation, and ankle inversion/eversion and rotation (Figures 3.1 and 3.2) that the six-channel, sagittal-only models do not use.

To capture these clinically relevant deviations, all four architectures were extended to an 18-channel formulation, jointly predicting all three anatomical planes for both legs. The 18 channels comprise, for each leg, the sagittal, frontal, and transverse angles of the hip, knee, and ankle, that is, three joints $\times$ three planes $\times$ two legs. For the phase-variable models the two phase-variable channels are appended to these as inputs, exactly as in the sagittal-only case.

This extension preserves the layer structure described above and changes only the input/output dimensionality and the layer widths, so that each network has sufficient capacity for the larger output space. The specific widths are reported in Table 3.1. Hyperparameters for the expanded models were re-selected using the same Optuna procedure described in Section 3.4.2, and converged to the same core values aside from the widened layers.

All results reported in Section 4.3 of Chapter 4, and the trajectories executed in the simulation of Chapter 5, use this all-planes formulation. The preceding sections of Chapter 4 use the sagittal-only formulation.

3.5 Chapter Summary

This chapter described the data and methodology underlying the gait prediction models developed in this thesis. The gait dataset provided by ELEPAP, comprising kinematic recordings from both typically developed children and children with cerebral palsy, was segmented into strides, filtered, normalized, and partitioned at the subject level so that held-out patients remain entirely unseen during training. The phase variable was then introduced as a continuous, monotonic representation of gait progression derived from the sagittal hip angle and foot contact, providing an alternative to indexing gait by elapsed time.

Four models were defined from the combination of two architectures, LSTM and CNN, with two temporal representations, timestamp-based and phase-variable-based, and each was subsequently extended from six sagittal channels to eighteen channels spanning all three anatomical planes. The formulation and implementation of these models were the subject of this chapter; their predictive performance and robustness are not evaluated here. Chapter 4 compares them in terms of prediction accuracy, stability under recursive

rollout, and generalization to held-out subjects, with particular attention to the effect of phase-variable conditioning.

Chapter 4

Comparison of Neural Network Models

This chapter compares four gait-prediction models: TS-LSTM, PV-LSTM, TS-CNN, and PV-CNN. All models receive an input window of 51 time steps and predict the next time step of the six joint angles (left/right hip, knee, ankle). The comparison is performed on cerebral palsy (CP) test data using both quantitative error metrics and qualitative analysis of predicted trajectories.

4.1 Comparison Using Error Metrics

To assess the effectiveness of the neural network models in forecasting human gait, two frequently utilized regression error metrics were calculated: Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE). These metrics measure the disparity between estimated values $\hat{y}_i$ and actual values y_i across a total of n samples.

Root Mean Squared Error (RMSE): RMSE 4.1 calculates the square root of the mean of the squared differences between the predicted values and the actual values. By squaring the errors prior to averaging, RMSE imposes a greater penalty on significant prediction errors compared to minor ones, thus making it especially responsive to outliers and infrequent large deviations [32]. In the realm of gait prediction, RMSE emphasizes cases where the model generates unstable or markedly incorrect joint angle predictions, which is particularly significant for sequential models where errors can build up over time.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{4.1}$$

Mean Absolute Error (MAE): MAE 4.2 represents the average of the absolute differences between predicted and ground truth values. Unlike RMSE, MAE weights all errors linearly and therefore provides a more intuitive measure of the typical prediction error magnitude [33]. In this work, MAE reflects the average deviation in joint angles (in degrees) and offers a robust indication of overall prediction accuracy that is less influenced by isolated large errors.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{4.2}$$

4.1.1 Interpretation of RMSE and MAE

RMSE and MAE capture complementary aspects of prediction error. RMSE penalizes large deviations more strongly due to the squaring operation and is therefore sensitive to transient prediction failures, particularly during rapid gait transitions such as heel-strike or toe-off. In contrast, MAE provides a linear and intuitive measure of the average prediction error and is less influenced by outliers.

In the context of gait prediction, low RMSE indicates robustness against large temporal or amplitude errors, while low MAE reflects consistent tracking accuracy during continuous operation. Reporting both metrics ensures that models are evaluated not only on average accuracy, but also on their ability to avoid large, potentially destabilizing errors.

Table 4.1 reports the MAE and RMSE for each joint angle, along with the mean error across all six joints.

Joint	**TS-LSTM** MAE / RMSE	**PV-LSTM** MAE / RMSE	**TS-CNN** MAE / RMSE	**PV-CNN** MAE / RMSE
L Hip	1.19 / 1.50	0.92 / 1.24	1.09 / 1.36	1.01 / 1.29
L Knee	2.21 / 2.63	2.33 / 2.77	2.53 / 3.09	2.27 / 2.83
L Ankle	1.73 / 2.56	1.75 / 2.48	1.24 / 1.78	1.20 / 1.71
R Hip	1.52 / 2.13	1.71 / 2.34	1.45 / 2.07	1.59 / 2.08
R Knee	1.51 / 1.90	2.06 / 2.60	2.32 / 2.98	1.89 / 2.49
R Ankle	1.39 / 2.00	1.35 / 1.81	1.22 / 1.68	1.26 / 1.69
Mean (all joints)	1.59 / 2.12	1.69 / 2.21	1.64 / 2.16	1.54 / 2.02

Table 4.1. *Per-joint prediction error (degrees) for all models evaluated on CP test data.*

Across all models, knee joints exhibit higher MAE and RMSE than hip and ankle joints, reflecting their larger dynamic range and sharper transitions during pathological gait. These characteristics increase sensitivity to small timing errors, which are amplified in RMSE. Consequently, higher knee errors primarily reflect biomechanical complexity rather than poor model performance.

On this metric all four models appear closely matched, with mean errors spanning only 1.54° to 1.69°, and PV-CNN attains the lowest mean MAE and RMSE of the four.

This ranking should not be taken as representative. It is obtained on a single fixed sagittal-plane segment, and Section 4.3 shows that it does not survive evaluation on multiple held-out patients across all three anatomical planes. In that broader setting the two CNN-based models are separated from the two LSTM-based models by roughly a factor of three in mean absolute error, and the small advantage of PV-CNN over TS-CNN on this segment does not persist in one-step-ahead accuracy, although it does reappear in the multi-stride stability results of Section 4.2.3 and Chapter 5. The narrow spread in Table 4.1 therefore reflects the fact that a single short segment is an easy test rather than any genuine equivalence between the architectures, and it illustrates why the evaluation

over all held-out patients presented in Section 4.3.1 is necessary before any model can be recommended.

4.2 Comparison of Prediction Behavior

Quantitative metrics alone do not fully capture prediction behavior relevant to biomechanical validity and deployment, such as temporal stability, phase consistency, and bias. Therefore, the models are further compared using: (i) one-step-ahead predictions, (ii) multi-stride predictions, (iii) rolling one-step-ahead rollouts, (iv) absolute error trajectories, and (v) residual distributions.

4.2.1 One-step-ahead predictions

Figures 4.1–4.4 show, for a single representative CP stride, the input sequence (blue), each model's one-step-ahead prediction (orange), and the corresponding ground-truth sample (green). Across all four models, the largest single-joint offset on this stride occurs at the knee, consistent with the higher per-joint MAE and RMSE already reported for the knee in Table 4.1; rather than repeating this observation for each model in turn, the paragraphs below focus on what distinguishes each model's overall six-joint prediction.

TS-LSTM (Figure 4.1) places its predicted marker close to the ground truth across all six joints, with the largest visible offset at the knee.

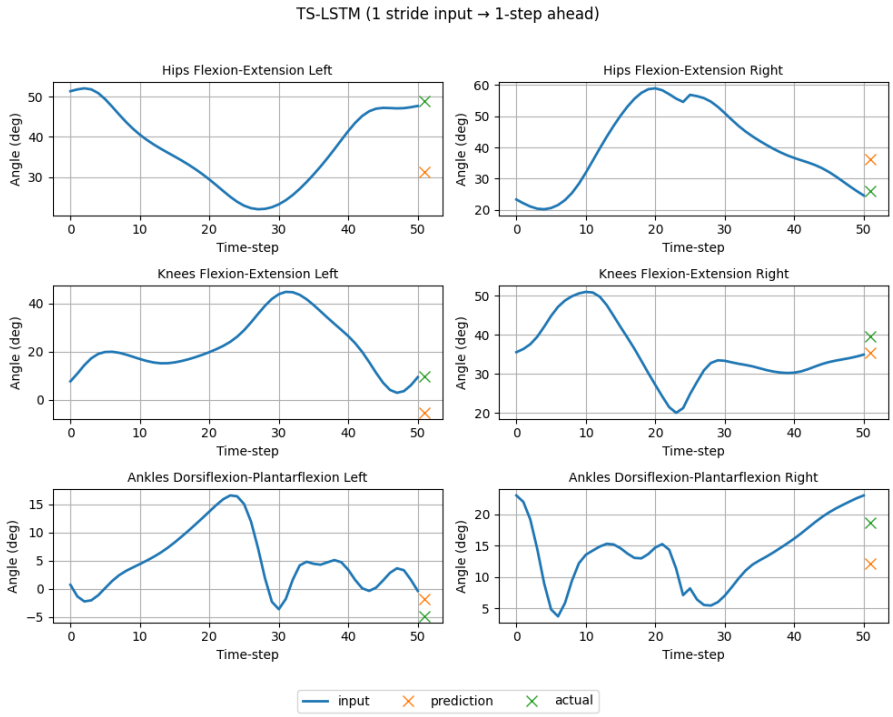


Figure 4.1. *TS-LSTM: one-stride input and one step ahead prediction.*

PV-LSTM (Figure 4.2) matches the ground truth equally closely on this stride and attains the lowest MAE of the four models at the left hip (Table 4.1).

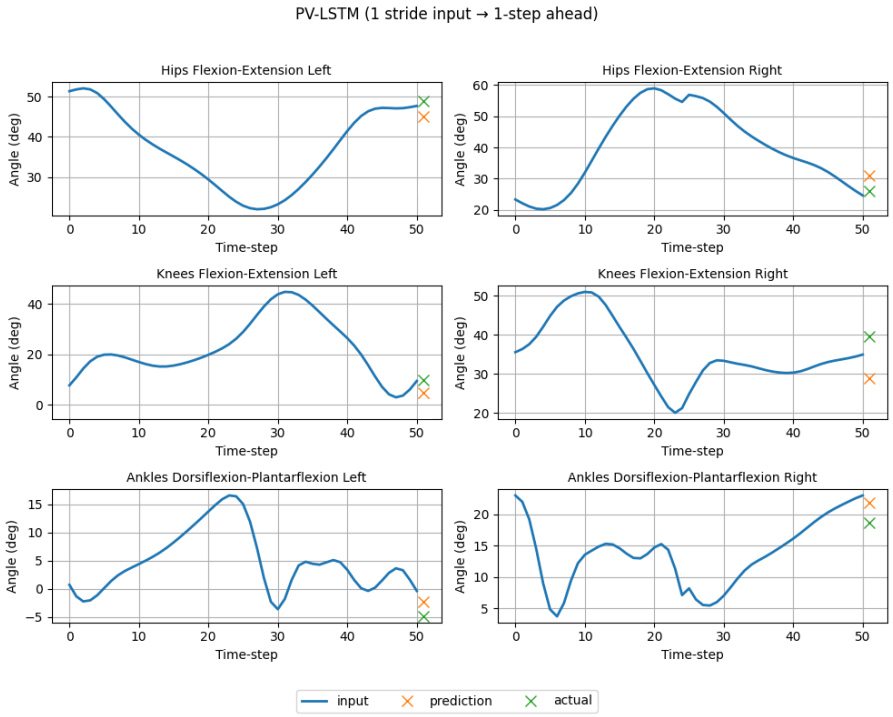


Figure 4.2. *PV-LSTM: one-stride input and one step ahead prediction.*

TS-CNN (Figure 4.3) also produces a plausible one-step continuation for every joint, though its left-knee prediction shows the largest single joint–model offset recorded in Table 4.1.

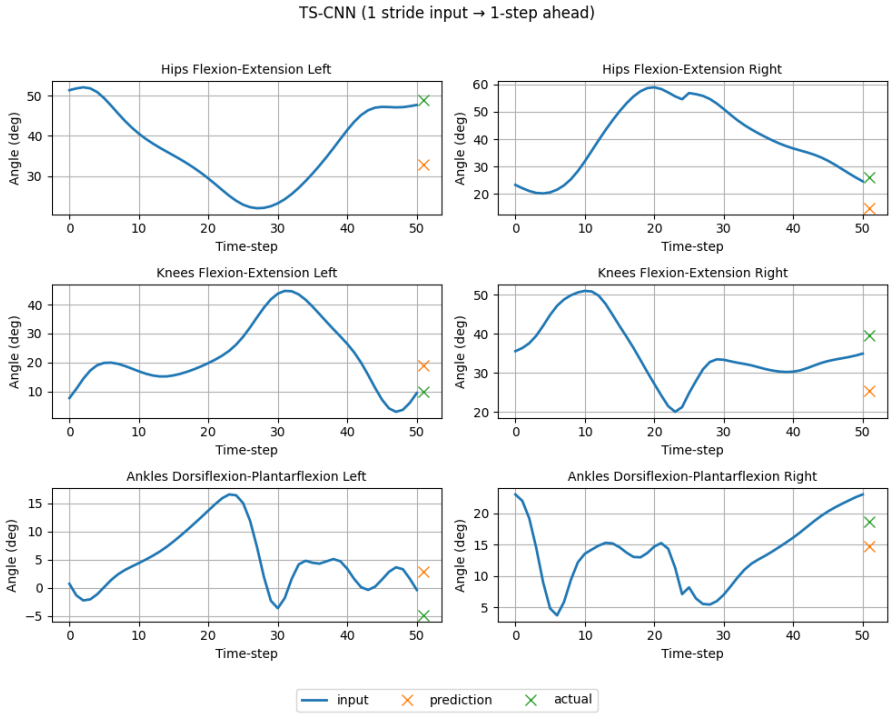


Figure 4.3. *TS-CNN: one-stride input and one step ahead prediction.*

PV-CNN (Figure 4.4) shows the closest overall match between predicted and ground-

truth markers of the four models on this stride, consistent with its lowest mean MAE and RMSE across all six joints (Table 4.1).

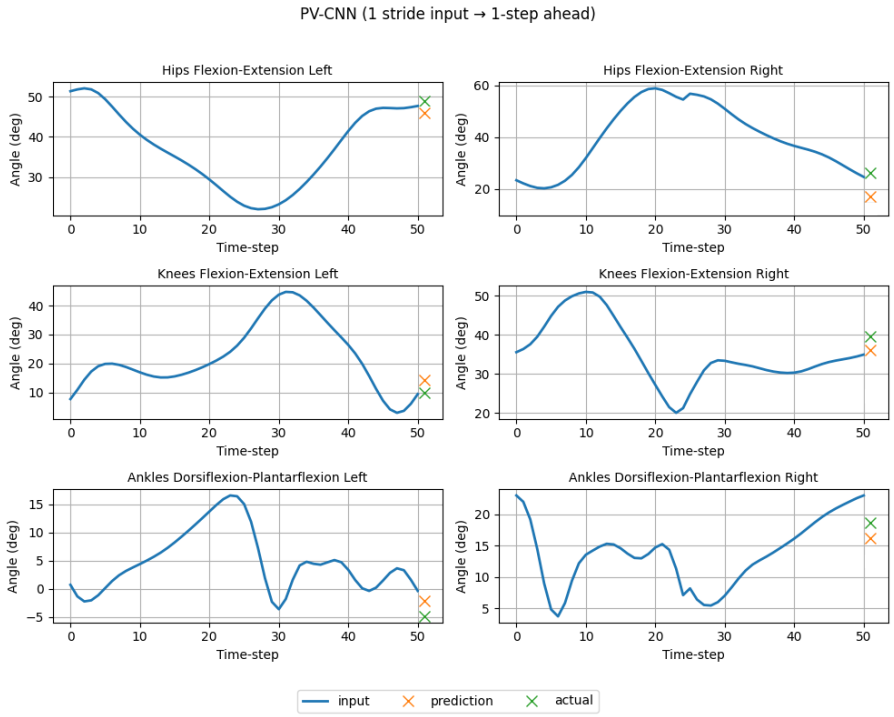


Figure 4.4. *PV-CNN: one-stride input and one step ahead prediction.*

4.2.2 Multi-stride predictions on CP data

Figure 4.5 extends this comparison from a single stride to a continuous multi-stride CP segment, plotting the ground truth against all four models together so that their relative performance over several consecutive gait cycles can be assessed at a glance.

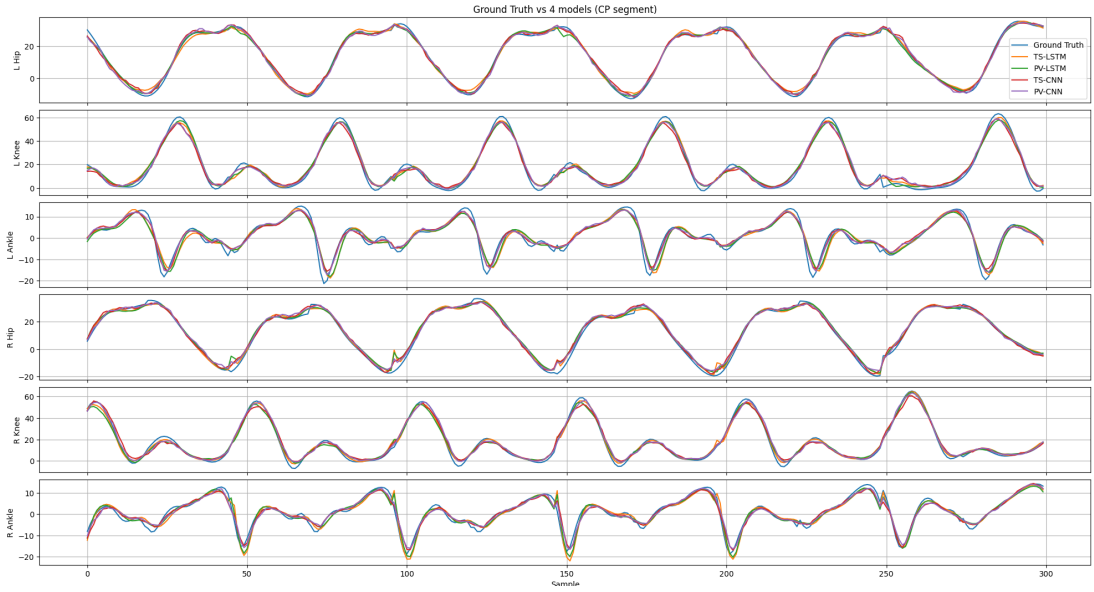


Figure 4.5. *Ground truth vs. all four models on the same CP segment.*

All four models capture the overall periodic structure of pathological gait, with particularly strong visual alignment at the hip and ankle joints; this indicates that the dominant low-frequency components of gait motion are learned effectively across architectures. Differences are more apparent at the knee, where variations in peak amplitude and waveform shape emerge during rapid flexion phases, reflecting greater sensitivity to timing and magnitude errors in this joint.

4.2.3 Rolling one-step-ahead prediction and error accumulation

While one-step-ahead predictions provide insight into local accuracy, practical deployment of gait prediction models (e.g., for assistive control or online estimation) requires repeated use of the model over time. To evaluate this behavior, a rolling one-step-ahead rollout was performed, where each model recursively feeds its own prediction back as input for the next time step. Figures 4.6–4.9 illustrate rolling predictions over six complete strides for all four models. This evaluation highlights the accumulation of prediction errors, long-term stability, and phase consistency across extended horizons.

TS-LSTM (Figure 4.6) remains reasonably accurate over short horizons; however, noticeable amplitude drift and phase misalignment emerge as the rollout progresses. This behavior is consistent with the recurrent accumulation of small temporal errors, which compound when predictions are recursively reused as inputs.

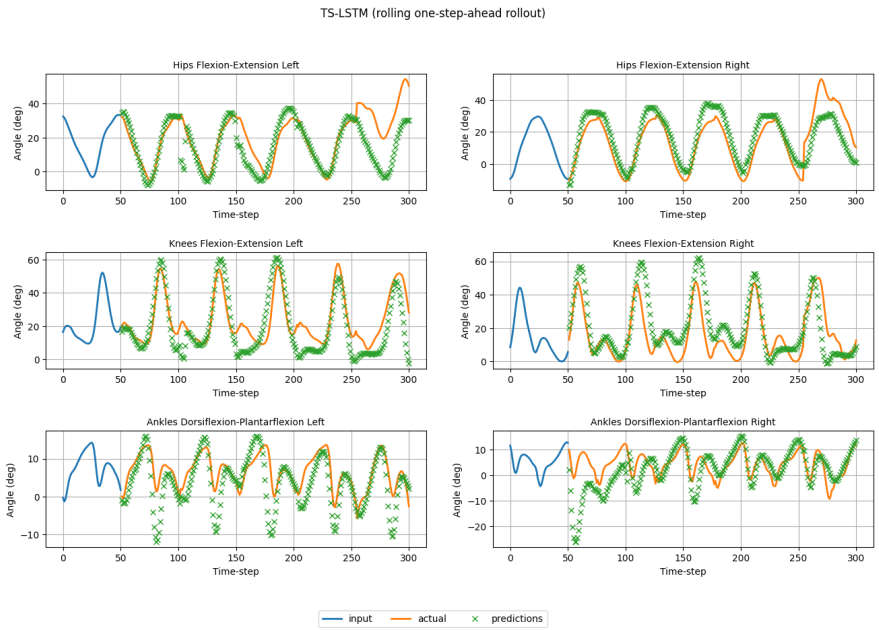


Figure 4.6. *TS-LSTM rolling one-step-ahead rollout over multiple strides.*

PV-LSTM (Figure 4.7) demonstrates improved phase alignment over time, particularly for hip and knee joints. The inclusion of the phase variable constrains predictions to remain synchronized with the gait cycle, reducing phase drift compared to the timestamp-based formulation. Nevertheless, localized amplitude errors persist, especially during high-dynamic knee flexion phases.

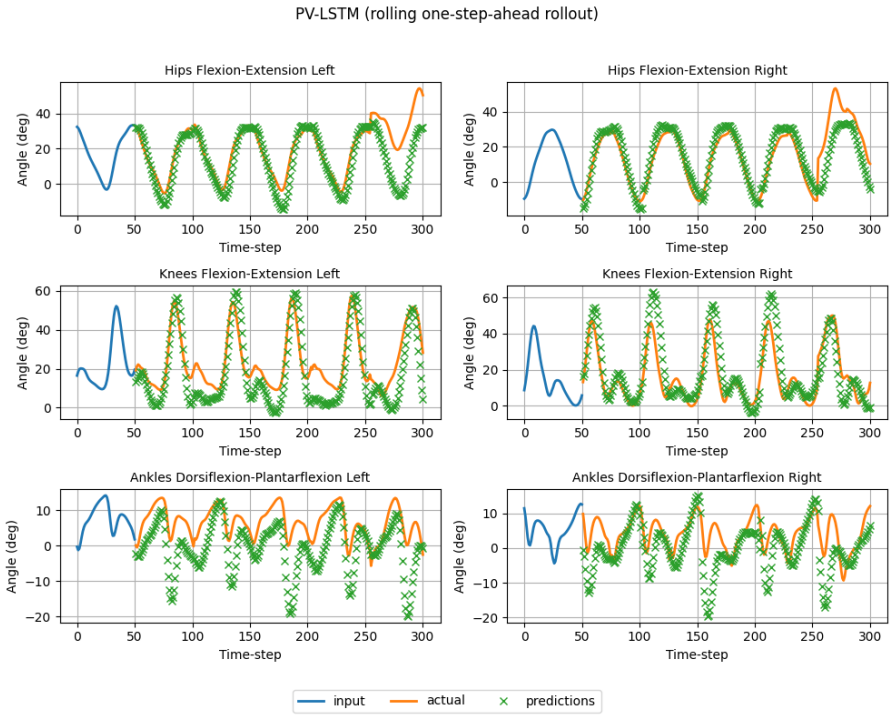


Figure 4.7. *PV-LSTM rolling one-step-ahead rollout over multiple strides.*

TS-CNN (Figure 4.8) captures local joint patterns effectively but is more sensitive to long-horizon drift than either LSTM-based model. As convolutional models lack explicit temporal memory, small prediction errors accumulate more rapidly once the model operates in a closed loop.

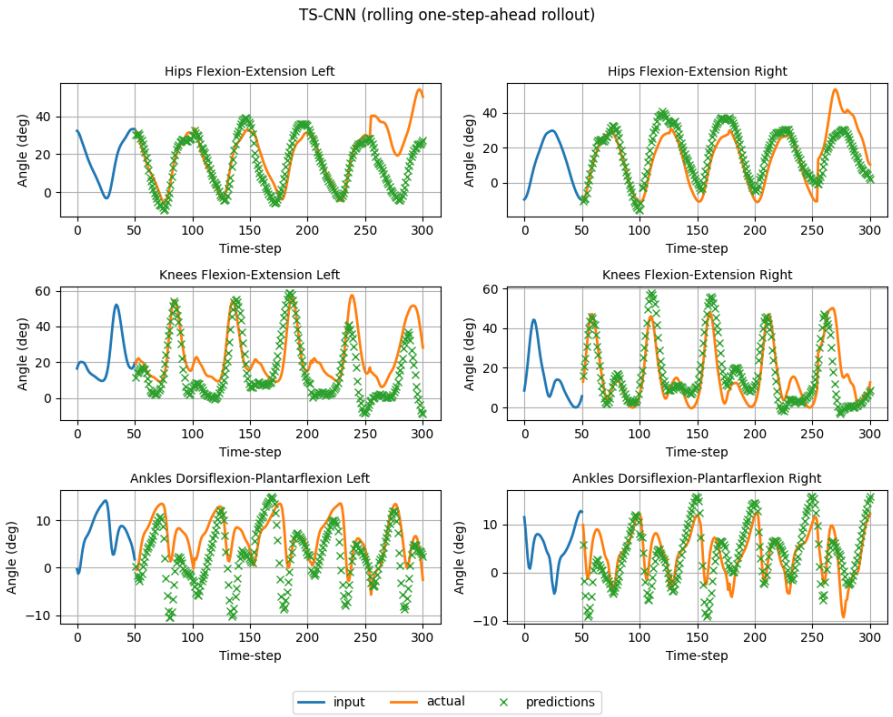


Figure 4.8. *TS-CNN rolling one-step-ahead rollout over multiple strides.*

PV-CNN (Figure 4.9) shows the most stable long-term behavior of the two CNN-based models on this rollout, remaining phase-consistent over multiple strides with reduced amplitude and timing drift relative to TS-CNN. This indicates that explicit phase conditioning provides a useful structural prior for the convolutional backbone, even though, as Section 4.3 shows, this prior is not sufficient to match the recurrent models once the evaluation is extended to unseen subjects.

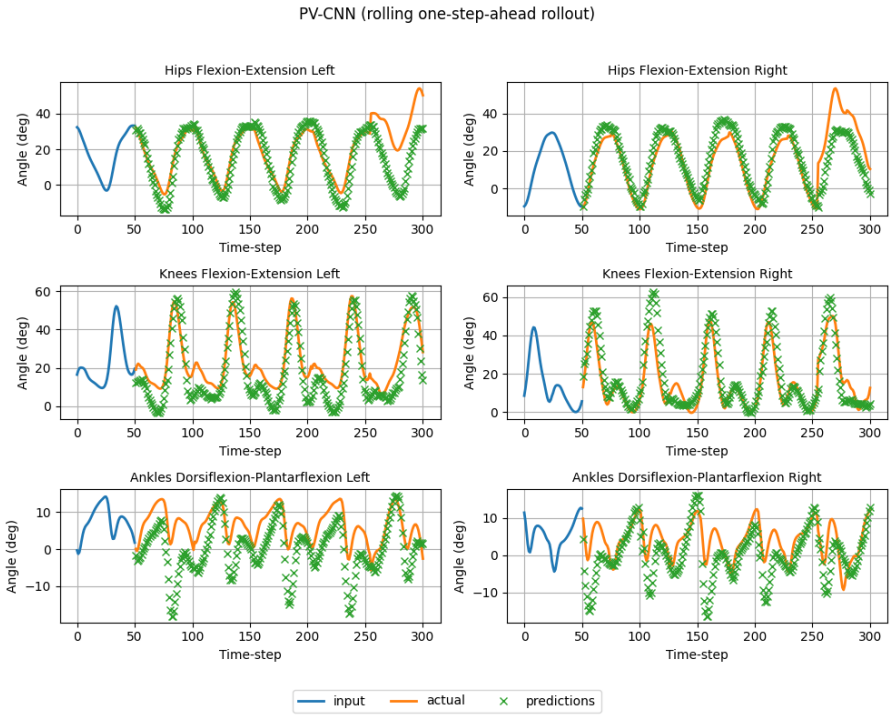


Figure 4.9. *PV-CNN rolling one-step-ahead rollout over multiple strides.*

4.2.4 Absolute error trajectories and failure cases

Figure 4.10 complements the overlay plot above by showing the absolute error $|e(t)|$ for each joint and model over six strides, making it possible to identify localized failures that would otherwise be averaged out in the global metrics of Table 4.1.

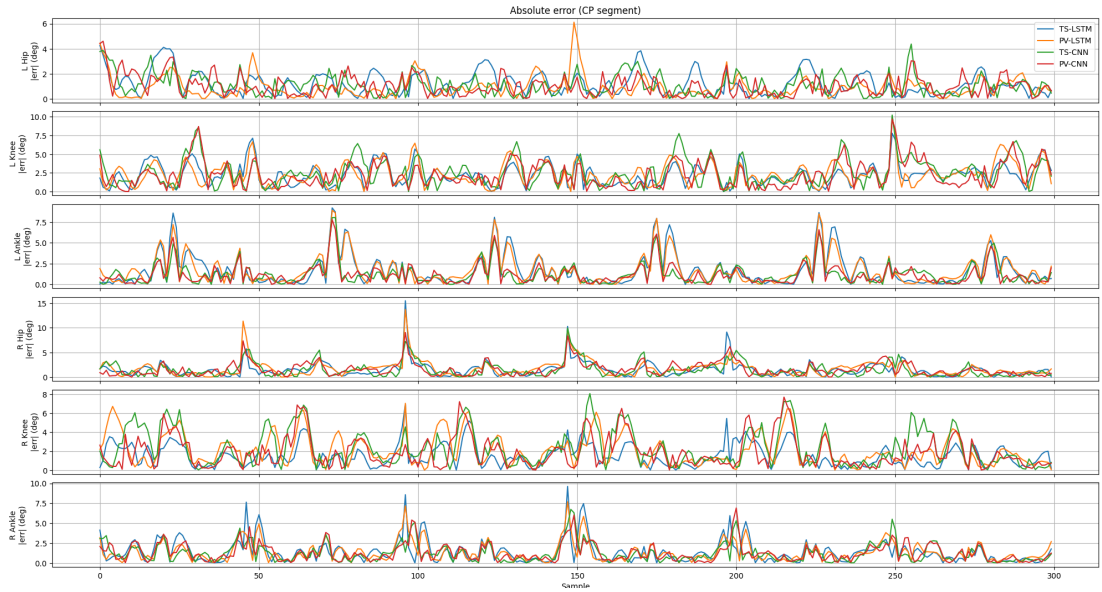


Figure 4.10. *Absolute error $|e(t)|$ on the CP segment for all four models.*

Error spikes for all four models cluster near fast kinematic transitions, such as heel-strike and toe-off, where joint angular velocities are highest. Since gait is a periodic process, this phase-dependence of error is examined in more detail in Section 4.3, where the mean absolute error is broken down by gait phase for a held-out CP subject (Figures 4.14–4.15), rather than for the single fixed segment used elsewhere in this chapter.

4.2.5 Residual distribution (bias and variance)

Figure 4.11 presents residual histograms for each joint angle and model, offering a view of prediction bias and variance that is not visible in Table 4.1 or in the trajectory plots above.

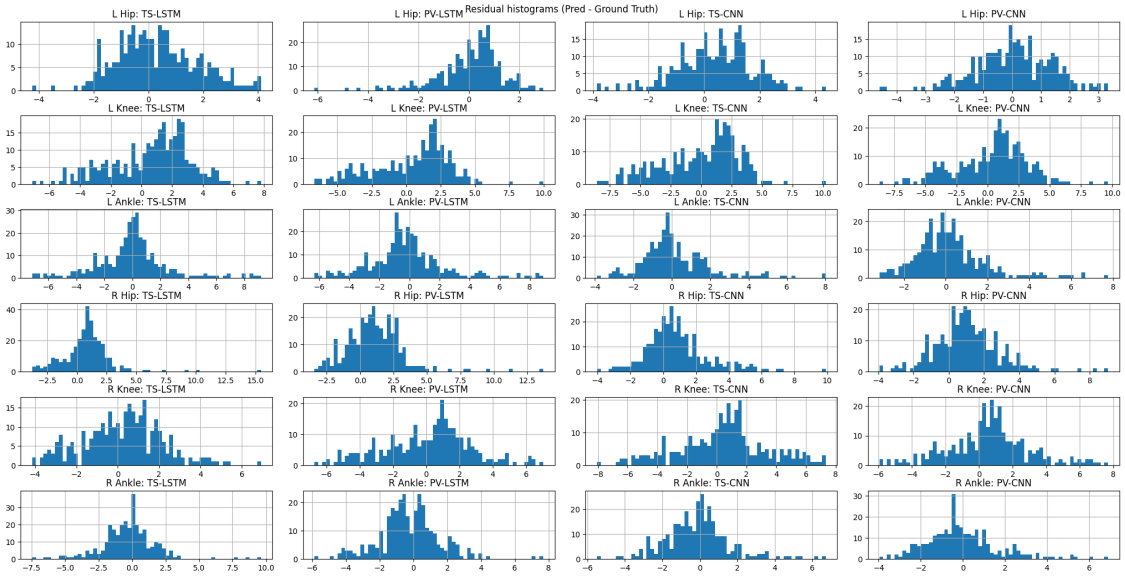


Figure 4.11. *Residual histograms $(\hat{y} - y)$ for each joint angle and model.*

For all four architectures, residuals are centered close to zero, indicating that no systematic over- or underestimation is present at a global level; this suggests that the models learn unbiased representations of joint kinematics on average. Differences between models instead show up in the spread and symmetry of the distributions [34]. Timestamp-based models (TS-LSTM and TS-CNN) produce wider residual distributions for the knee joints, particularly the right knee, indicating higher variance and increased sensitivity to rapid joint dynamics. In contrast, the phase-variable models produce more concentrated residual distributions, suggesting improved consistency and reduced variability in prediction error. On this CP segment, PV-CNN shows the least dispersed and most balanced residual distributions of the four models, consistent with its lower RMSE values and the enhanced rolling-rollout stability noted above. Residual skewness is most evident in the ankle joints and reflects the asymmetric characteristics of dorsiflexion and plantarflexion in pathological gait, rather than any systematic model bias. Overall, these residual histograms suggest that phase-variable conditioning leads to predictions that are more stable and less biased – a trend that reappears, this time in favor of PV-LSTM, in the broader multi-subject evaluation of Section 4.3.

4.3 Generalization Across Held-Out Subjects and Gait Phases

The evaluation presented so far is based on a single, fixed CP test segment and on six sagittal joint angles. To assess whether the same conclusions hold under more demanding conditions, this section evaluates the 18-channel, all-planes formulation of the four architectures introduced in Section 3.4.5, in which each model jointly predicts the sagittal, frontal, and transverse planes for both legs.

Apart from the change in output dimensionality and the accompanying layer widths, the pipeline is unchanged from the six-channel models used in the preceding sections. The same subject-level train/validation/test split of Section 3.2.4 applies, together with the same rolling-window construction, teacher-forced one-step-ahead training, and phase-variable computation from the sagittal hip angle. Because the split is at the subject level, every stride belonging to a test subject is entirely unseen during training, and the rolling input windows for each held-out subject are rebuilt independently from that subject's own trials so that no window ever crosses a subject boundary. Predictions were generated in a teacher-forced, one-step-ahead fashion across the full length of each subject's recording. All figures and metrics from this section onward (Figures 4.12–4.21) were produced with these 18-channel networks, in contrast to the sagittal-only networks used in the preceding sections of this chapter.

To relate prediction error to the underlying biomechanics, errors were further analyzed as a function of position within the gait cycle, using the seven phases defined in Section 2.1.1: Loading Response (LR), Mid Stance (MSt), Terminal Stance (TSt), Pre-Swing (PSw), Initial Swing (ISw), Mid Swing (MSw) and Terminal Swing (TSw). Rather than relying on the textbook 60% stance/swing split, the toe-off event that separates the stance phases from the swing phases was anchored to each subject's own *measured* mean toe-off percentage, obtained from the Foot-Off channels in the corresponding trial.

This per-subject anchoring concerns only how prediction error is *reported* as a function of gait phase, and should not be confused with the phase variable supplied to the networks. As stated in Section 3.3, the scaling constant c in the phase-variable formulation is fixed at $c = 0.53$ for every subject and every stride; it does not vary from patient to patient. The measured toe-off percentage is used solely to place the phase boundaries in the figures of this section, so that the error attributed to, for example, Pre-Swing corresponds to the interval in which that patient was actually in pre-swing. Using a single fixed boundary for all patients would otherwise misattribute error between the stance and swing phases for patients whose toe-off departs from the textbook value, which is common in cerebral palsy.

Within each period the phases occupy fixed fractions: of the stance period, LR spans the first 16.7%, MSt from 16.7% to 50%, TSt from 50% to 83.3%, and PSw the remainder; of the swing period, ISw spans the first 32.5%, MSw from 32.5% to 67.5%, and TSw the remainder. For a subject whose toe-off falls at the textbook 60% this reproduces the conventional boundaries, but for subjects with markedly delayed toe-off, which is common in cerebral palsy, the phase boundaries shift accordingly.

4.3.1 Results Across All Held-Out Patients

Table 4.2 reports each model's performance aggregated over all 16 held-out cerebral palsy patients, and Table 4.3 gives the corresponding per-subject coefficients of determination. Together these replace the single-segment view of Section 4.1 with an estimate of how each architecture behaves on previously unseen patients.

Model	**MAE (deg)**	**RMSE (deg)**	R^2
TS-LSTM	2.20 ± 1.02	2.71 ± 1.17	**0.56 ± 0.38**
PV-LSTM	**2.10 ± 0.87**	**2.60 ± 0.98**	0.55 ± 0.45
TS-CNN	6.12 ± 1.90	7.14 ± 2.04	−2.74 ± 5.01
PV-CNN	6.56 ± 1.77	7.71 ± 1.97	−3.29 ± 6.96

Table 4.2. *Performance across all 16 held-out CP patients, averaged over the 18 joint-plane channels. Values are mean ± standard deviation across subjects.*

The separation between the recurrent and convolutional architectures is the dominant effect and is far larger than any difference attributable to the temporal representation. Both LSTM-based models keep the mean absolute error near 2°, whereas both CNN-based models exceed 6°, a threefold difference. The distinction is starker still in R^2: the LSTM models achieve a positive coefficient of determination on almost every subject, while the CNN models are negative on the large majority, meaning their predictions explain less of the variance than simply predicting each subject's mean posture would.

By contrast, the difference between the timestamp and phase-variable formulations is small and not consistent in sign. PV-LSTM attains the lower mean absolute and root-mean-square error, and is the better of the two recurrent models on 11 of the 16 subjects, but TS-LSTM attains a marginally higher mean R^2 and is positive on 15 subjects rather than 13. The gap between them, 0.01 in mean R^2, is an order of magnitude smaller

than the between-subject standard deviation of roughly 0.4, so the two recurrent models should be regarded as comparable in accuracy rather than clearly separated. For the convolutional backbone, TS-CNN in fact outperforms PV-CNN on every aggregate in Table 4.2.

This last observation should not be read as evidence that phase-variable conditioning is unhelpful for the convolutional architecture. Table 4.2 is computed in the teacher-forced, one-step-ahead mode described in Section 4.3.2, in which the model is re-anchored to the measured trajectory at every time step. That is precisely the regime in which a phase variable can contribute least, because the input window already carries the correct recent history and no timing error is allowed to accumulate. The property the phase variable is intended to provide, namely a representation of gait progression that does not drift when the model must rely on its own output, becomes observable only once predictions are fed back recursively. Under that condition the ordering reverses: Section 4.2.3 shows PV-CNN maintaining phase consistency over multiple strides where TS-CNN accumulates amplitude and timing drift, and the closed-loop execution results of Chapter 5 show the same distinction on the simulated exoskeleton, where the timestamp models progressively lose alignment with the commanded trajectory while both phase-variable models remain synchronized across consecutive gait cycles. The two measurements are therefore complementary rather than contradictory: within the convolutional backbone the phase variable trades a small amount of per-step accuracy for a substantial gain in multi-stride stability.

Subject	**TS-LSTM**	**PV-LSTM**	**TS-CNN**	**PV-CNN**
42043	−0.29	−0.63	−19.77	−28.80
42156	0.90	0.90	0.15	0.00
47654	0.72	0.69	−1.35	−1.73
50574	0.73	0.60	−0.73	−1.96
51291	0.83	0.84	−0.17	−0.74
52398	0.90	0.92	−0.00	0.06
52789	0.91	0.89	0.23	−0.30
52800	0.81	0.81	−0.34	−0.30
52827	0.06	−0.08	−6.92	−4.86
52880	0.80	0.80	−0.72	−0.47
52888	0.24	0.43	−3.19	−3.63
52909	0.17	0.54	−1.40	−2.08
52932	0.47	0.53	−2.48	−2.78
52934	0.80	0.82	0.05	−0.30
52946	0.06	−0.17	−5.82	−3.40
52962	0.81	0.86	−1.30	−1.27
Mean	0.56	0.55	−2.74	−3.29
Median	0.76	0.75	−1.02	−1.50
$R^2 > 0$	15/16	13/16	3/16	2/16

Table 4.3. *Coefficient of determination per held-out CP patient, averaged over all 18 joint-plane channels.*

Two features of Table 4.3 deserve comment. First, the spread across patients is large for every architecture: even PV-LSTM ranges from 0.92 on the most favourable

subject to −0.63 on the least. Reporting a single subject would therefore give a misleading impression of reliability, which motivates the best, median and worst cases examined in the following sections.

Second, a negative R^2 does not necessarily indicate a large absolute error. The coefficient of determination normalises the residual by the variance of the subject's own ground truth, so a patient who moves through a small range of motion can produce a poor R^2 even when the prediction is accurate in degrees. Subject 42043 is the clearest example: PV-LSTM achieves a mean absolute error of only 1.58° on this patient, comparable to the average across patients, yet scores $R^2 = -0.63$ because the measured joint excursion is unusually small. The two metrics should therefore be read together, with MAE describing the magnitude of the error and R^2 describing how much of the patient's own movement pattern the model reproduces.

4.3.2 From One-Step-Ahead Predictions to Stride- and Phase-Averaged Trajectories

All four models share the same underlying prediction mechanism: given an input window of 51 consecutive time steps, each produces an estimate of the six (or, in the 18-channel formulation of this section, eighteen) joint angles at the very next time step, $t+1$. No architecture in this study outputs a whole phase segment or a complete stride in a single forward pass. The one qualification is that the all-planes timestamp CNN carries a multi-step output head, emitting ten consecutive future samples per forward pass as described in Section 3.4.4; only the first of these is used, so its effective prediction horizon is identical to that of the other three models and all comparisons in this chapter remain like-for-like. What differs between the plots shown throughout this chapter is not the underlying model output, but how repeated one-step-ahead predictions are collected, fed back into the model, and subsequently displayed. Three distinct readouts of this same one-step-ahead mechanism are used:

- **Next timestamp (teacher-forced one-step-ahead).** This is the most direct readout, used in Figures 4.1–4.4. At every time step, the model receives the true, measured 51-sample window as input and predicts only the single next sample. The predicted value is then discarded as an input; at the next time step, the window again slides forward using ground-truth data. Because the model never sees its own previous prediction, this mode isolates the model's raw, local forecasting accuracy from any accumulation of error, and is also the mode used to compute the per-joint MAE/RMSE of Table 4.1.

- **Next complete stride (recursive rollout).** Used in the rolling one-step-ahead rollouts of Figures 4.6–4.9. Here the model's own one-step-ahead prediction is appended to the input window in place of the (unavailable) ground-truth sample, and this updated window is used to generate the following prediction, and so on, for the length of an entire stride or several consecutive strides. No teacher forcing is applied after the first step: the model is effectively predicting several seconds of

future motion using only its own past outputs as memory. This is what allows Section 4.2.3 to discuss error accumulation, amplitude drift, and phase misalignment, none of which can be observed under teacher forcing, since a teacher-forced model is re-anchored to the truth at every step regardless of how poor its previous prediction was.

- **Next phase (stride-averaged, teacher-forced).** Used in Figures 4.12–4.21 and in the phase-resolved error bars of Figures 4.14–4.15. This readout is generated from the same teacher-forced one-step-ahead predictions described above, run continuously across the full length of a subject's recording (i.e. across many consecutive strides), rather than from a single generated stride. Each individual prediction is then tagged with its position in the gait cycle, expressed as a percentage of stride (0–100%), using the subject's own measured toe-off timing to anchor the stance/swing boundary. All predictions (and all ground-truth samples) that fall within the same phase percentage bin, across every stride in the recording, are then averaged together, yielding one mean predicted curve and one mean ground-truth curve over a single normalized gait cycle, together with a ±1 SD band computed from the underlying strides. Averaging over many strides in this way suppresses stride-to-stride noise and makes systematic, phase-locked deviations between prediction and ground truth (for example, the suppressed knee flexion range discussed in Section 4.3) considerably easier to identify than in any single raw stride.

It is worth emphasizing that the stride-averaged plots of this section are, in this sense, a statistical summary of many one-step-ahead predictions rather than evidence of long-horizon forecasting: they show how accurately each model tracks the gait cycle when it is always given the true recent history as input, whereas the rolling rollouts of Section 4.2.3 (Figures 4.6–4.9) show how that same accuracy degrades once the model must rely on its own previous outputs. The two readouts are therefore complementary: one characterizes best-case, per-step accuracy across the full gait cycle, and the other characterizes worst-case, compounding error over time.

4.3.3 Stride-Averaged Predictions Over the Gait Cycle

Figures 4.12 and 4.13 show, for held-out subject 52398, the mean predicted trajectory of all four models plotted against the mean ground truth (±1 SD band) over one normalized gait cycle, for every joint–plane combination on the left and right leg respectively; the inset box in each figure reports the overall MAE, RMSE and R^2 for that side. As Table 4.3 shows, this subject is the most favourable of the 16 held-out patients, and the figures should therefore be read as a best case rather than as a typical one. The median and worst cases are presented in Sections 4.3.6 and 4.3.7.

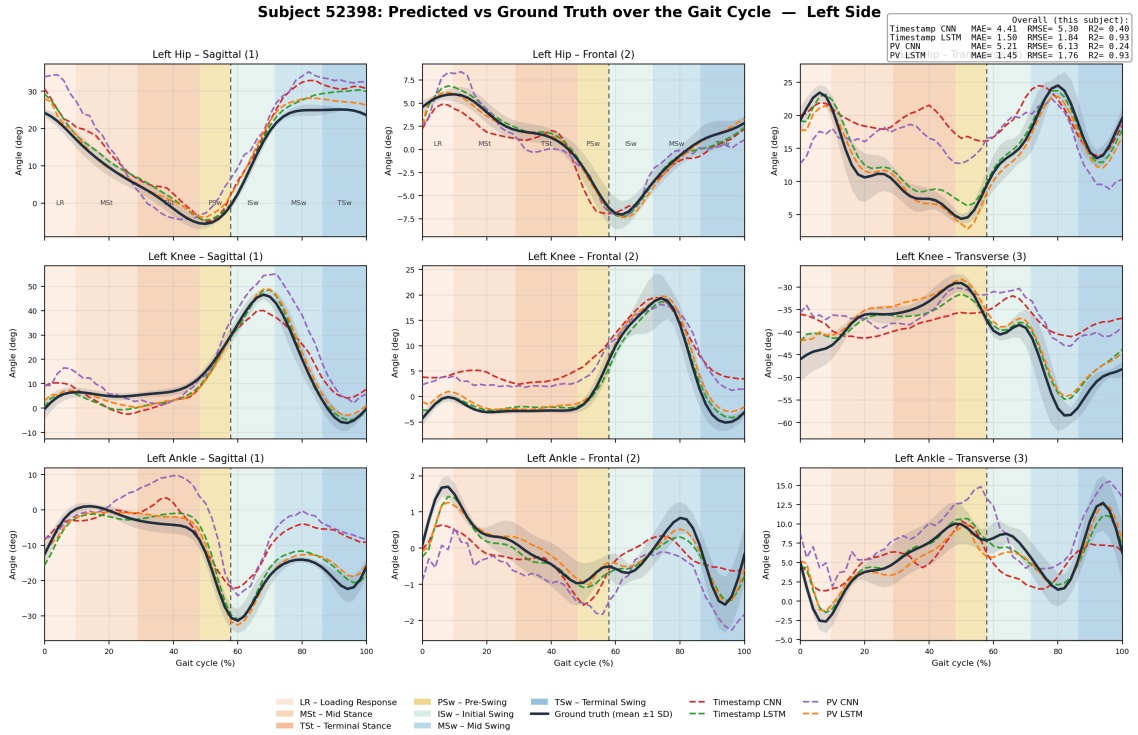


Figure 4.12. *Stride-averaged predicted vs. ground-truth trajectories over the gait cycle, left side, for a held-out CP test subject. Background shading indicates the seven gait phases, anchored to the subject's own measured toe-off.*

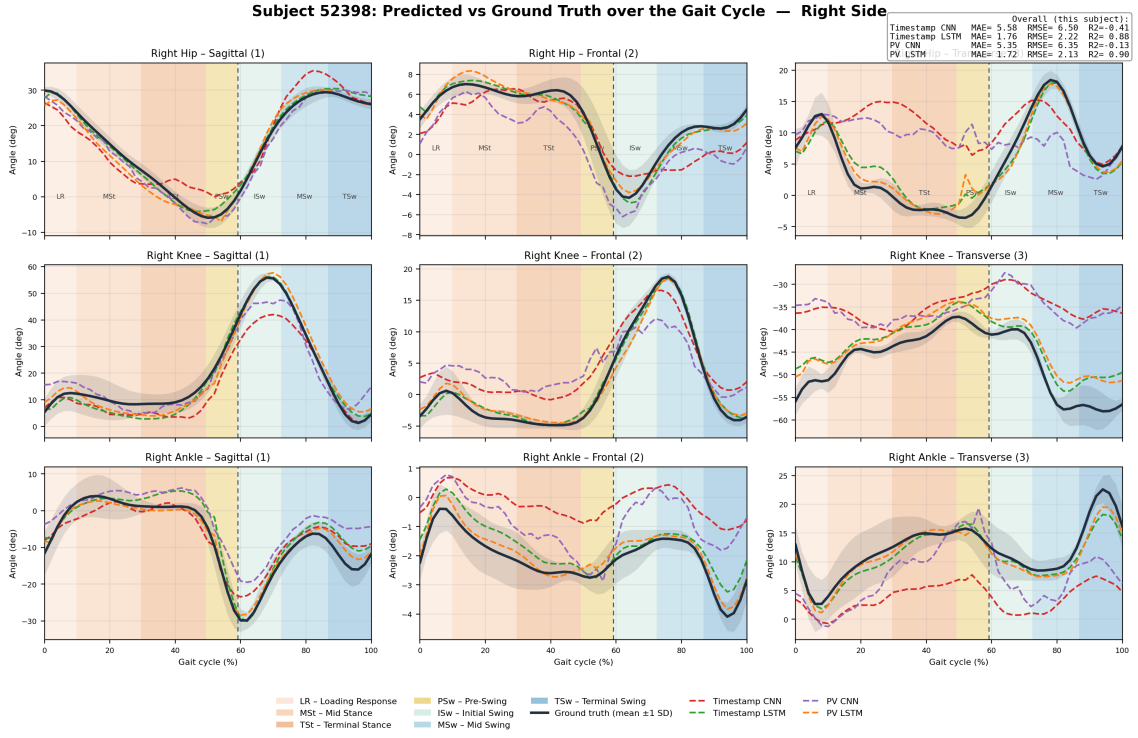


Figure 4.13. *Stride-averaged predicted vs. ground-truth trajectories over the gait cycle, right side, for the same held-out CP test subject as in Figure 4.12.*

For this subject, the two LSTM-based models track the ground-truth trace closely

across nearly every joint and plane on both legs, reaching $R^2 = 0.93$ on the left side for both TS-LSTM and PV-LSTM, whereas the two CNN-based models deviate noticeably, particularly around the hip and knee ($R^2 = 0.40$ and 0.24 respectively on the same side). Even on this most favourable subject, the convolutional models therefore explain substantially less of the measured motion than the recurrent ones, and the gap widens considerably on the remaining patients (Table 4.3).

This is a reversal of the ordering in Table 4.1, where PV-CNN was the strongest model on the single fixed sagittal segment. It indicates that the recurrent architectures generalize far more reliably to unseen patients and to the additional frontal and transverse channels than the convolutional ones, and that phase-variable conditioning does not compensate for this once the backbone is a CNN.

4.3.4 Error Distribution Across Gait Phases

Figures 4.14 and 4.15 take the same held-out subject and break down the mean absolute error of each model within each of the seven gait phases, averaged over all nine joint–plane channels of the left and right leg respectively.

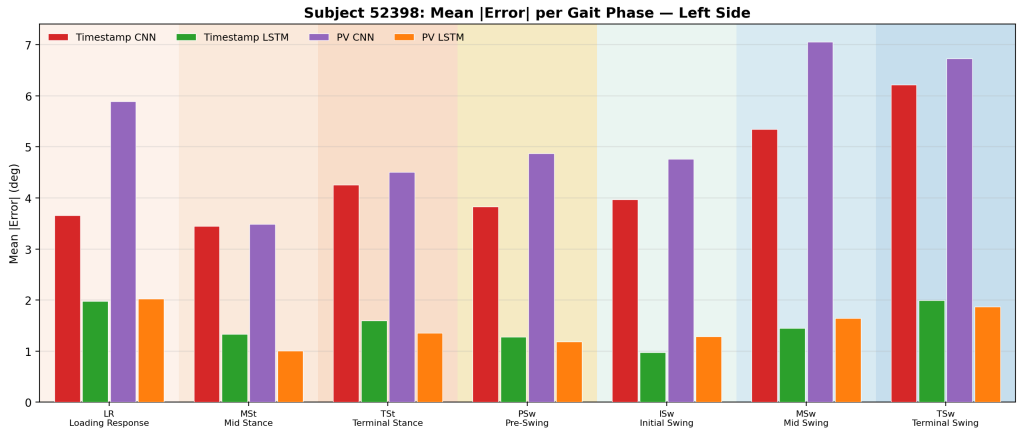


Figure 4.14. *Mean absolute error per gait phase, left side, for the held-out CP subject of Figure 4.12.*

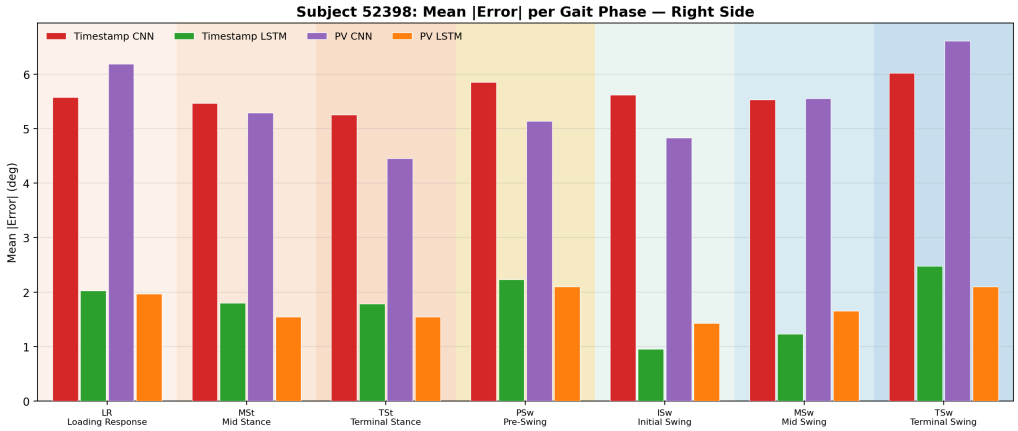


Figure 4.15. *Mean absolute error per gait phase, right side, for the same held-out CP subject.*

This phase-resolved breakdown confirms that the gap between LSTM-based and CNN-based models seen in Figures 4.12 and 4.13 is not an artifact of averaging: TS-LSTM and PV-LSTM keep mean errors below roughly 2.5° in every one of the seven phases on both legs, while TS-CNN and PV-CNN remain at 4–7.5° throughout, with the largest CNN errors occurring in Pre-Swing and Terminal Swing – the phases bordering the swing-to-stance transition. This reinforces that rapid kinematic transitions around toe-off remain the most difficult region of the cycle to predict, independent of conditioning scheme.

4.3.5 Subject-Specific Failure Modes

Aggregate, phase-binned errors of the kind shown above can still hide clinically relevant deviations in a single joint. Figure 4.16 isolates the left knee sagittal angle for the same held-out subject to illustrate this.

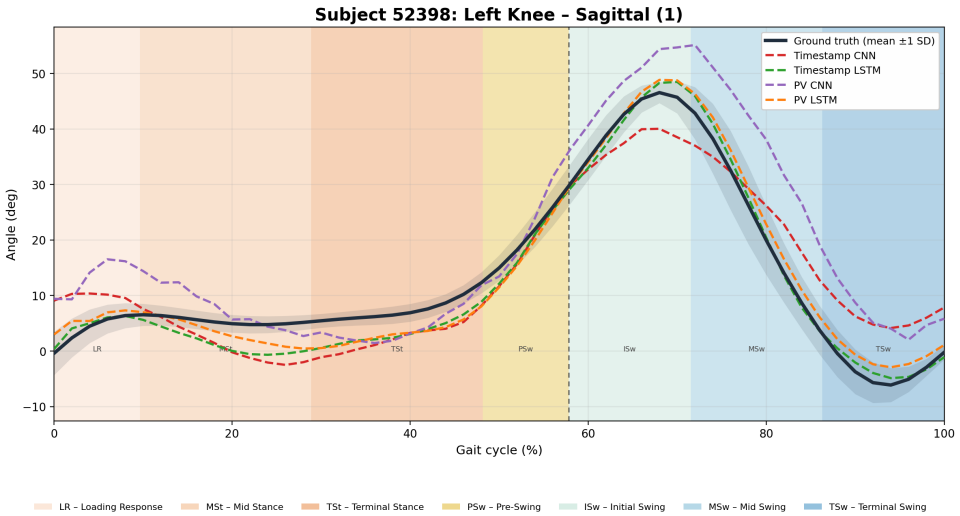


Figure 4.16. *Left knee, sagittal plane, stride-averaged prediction vs. ground truth for the held-out CP subject. The ground truth shows a markedly reduced knee flexion/extension range during stance, characteristic of a stiff-knee or crouch-type gait pattern.*

This subject's knee remains close to 0°–10° throughout stance, far below the 15°–20° stance-phase flexion typically reported for unimpaired gait. TS-CNN fails to capture this reduced range of motion, instead predicting a stance-phase flexion bump of roughly 15°–18° that is closer to a typical gait pattern than to this subject's own measured kinematics; PV-CNN shows a smaller but similar overshoot during terminal stance. Both LSTM-based models, in contrast, reproduce the suppressed stance-phase flexion almost exactly. This suggests that the convolutional models, lacking recurrent memory of the subject's own recent kinematics within the input window, fall back on a more "average" gait pattern learned from the training distribution, whereas the recurrent models adapt more readily to atypical, subject-specific joint ranges.

4.3.6 The Median Case

Subject 52880 sits at the median of the test patients, with PV-LSTM achieving R^2 = 0.80 against an average of 0.55 over all sixteen. Figure 4.17 shows the stride-averaged trajectories for the left side and Figure 4.18 the corresponding phase-resolved errors.

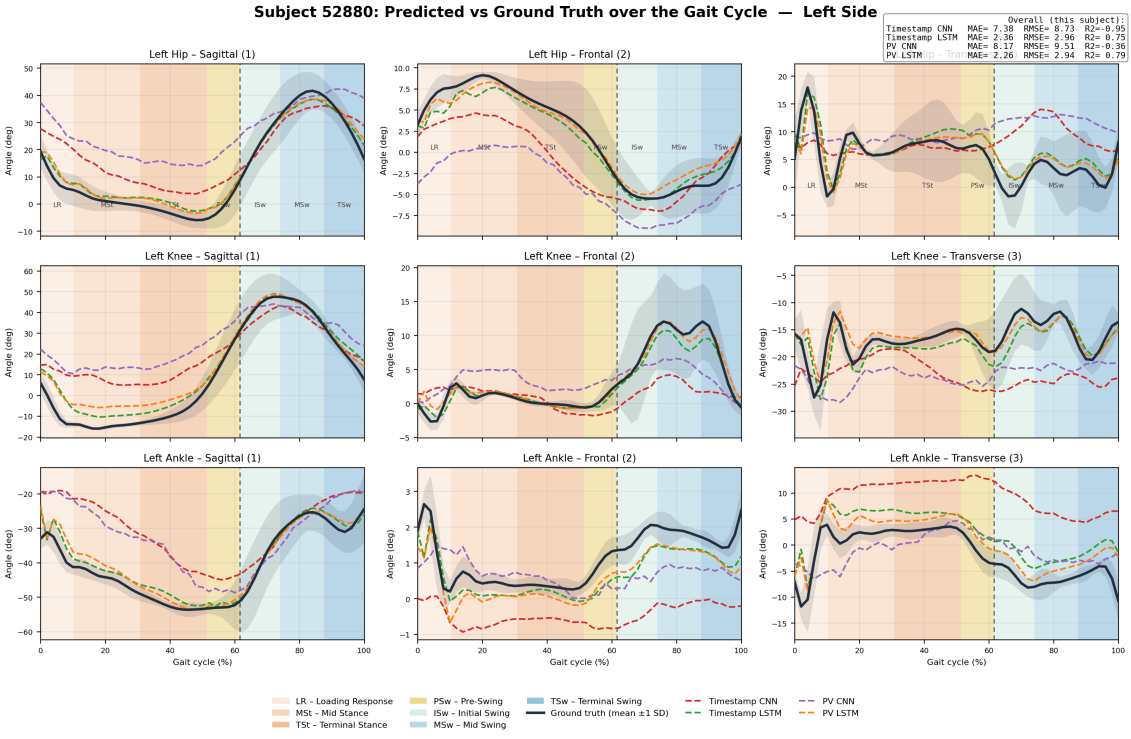


Figure 4.17. *Stride-averaged predicted vs. ground-truth trajectories over the gait cycle, left side, for held-out CP subject 52880, the median performer of the 16 test patients.*

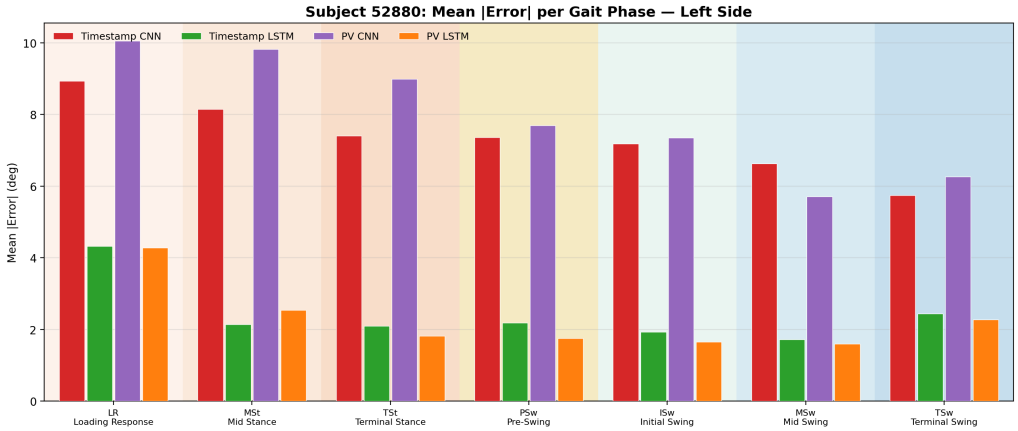


Figure 4.18. *Mean absolute error per gait phase, left side, for the median CP subject 52880.*

The qualitative picture matches the best case: both recurrent models follow the ground truth through all three planes, while both convolutional models depart from it substantially, with $R^2 = -0.95$ for TS-CNN and -0.36 for PV-CNN on this side. The recurrent models remain within roughly 2.4° mean absolute error and the convolutional models exceed 7°. The separation between architectures is therefore not an artifact of the particular subject chosen in Section 4.3, but the consistent behaviour across all the test patients.

4.3.7 The Worst Case

Subject 42043 is the least favourable of the 16 patients for every architecture, and is instructive precisely because of that. Figures 4.19 and 4.20 show the stride-averaged trajectories and phase-resolved errors for the left side.

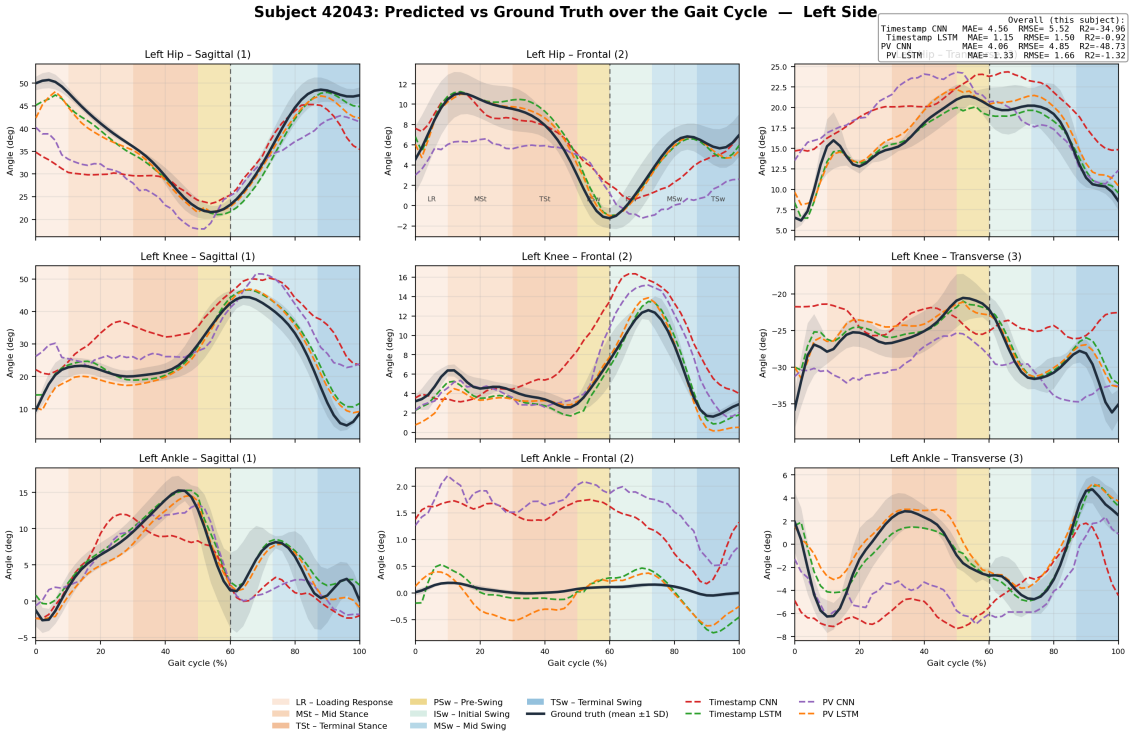


Figure 4.19. *Stride-averaged predicted vs. ground-truth trajectories over the gait cycle, left side, for held-out CP subject 42043, the worst case of the 16 test patients.*

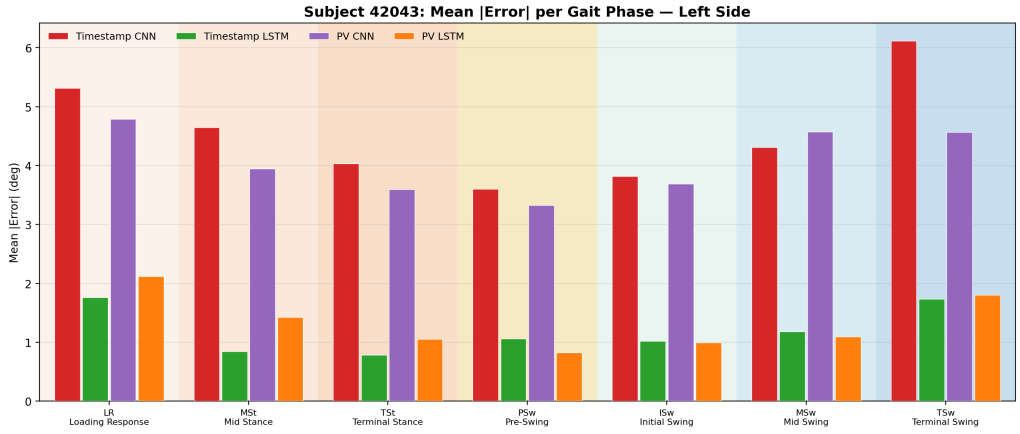


Figure 4.20. *Mean absolute error per gait phase, left side, for the worst-case CP subject 42043. Even here the recurrent models remain below roughly 2° in every phase.*

The convolutional models fail outright on this patient, with $R^2 = -19.77$ for TS-CNN and −28.80 for PV-CNN over the 18 channels, values that indicate predictions considerably worse than simply returning the subject's mean posture. The recurrent models also record their poorest coefficients of determination here (−0.29 and −0.63), yet their absolute errors remain close to their average across all test patients: PV-LSTM achieves 1.58° mean absolute error, and Figure 4.20 shows it staying below roughly 2° in every gait phase. As discussed in Section 4.3.1, this apparent contradiction arises because this patient's joint excursion is unusually small, so the variance normalisation in R^2 penalises even accurate predictions. The practical conclusion is that the recurrent models degrade

gracefully across the test patients, whereas the convolutional models fail catastrophically on the harder patients.

4.3.8 Comparison with a Typical-Gait Subject

To separate generalization failure from a more fundamental architectural limitation, the same four models were also evaluated on a typical-gait subject, shown in Figure 4.21. This subject's trials were part of the bulk typical-gait training data and therefore do not constitute a held-out test, but the comparison is still informative.

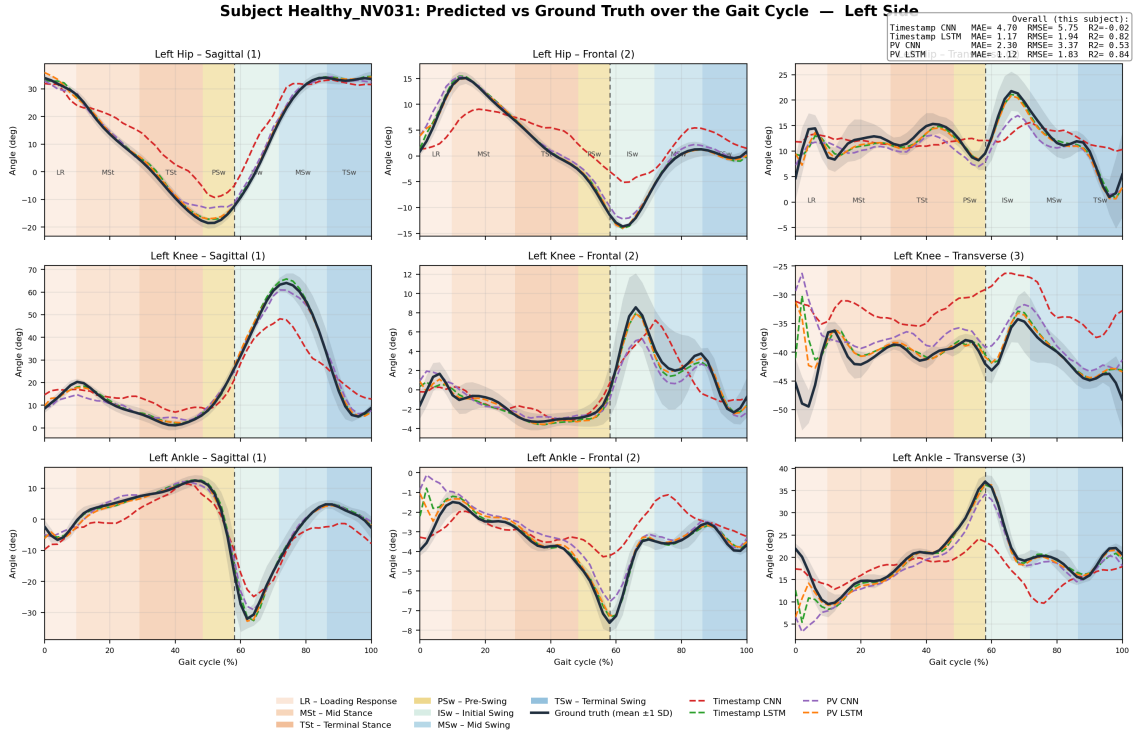


Figure 4.21. *Stride-averaged predicted vs. ground-truth trajectories over the gait cycle, left side, for a typical-gait subject (part of the training distribution, not held out).*

Even on this in-distribution subject, TS-CNN remains the weakest model ($R^2 = -0.02$), while PV-CNN improves substantially ($R^2 = 0.53$) but still trails both LSTM-based models ($R^2 = 0.82$ and 0.84). Since this subject was not held out, the persistent underperformance of TS-CNN cannot be explained by generalization to an unseen subject alone; it points instead to a more structural limitation of the purely time-indexed convolutional formulation. Phase-variable conditioning meaningfully narrows this gap for the convolutional backbone (TS-CNN to PV-CNN), but recurrence appears to be the dominant factor for robust tracking across the full 18-channel, multi-plane output space studied in this section.

It should be emphasized that because this subject's strides formed part of the typical-gait training data, these values are a sanity check on the models' behaviour within the training distribution and not a measure of generalization. They are not directly comparable with the held-out results of Table 4.2 and are not included in any aggregate statistic

reported in this chapter.

4.3.9 Clinical Weighting of the Three Anatomical Planes

Extending the models to all three anatomical planes broadens their descriptive scope, but the three planes are not equally reliable as measurement channels, and the results of this section should be weighted accordingly. This consideration was raised during discussion of the predictions with the clinical staff at ELEPAP who supervised the data collection, and it reflects a well-documented property of marker-based clinical gait analysis rather than a limitation particular to the models developed here.

Sagittal-plane kinematics are the most dependable of the three. The joint excursions are large, averaging approximately 47° per channel across the test patients, so the uncertainty introduced by marker placement and soft-tissue movement is small in comparison to the signal being measured. Sagittal-plane measurements are correspondingly the most repeatable between sessions and between observers, and they carry the greater part of the weight in clinical interpretation and surgical decision-making. The flexion–extension patterns of the hip, knee and ankle are what characterise the crouch, stiff-knee and equinus presentations discussed in Section 4.3.5, and they are the quantities an assistive device would primarily need to track.

Transverse-plane kinematics sit at the opposite end of this scale. Axial rotations are measured from the relative orientation of segment coordinate frames whose transverse axes are themselves defined by marker placement, so a small error in placing a marker translates directly into an offset in the reported rotation. Soft-tissue movement over the thigh and shank contributes further error of comparable magnitude. A transverse-plane trace may therefore reflect, in part, the measurement configuration rather than the patient's actual motion, and this is generally recognised as the least reliable of the three planes in routine clinical gait analysis.

This ordering is visible in the model errors themselves. Table 4.4 reports the mean absolute error of each model separately by plane, together with the mean range of motion of the channels in that plane. Expressed in degrees the transverse-plane error is the smaller of the two, but relative to the motion actually present it is the largest: for PV-LSTM the error amounts to roughly 6% of the sagittal range and 6% of the frontal range, but 10% of the transverse range. The same pattern holds for all four models.

Model	Sagittal	Frontal	Transverse
TS-LSTM	2.95	0.95	2.69
PV-LSTM	2.87	0.95	2.48
TS-CNN	8.07	2.96	7.32
PV-CNN	8.99	3.25	7.43
Mean range of motion	47.5	15.9	23.8

Table 4.4. *Mean absolute error in degrees by anatomical plane, averaged over the 16 held-out CP patients, with the mean range of motion of the channels in each plane for reference. Relative to the available range of motion, the transverse plane is the least accurately predicted for every model.*

Two consequences follow for the interpretation of this chapter. First, the aggregate figures of Table 4.2 weight all eighteen channels equally, which is appropriate as a measure of overall modelling capability but does not correspond to clinical importance; a reader assessing the models for clinical use should give greater weight to the sagittal-plane results. Second, the transverse-plane predictions are best regarded as indicative of a trend rather than as quantitatively reliable estimates, since the ground truth against which they are evaluated carries a comparatively high measurement uncertainty of its own.

A qualification applies to the foot progression angle. Although it is a transverse-plane quantity, it is among the most clinically used descriptors of the plane, since in-toeing and out-toeing are common presentations in cerebral palsy and directly inform treatment decisions. It is also more robust than the segmental axial rotations discussed above, being expressed relative to the direction of walking rather than to a segment axis defined by marker placement. The ELEPAP recordings include foot progression as a separate measured channel, but it is not among the eighteen joint angles predicted by the models developed here, which cover the hip, knee and ankle in each plane. Including it is a natural and clinically well-motivated extension of the present formulation, and is noted as such in Chapter 6.

4.4 Summary of Findings

Two evaluation regimes were used in this chapter, and they do not rank the four models identically. On the single fixed sagittal-plane CP segment (Table 4.1 and the preceding sections of this chapter), all four models appear closely matched and PV-CNN attains the lowest mean error. That result does not survive a broader test. Once the same four architectures are re-evaluated across all 16 held-out CP patients and all three anatomical planes (Table 4.2), the models separate primarily by *architecture* rather than by temporal representation.

The dominant finding is the gap between the recurrent and convolutional backbones. Both LSTM-based models hold mean absolute error near 2.1° and achieve a positive R^2 on 13 to 15 of the 16 patients, whereas both CNN-based models exceed 6° and are negative on 13 to 14 of them, meaning that on most unseen patients their predictions explain less variance than a constant would. This separation is roughly threefold in absolute

error and is consistent across the best, median and worst cases examined in Sections 4.3 to 4.3.7.

The effect of phase-variable conditioning on one-step-ahead accuracy is by comparison small. Within the recurrent architecture, PV-LSTM attains the lower mean absolute and root-mean-square error and is the better of the two on 11 of the 16 patients, while TS-LSTM attains a marginally higher mean R^2 (0.56 against 0.55) and is positive on more subjects. Since this difference is an order of magnitude smaller than the between-subject standard deviation, the two recurrent models are best described as comparable in accuracy, with PV-LSTM slightly ahead on error magnitude. Within the convolutional architecture it is TS-CNN that attains the better accuracy of the two across the test patients.

Accuracy under teacher forcing, however, measures only one of the two properties a gait predictor must have. Because that mode re-anchors the model to the measured trajectory at every step, it cannot reveal whether a model holds its timing when it must run on its own output, which is the condition under which any such model would actually be deployed. Evaluated that way, the ordering changes: the recursive rollouts of Section 4.2.3 and the closed-loop execution results of Chapter 5 show both phase-variable models preserving stride-to-stride alignment over extended runs, while both timestamp models accumulate phase drift, to the point that the timestamp models eventually diverge far enough for the controller to halt. This holds for both backbones, including the convolutional one, where PV-CNN is markedly the more stable of the two despite its slightly higher per-step error.

Therefore, the PV-LSTM is recommended as the best model of this study, as the only architecture that is strong on both axes: it achieves the lowest prediction error across the test patients and, as Chapter 5 shows, the most stable behaviour during extended execution. The two contributing mechanisms are largely independent and act on different properties, as summarized in Table 4.5. Recurrence is what drives *generalization accuracy* to unseen patients, and it does so decisively, accounting for the threefold separation in error. Phase-variable conditioning is what drives *temporal stability* under recursive execution, and it does so regardless of backbone, at the cost of little or no per-step accuracy. Neither property alone is sufficient for a deployable predictor, and PV-LSTM is the only combination that provides both.

Model	**Accuracy (Ch. 4)**	**Stability (Ch. 5)**
PV-LSTM	Best: MAE 2.10°, $R^2 = 0.55 \pm 0.45$	Most stable
TS-LSTM	Comparable: MAE 2.20°, $R^2 = 0.56 \pm 0.38$	Drifts over time
PV-CNN	Poor: MAE 6.56°, $R^2 = -3.29 \pm 6.96$	Stable
TS-CNN	Poor: MAE 6.12°, $R^2 = -2.74 \pm 5.01$	Least stable

Table 4.5. *The two properties are driven by different design choices. Recurrence (LSTM) is decisive for accuracy on unseen patients; phase-variable (PV) conditioning is decisive for stability under recursive execution, for either backbone. PV-LSTM is the only model strong on both.*

The knee joints remain the most challenging to predict, which may be attributed to

their biomechanical complexity and the rapid velocity changes that occur around the toe-off event and during the mid swing phase. In addition, elevated prediction errors are consistently observed near the beginning and end of each stride across all joints and neural network architectures. These stride-to-stride discrepancies suggest that human gait may not be adequately represented as a purely smooth and continuous process. Instead, human locomotion can be viewed as exhibiting hybrid characteristics, combining continuous joint motion with discrete events such as heel strike and toe-off that introduce abrupt changes in system dynamics. Such transitions are difficult to capture using fixed-length, data-driven prediction models, particularly in recursive prediction settings.

Phase variable conditioning enhances temporal alignment and execution stability by connecting model inputs to the advancement of the gait cycle instead of fixed time. When integrated with a recurrent architecture, as in PV-LSTM, this approach offers the best balance between immediate accuracy and long-term resilience found in this study, although the results across patients show that the accuracy benefit over the equivalent timestamp model is marginal and that recurrence is what secures generalization to unseen patients. The outstanding prediction errors, particularly around stride boundaries and in the knee joints, together with the wide spread in performance across patients evident in Table 4.3, suggest that additional exploration is necessary, especially regarding modeling methods that clearly incorporate discrete gait events or hybrid dynamics.

Chapter 5

Visualization of Neural Network Gait Predictions using ROS2

5.1 ROS2 Framework

Utilizing ROS2 allows for modification of only one software architecture for gait pattern customization, removing the necessity for software duplication among various system components. This method offers a unified and modular system for organizing, handling, and displaying gait patterns in the simulated exoskeleton setting.

ROS2 is a middleware that is open-source and intended to facilitate the creation of distributed robotic systems, offering enhanced robustness, scalability, and real-time capabilities compared to the earlier version. Despite not being a real-time operating system, ROS2 is explicitly crafted to work with real-time execution frameworks and offers a versatile communication structure for managing independent software processes known as nodes. The modular architecture, vibrant community, and vast ecosystem of tools have resulted in broad usage in both academic research and industrial robotics settings.

A ROS2-based system is organized around a **workspace**, which typically contains the folders *src*, *build*, *install*, and *log*. The *src* directory contains user-defined **packages**, which are the fundamental organizational units in ROS2. The *build* and *install* directories are generated during compilation and contain compiled binaries, libraries, and runtime resources required for execution. Each package may include one or more **nodes**, configuration files (e.g., YAML), robot descriptions (URDF), and launch files, allowing related functionality to be grouped in a modular and reusable manner [35], [36].

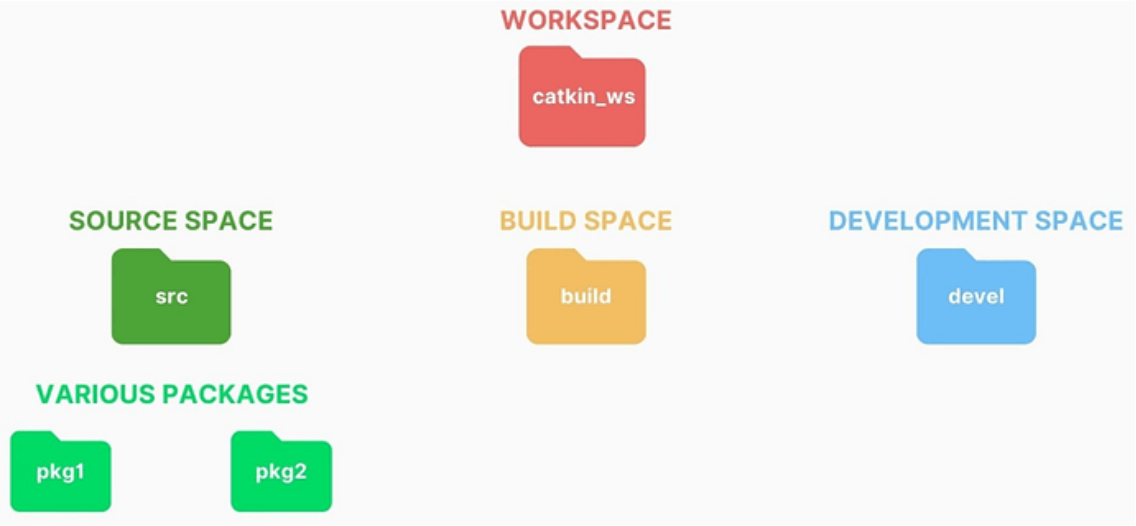


Image 5.1: *ROS2 workspace.*

In ROS2, nodes interact through a decentralized communication framework grounded in the Data Distribution Service (DDS) standard. The primary communication method is via **topics**, which utilize a publish–subscribe model for ongoing and asynchronous data sharing. Information sent through topics is contained in strongly typed **messages**, outlined in `.msg` interface files. Typical message types utilized in this research comprise `sensor_msgs/JointState` for feedback on joints and `trajectory_msgs/JointTrajectory` for executing commands at the joint level [37], [38].

For interactions that require synchronous request–response behavior, ROS2 provides **services**, defined using `.srv` files. Services are typically used for tasks such as controller configuration, parameter updates, or triggering specific system actions [39]. In addition, ROS2 introduces **actions**, which are designed for long-running goals that require feedback during execution, such as trajectory tracking or motion planning.

To manage the startup and coordination of complex robotic systems, ROS2 employs **launch files** written in Python. These launch files enable the simultaneous initialization of multiple nodes, parameter configurations, and namespaces, providing fine-grained control over system execution [40]. Robot models are described using the **Unified Robot Description Format (URDF)**, which specifies the kinematic structure, joints, links, and inertial properties of the robot [41].

For control applications, ROS2 integrates tightly with the **ros2_control** framework. This framework provides a standardized interface between high-level controllers and low-level hardware or simulation backends. Controllers such as position, velocity, and effort controllers are managed through a centralized `controller_manager`, while the `hardware_interface` abstraction enables seamless switching between simulated and physical hardware without modifying the controller logic [42].

A key component in the ROS2 ecosystem is **rosbag2**, which enables the recording and playback of topic data via the DDS communication layer. This feature is especially useful for troubleshooting, conducting analyses without internet, and assessing the performance of control and prediction algorithms [43]. Simulation assistance is offered via close inte-

gration with the **Gazebo** simulator, allowing for physics-driven operation of robot models in virtual settings. Gazebo provides access to simulated joint states and processes control commands via ROS2-compatible plugins, using identical interfaces to actual robotic hardware, thereby enabling swift prototyping and validation [44].

While ROS2 offers a rich and extensible ecosystem, the structure of a robotic workspace can vary depending on application complexity. Some systems may include additional packages for controllers, visualization, or data processing. Nevertheless, the components described above form the core building blocks for developing, simulating, and deploying robotic systems using ROS2. An overview of these concepts is illustrated in Image 5.2.

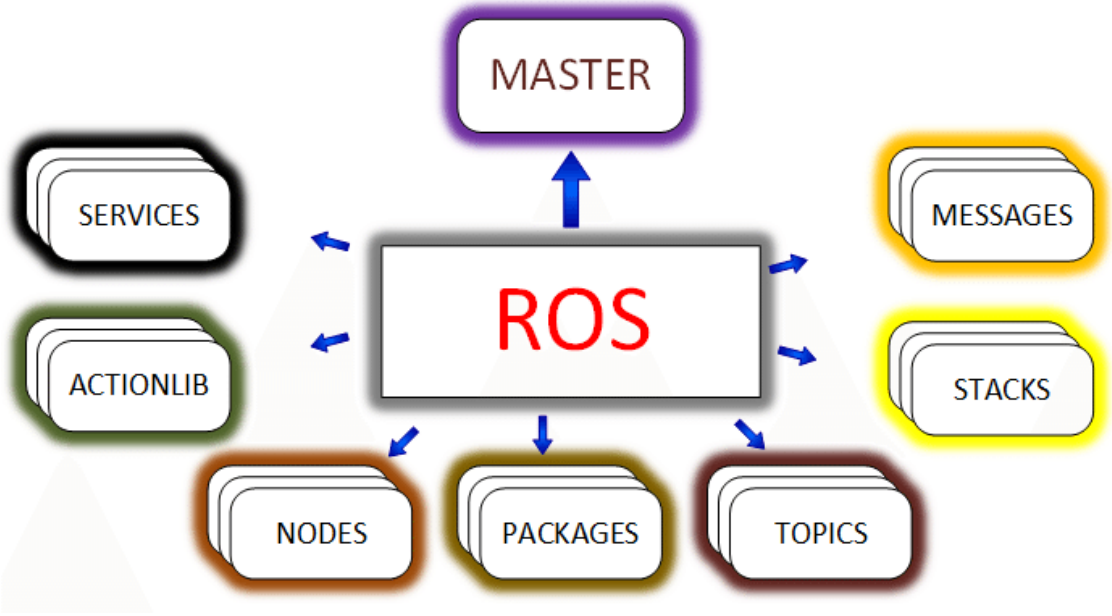


Image 5.2: *The core concepts of ROS2 Framework.*

5.1.1 Motivation for Using ROS2

While the original Robot Operating System (ROS1) has seen extensive application in robotic research, ROS2 was chosen for this project because of its contemporary communication framework, enhanced modularity, and sustained support for simulation and hardware integration. These features render ROS2 especially appropriate as a cohesive platform for merging learning-driven gait prediction models with robotic simulation contexts. A fundamental difference between ROS1 and ROS2 is found in the communication middleware they use. ROS2 relies on the Data Distribution Service (DDS) standard, allowing decentralized, peer-to-peer communication among nodes without depending on a central master. This design enhances resilience and scalability while streamlining system integration, especially when managing various elements like neural network inference, trajectory publishing, and simulation feedback.

Another important advantage of ROS2 is its support for configurable Quality of Service (QoS) policies. QoS settings allow control over message reliability, latency, and buffering behavior, which is beneficial when streaming joint trajectories and monitoring joint state feedback in simulation. Although real-time guarantees are not required in this work, QoS

configurability enables predictable and stable data flow during visualization and playback of gait trajectories.

5.2 Gazebo Simulation

To simulate the exoskeleton, a combination of a *ROS2 Control* node and Gazebo is used. In this application, ROS Control is used to establish a controller manager to subscribe to each joint position topic (a joint position interface to store the simulated joint positions). To replicate the motion of the exoskeleton, the joint position commands published from the Joint Publisher Node are transmitted to each joint's designated position controller in Gazebo. The ROS2 command *ros2 bag* can then be used to save the simulated joint position data for comparison and plotting.

The exoskeleton model employed in the Gazebo simulation is a simplified representation inspired by the *March VI* exoskeleton model [45]. While the overall kinematic structure, joint arrangement, and anatomical proportions of the lower limbs are preserved, the model does not replicate the full mechanical, actuation, or sensing complexity of the *March* system. The physical configuration, dimensions and specifications of the simplified model are shown in Table 5.1. The parameters are extracted from the MARCH VI URDF and YAML simulation description. Only the principal moments of inertia are reported for clarity. The 3D simulated model of the exoskeleton is also presented in Image 5.3.

Link	**Base**	**Hip**	**Shank**	**Foot**
Length [m]	0.145	0.410	0.390	0.243
Mass [kg]	8.686	8.161	4.997	0.789
I_{xx} [kg m^2]	0.3035	1.6878	0.5672	0.00389
I_{yy} [kg m^2]	0.2393	1.6845	0.5612	0.00659
I_{zz} [kg m^2]	0.1182	0.2014	0.0625	0.00630
Joint Maximum Limits		**Hip**	**Knee**	**Ankle**
Flexion	–	100	100	20
Extension	–	20	5	15

Table 5.1. *Specifications of the simulated exoskeleton, including the base backpack unit and lower-limb links from the March 6 Exoskeleton Simulated Model. The Ixx, Iyy , Izz, are the moment of inertia tensor's diagonal values.*

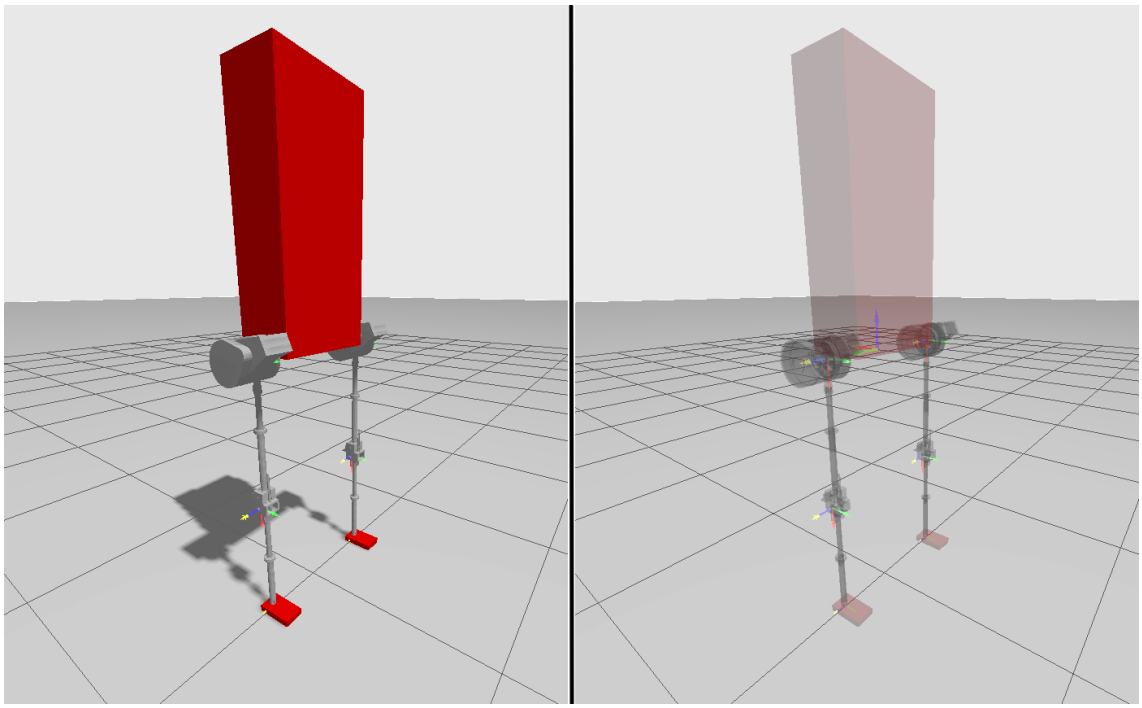


Image 5.3: *Simulated model of the exoskeleton in gazebo with view of transformation frames..*

5.2.1 Simulation Setup

The Gazebo simulator is used to execute a physics-based simulation of the exoskeleton model under closed-loop control. Gazebo provides a rigid-body dynamics engine that resolves joint motion, gravity, and contact interactions at each simulation step, allowing the evaluation of joint-level control commands in a dynamically consistent environment. In the proposed framework, Gazebo operates as the execution layer of the control pipeline. Joint position commands generated by the ROS2 control stack are applied to the simulated actuators, while joint state feedback is continuously computed by the physics engine and published back to ROS. This bidirectional data exchange enables real-time monitoring of tracking performance and stability during gait execution.

The simulation is configured to prioritize numerical stability and repeatability. Fixed-step physics updates are used to ensure deterministic behavior across simulation runs, which is particularly important when evaluating the effects of neural network generated trajectories. Although Gazebo does not model human muscle dynamics or compliant soft tissue interaction, its rigid-body formulation is sufficient for assessing joint kinematics, controller behavior, and constraint enforcement.

5.2.2 Kinematic Constraints and Joint Modeling

Kinematic constraints are explicitly enforced within the Gazebo simulation through the joint definitions specified in the URDF model. In an earlier version of this simulation, every lower-limb joint of the exoskeleton was represented as a single-degree-of-freedom revolute joint restricted to the sagittal plane, since driving all three anatomical planes with

the same K_p and K_d gains at every joint caused the model to become unstable and fall. For that reason, that earlier version of the visualization only reproduced the sagittal-plane output of the neural network models.

To take advantage of the 18-channel, all-planes networks introduced in Section 4.3 of Chapter 4, the hip and ankle joints were subsequently upgraded to multi-degree-of-freedom joints. Each is implemented as a short series of single-axis revolute joints, one per anatomical plane (sagittal, frontal, transverse), connected by small, massless intermediate links, which is the standard way of approximating a universal or spherical joint within a URDF description. Each of these additional revolute axes is driven independently, using the same K_p and K_d gains reported in Table 5.2 for the corresponding joint, rather than requiring separate per-plane tuning. The knee joint, in contrast, was left as a single-degree-of-freedom sagittal revolute joint, since its physiological range of motion in the frontal and transverse planes is comparatively minor and largely constrained by the surrounding ligaments, so modeling it as a single hinge remains an adequate approximation for gait visualization purposes. With this change, the hip and ankle joints of the exoskeleton reproduce the full 18-channel (sagittal, frontal, and transverse) predictions of the neural network models directly, rather than only their sagittal-plane component, resulting in a visibly more anatomically natural walking motion in simulation.

Joint limits are defined to constrain the allowable range of motion for each joint, and, for the hip and ankle, for each of their sagittal, frontal, and transverse revolute axes individually, reflecting both biomechanical feasibility and mechanical restrictions of the exoskeleton structure. The frontal and transverse limits for the hip and ankle are enforced in the URDF in the same way as the sagittal limits already reported in Table 5.1, but are omitted from that table for brevity. These limits are enforced directly by the Gazebo physics engine, ensuring that commanded trajectories cannot drive the system into physically unrealistic configurations. As a result, the simulation naturally rejects unsafe joint commands before they propagate through the control loop. In addition to positional limits, joint-level damping and friction parameters are introduced to improve numerical stability and reduce oscillatory behavior during trajectory tracking. These parameters play an important role when tracking rapidly varying reference trajectories, as they prevent excessive overshoot and contribute to smoother joint motion. The combined use of revolute joint constraints, range limits, and passive joint dynamics ensures that the simulated exoskeleton exhibits predictable and stable kinematic behavior during gait execution, allowing meaningful evaluation of the control architecture.

Lastly, for the phase variable models, the foot contact needs to be measured, so the model has valid inputs for phase consistency. For that reason two contact sensors were added shown in Image 5.4 on the bottom of the feet of the model using the Gazebo *Contact Sensor Plugin* [46], which gives true or false outputs when its in stance phase (touching the floor) or in swing phase (mid-air), respectively.

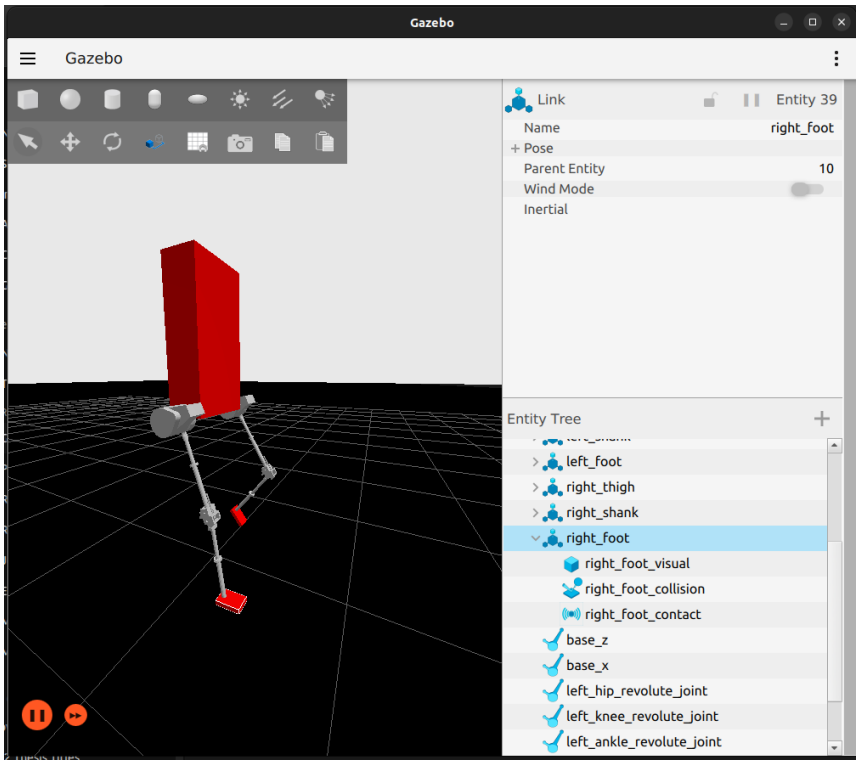


Image 5.4: *Screenshot showing feet of the Gazebo model and foot contacts on Entity Tree.*

5.3 Control Architecture

The control and communication architecture seen in Figure 5.1 consists of the following main modules: 1) the Joint Publisher Node, which is responsible for sending joint position commands to the entire model, and 2) the Gazebo Simulation, which visualizes the kinematics of the publisher on the model.

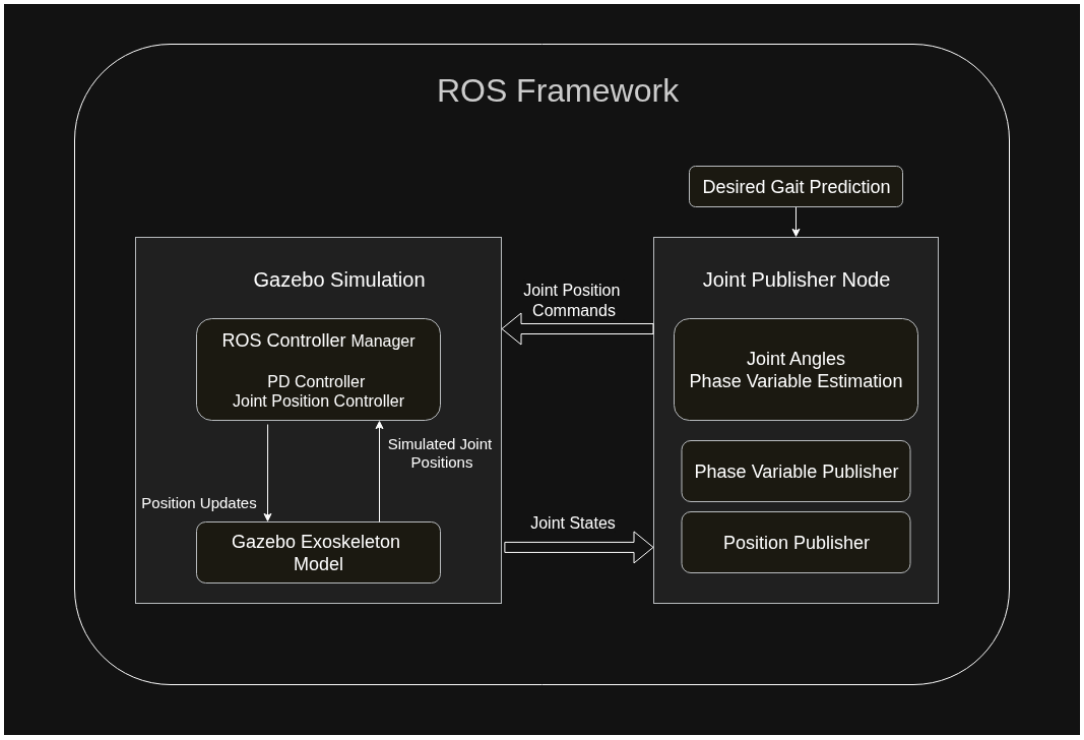


Figure 5.1. *The overall simulation architecture.*

On the right side of the architecture, the Joint Publisher Node is responsible for receiving predicted joint trajectories, estimating the continuous phase variable from hip angles and heel strike, and publishing the necessary commands. This node contains submodules that process incoming data and send updated joint angles and phase information to the simulation. On the left side of the figure, the Gazebo Simulation environment includes a ROS-integrated controller manager that executes position commands using a PD (Proportional-Derivative) control loop. The controller applies updates to the Gazebo Exoskeleton Model, allowing the joints to follow the published trajectories. In return, the simulated joint states are sent back to the joint publisher node, closing the feedback loop.

5.3.1 Integration with the `ROS2 Control` Framework

To ensure modularity, scalability, and compatibility with both simulation and future hardware deployment, the proposed exoskeleton control architecture is built upon the `ros2_control` framework [47]. This framework provides a standardized interface between high-level control algorithms and low-level actuation mechanisms, enabling a clean separation between control logic and hardware-specific implementations.

At the core of `ros2_control` lies the **controller manager** [48], which is responsible for loading, configuring, activating, and switching between controllers at runtime. In the presented simulation, the controller manager operates within the Gazebo environment and interfaces directly with the simulated exoskeleton model through a position-based hardware interface. This design allows the same control structure to be reused with minimal modifications when transitioning from simulation to a physical robotic system. The hardware abstraction layer provided by `ros2_control` exposes joint state interfaces (position,

velocity, and effort) and command interfaces that controllers can access independently of the underlying hardware. In the Gazebo simulation, these interfaces are implemented through a virtual hardware plugin that mirrors the behavior of real actuators. As a result, the controller logic remains agnostic to whether it is interacting with a simulated or physical device.

A position-controlled architecture was selected, where desired joint angles are published to the controller manager and tracked using internal Proportional-Derivative (PD) control loop expressed as:

$$\tau(t) = K_p \left(q_d(t) - q(t)\right) + K_d \left(\dot{q}_d(t) - \dot{q}(t)\right), \tag{5.1}$$

where $q_d(t)$ and $\dot{q}_d(t)$ denote the desired joint position and velocity, respectively, $q(t)$ and $\dot{q}(t)$ are the measured joint position and velocity, and K_p and K_d are the proportional and derivative gain matrices. Position control was chosen due to its inherent stability, simplicity, and suitability for early-stage validation of gait trajectories. While torque or impedance control may offer increased compliance and adaptability, position control provides a reliable baseline for evaluating the accuracy and smoothness of neural network-generated gait predictions. The controller manager executes each controller at a fixed update rate synchronized with the Gazebo simulation timestep. This ensures consistent interaction between the control commands and the physics engine, minimizing numerical instability and oscillatory behavior. Furthermore, the modular structure of `ros2_control` allows multiple controllers to coexist, enabling future extensions such as adaptive gain scheduling, impedance control, or hybrid position-torque strategies. The PD gains based on the *March exoskeleton model* shown in Table 5.2, the system achieves a balance between responsiveness and smoothness, ensuring reliable trajectory execution.

Joint	**Proportional Gain (K_p)**	**Derivative Gain (K_d)**
Hips	5000.0	100.0
Knees	5000.0	100.0
Ankles	500.0	10.0

Table 5.2. *PD Controller Gains Used in ROS2-Gazebo Controller. For the multi-degree-of-freedom hip and ankle joints (Section 5.2.2), the same gains are applied independently to each of the sagittal, frontal, and transverse revolute axes.*

5.3.2 Joint Publisher Node

To better understand the logic behind the visualization of predicted data, Figure 5.2 presents a detailed view of the ROS2 Node Architecture responsible for executing the predicted gait loop. This node plays a central role in bridging the high-level gait prediction model and the visualization within the Gazebo environment.

The Joint Publisher Node acts as the central execution unit that translates neural network gait predictions into executable joint commands for the simulated exoskeleton. At startup, the node loads cerebral palsy gait data, the trained Keras neural network model, and the associated data scaler used during training. The node loads the 18-channel, all-

planes version of the selected model (Section 4.3 of Chapter 4), rather than the sagittal-only, six-joint version used in the earlier simulation setup described in Section 5.2.2. Depending on the selected model formulation, either timestamp-based (TS-LSTM, TS-CNN) or phase-variable-based (PV-LSTM, PV-CNN), the node optionally computes continuous left and right phase variables from hip kinematics and toe-off events. These phase variables are used exclusively by the phase-variable networks to condition the prediction process.

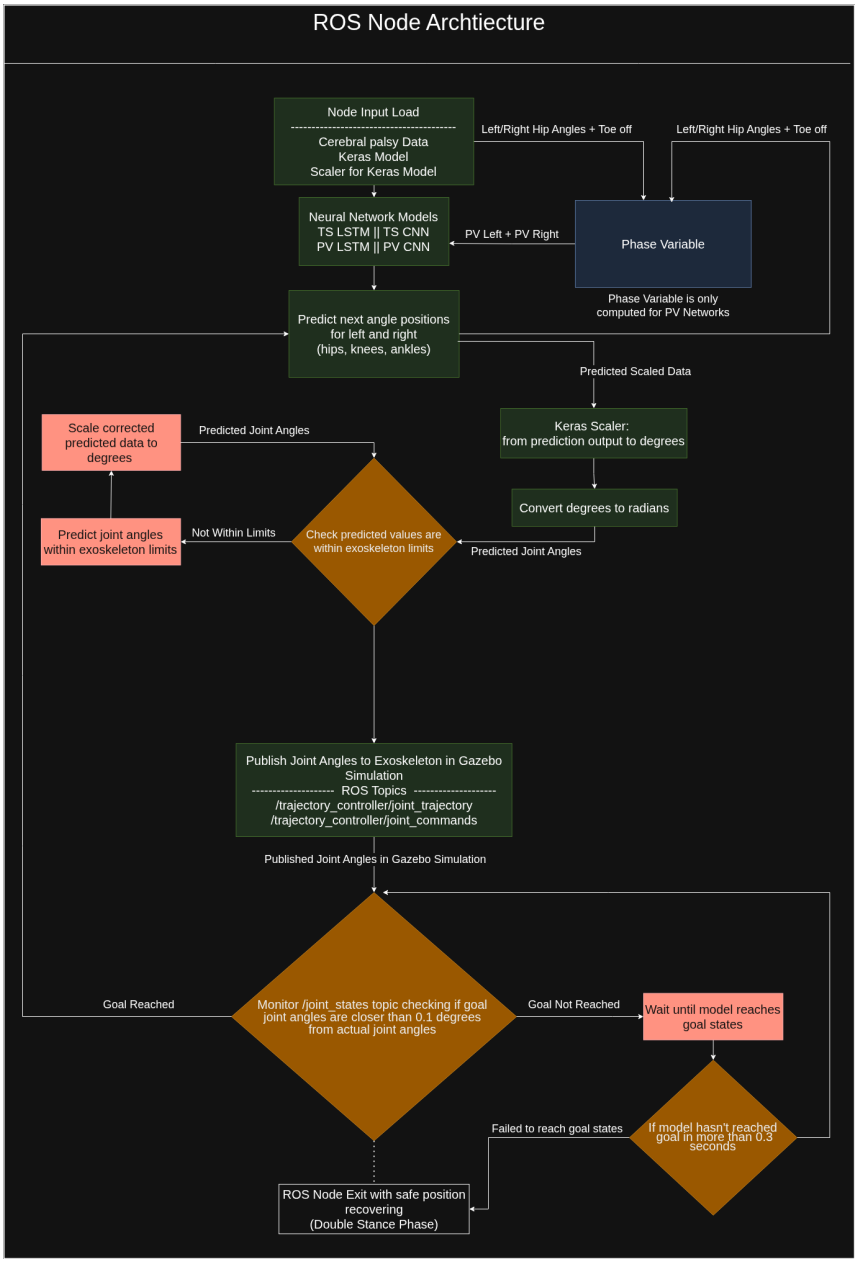


Figure 5.2. *Detailed diagram of ROS2 Node used for gait prediction and joint command publishing.*

Using the loaded input window, the neural network predicts the next set of 18 joint-angle channels (sagittal, frontal, and transverse planes) for the left and right hip, knee, and ankle in a scaled representation. The sagittal channel of each joint continues to drive

the knee's single-degree-of-freedom joint directly, while all three predicted channels of the hip and ankle are each mapped to the corresponding revolute axis of the multi-degree-of-freedom joint stack described in Section 5.2.2. The predicted outputs are subsequently transformed back to physical units using the inverse scaling operation, yielding joint angles in degrees. These predicted angles are then converted to radians to comply with ROS control conventions. Before being sent to the simulated exoskeleton, the node verifies that all predicted joint values lie within the predefined kinematic limits of the exoskeleton model. If violations are detected, corrective scaling is applied to ensure safe and physically feasible motion.

Validated joint trajectories are published to the Gazebo simulation through ROS messages associated with the `ros2_control` framework, such as `/trajectory_controller/joint_trajectory`. Then, the node publishes short multi-point trajectory segments at a fixed rate to ensure smooth temporal execution and continuity between successive predictions. During execution, the node continuously monitors the `/joint_states` topic to assess whether the simulated joints are converging toward the commanded positions. If the joint angles do not reach the desired targets within a predefined tolerance and time threshold, the node waits until convergence is achieved or initiates the safe exit procedure. In the event of failure or shutdown, the system transitions the exoskeleton to a predefined safe configuration corresponding to a double-stance phase, ensuring stable termination of the simulation.

In addition, this node functions as a visualization and integration layer, translating neural network outputs into executable reference trajectories for the Gazebo simulation. Safety and robustness considerations are incorporated through a controlled shutdown procedure, which returns all joints to a neutral position and deactivates the controllers upon termination. Within the scope of this thesis, the Joint Publisher Node is therefore used exclusively to animate and qualitatively assess the kinematic behavior of neural network generated strides. To ensure reliability, the node monitors the `/joint_states` topic in real time. If the published joint angles differ from the actual joint positions by more than 0.1 degrees, a fail-safe mechanism is triggered. The system then transitions into a safe recovery mode by reverting the robot to a double-stance position, a pose suitable for halting and resetting the simulation.

5.4 ROS2 Data Visualization and Results

The primary data visualization tool used is PlotJuggler [49], a ROS2-compatible time-series visualization utility that enables real-time and offline inspection of numerical data streams. PlotJuggler was used to monitor and compare joint-level signals published within the ROS2 ecosystem, specifically the desired joint trajectories generated by the controller and the corresponding joint states reported by the Gazebo simulation. Using PlotJuggler, joint position data for the hip, knee, and ankle joints of both legs were visualized (Images 5.5, 5.6 and 5.7) as synchronized time-series plots. This enabled direct qualitative assessment of trajectory tracking performance, temporal alignment between commanded and measured signals, and overall stability of the control architecture during

simulated walking.

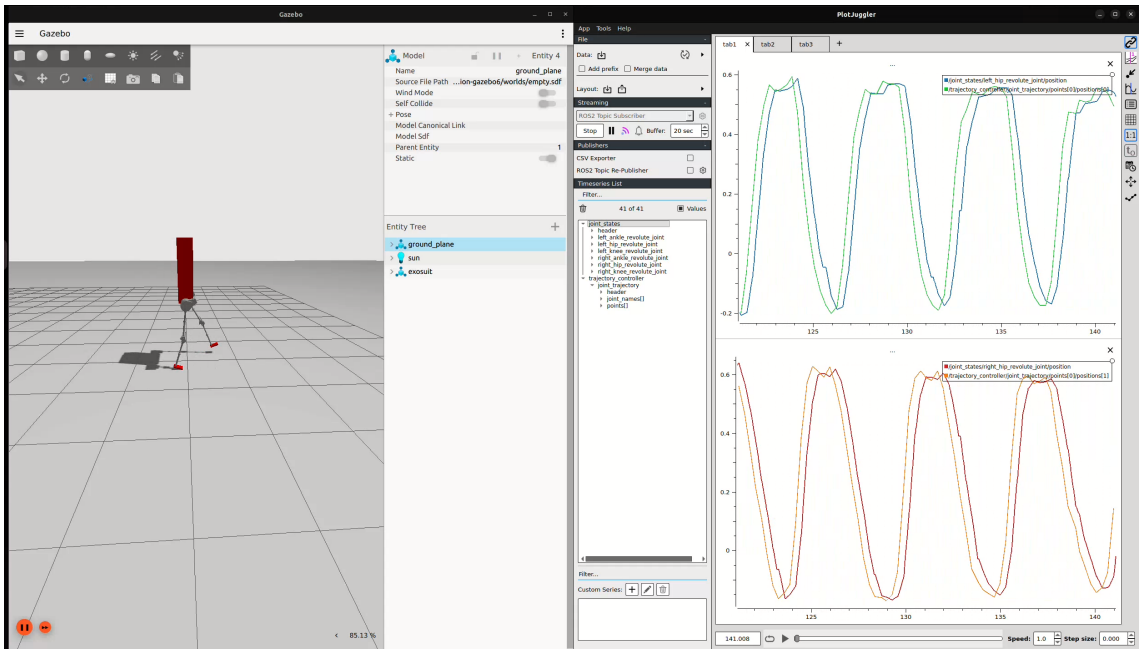


Image 5.5: *Gazebo - PlotJuggler Hips Visualization*
The left side shows the running Gazebo Sim and the right side shows data for the left and right hips from joint_states and the trajectory_controller.

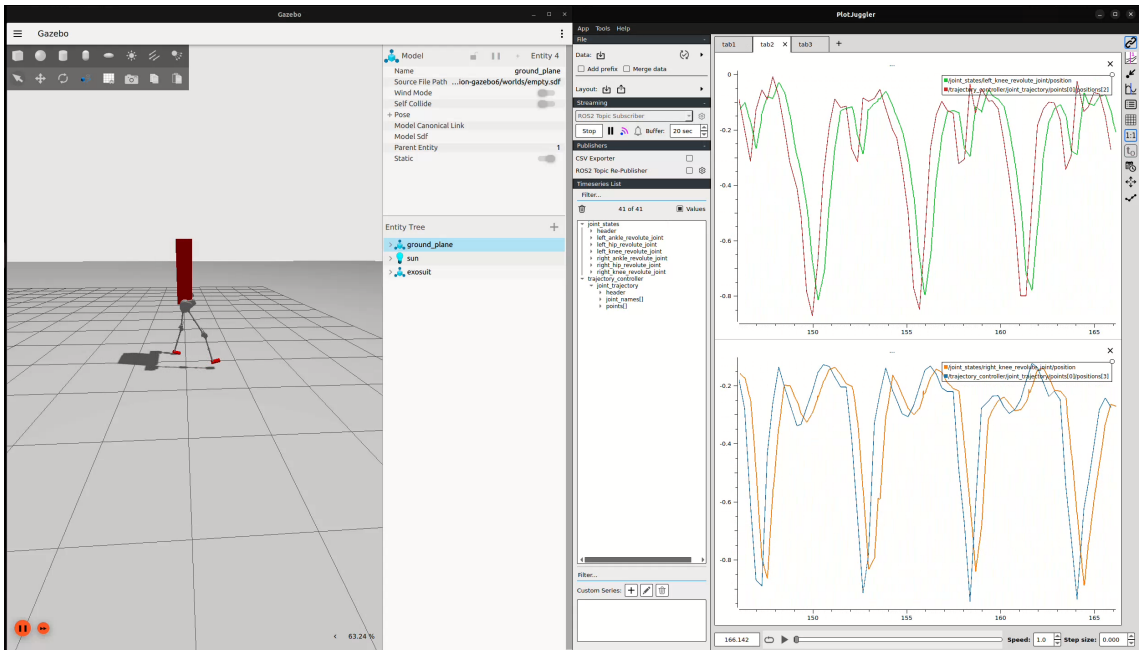


Image 5.6: *Gazebo - PlotJuggler Knees Visualization*
The left side shows the running Gazebo Sim and the right side shows data for the left and right knees from joint_states and the trajectory_controller.

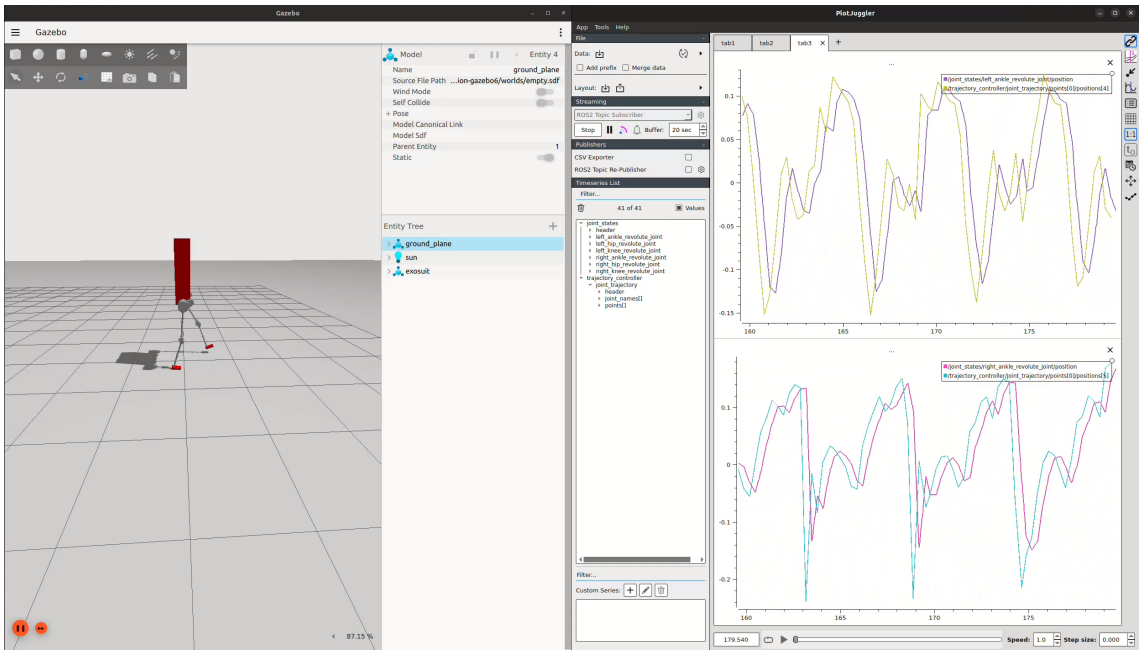


Image 5.7: *Gazebo - PlotJuggler Ankles Visualization*
The left side shows the running Gazebo Sim and the right side shows data for the left and right ankles from joint_states and the trajectory_controller.

In addition to real-time visualization using PlotJuggler, an offline analysis was conducted to directly compare the commanded joint trajectories generated by each neural network model. This analysis enables a consistent comparison across models under identical simulation and control conditions. Figures 5.3, 5.4, 5.5, and 5.6 present time-series comparisons between commanded and measured joint angles for the hip, knee, and ankle joints of both legs, where each model was running for 20 seconds. Across all figures, the blue curves represent the actual joint positions obtained from the Gazebo simulation (`joint_states`), while the orange curves correspond to the commanded joint positions produced by the neural network and tracked by the position controller. A close overlap between these curves indicates both accurate trajectory prediction and stable execution within the ROS2 control architecture.

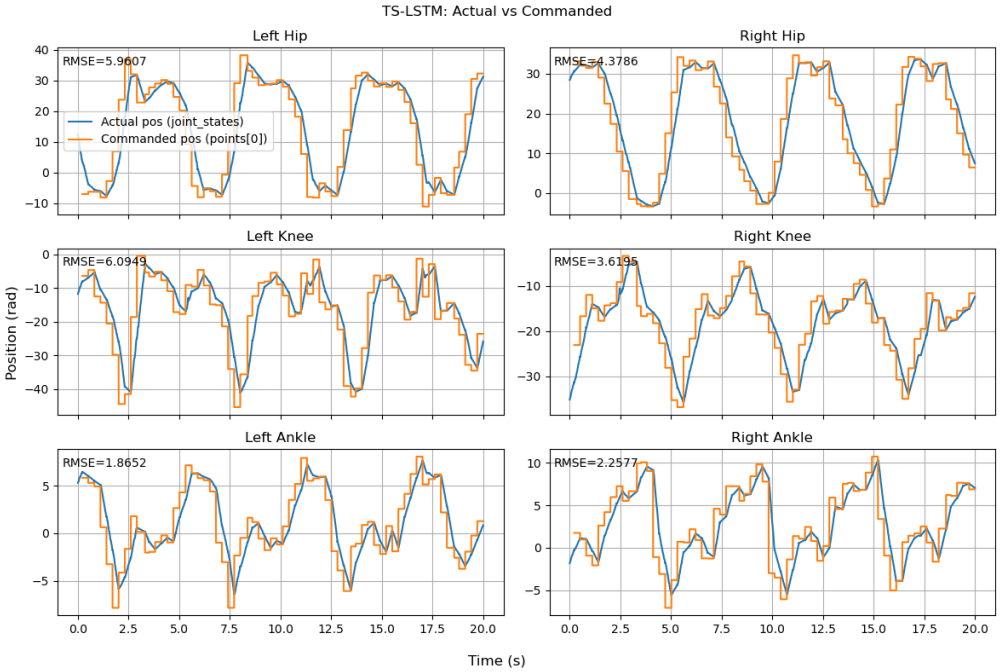


Figure 5.3. *joint_states vs commanded_joint_positions for TS LSTM Model*

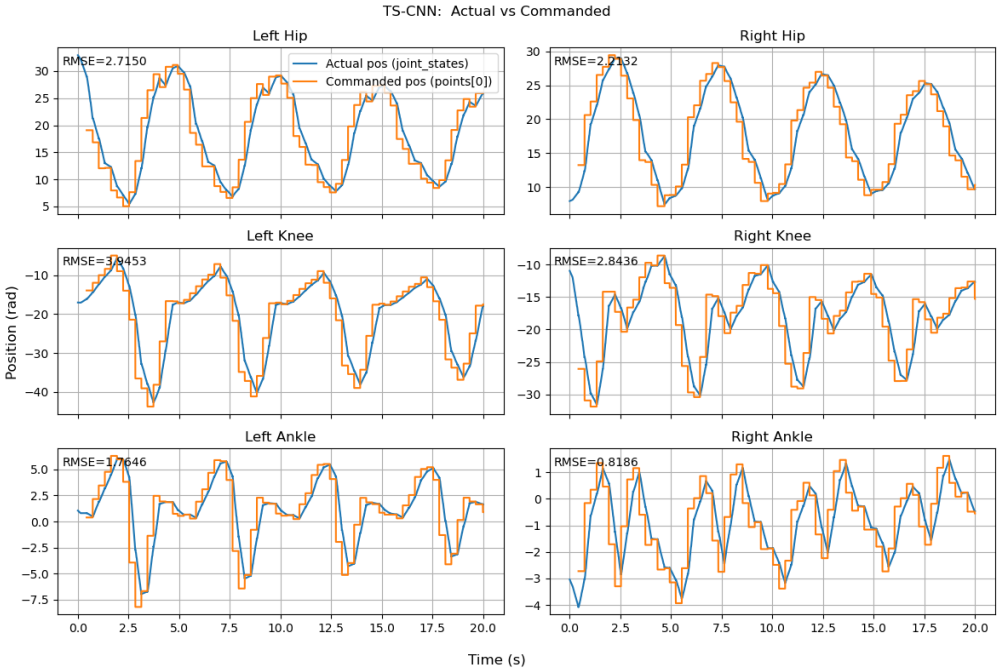


Figure 5.4. *joint_states vs commanded_joint_positions for TS CNN Model*

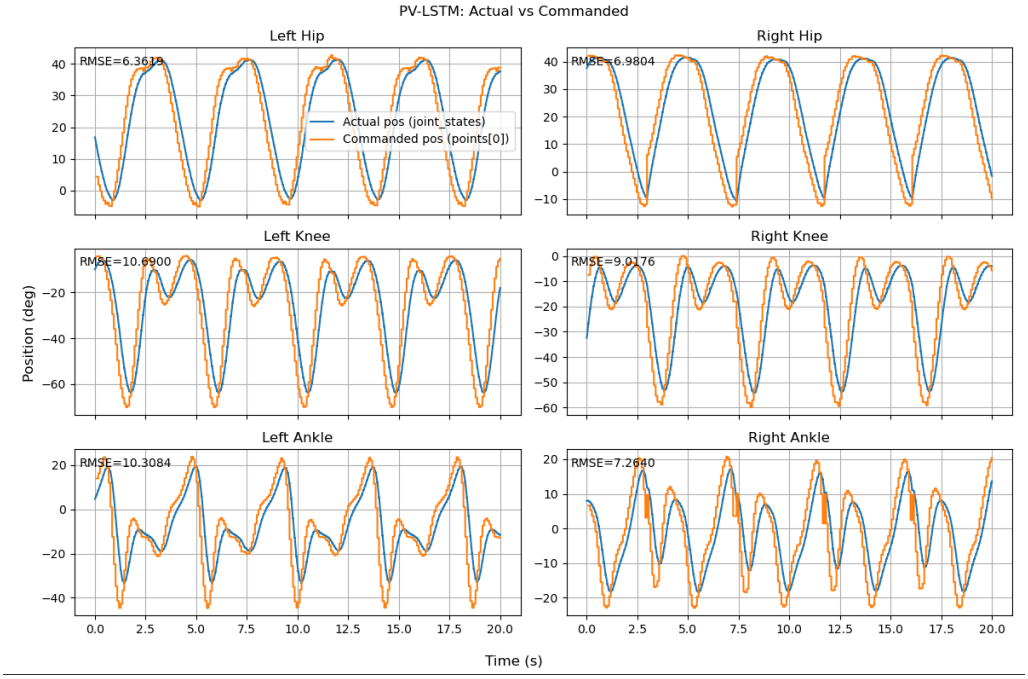


Figure 5.5. *joint_states vs commanded_joint_positions for PV LSTM Model*

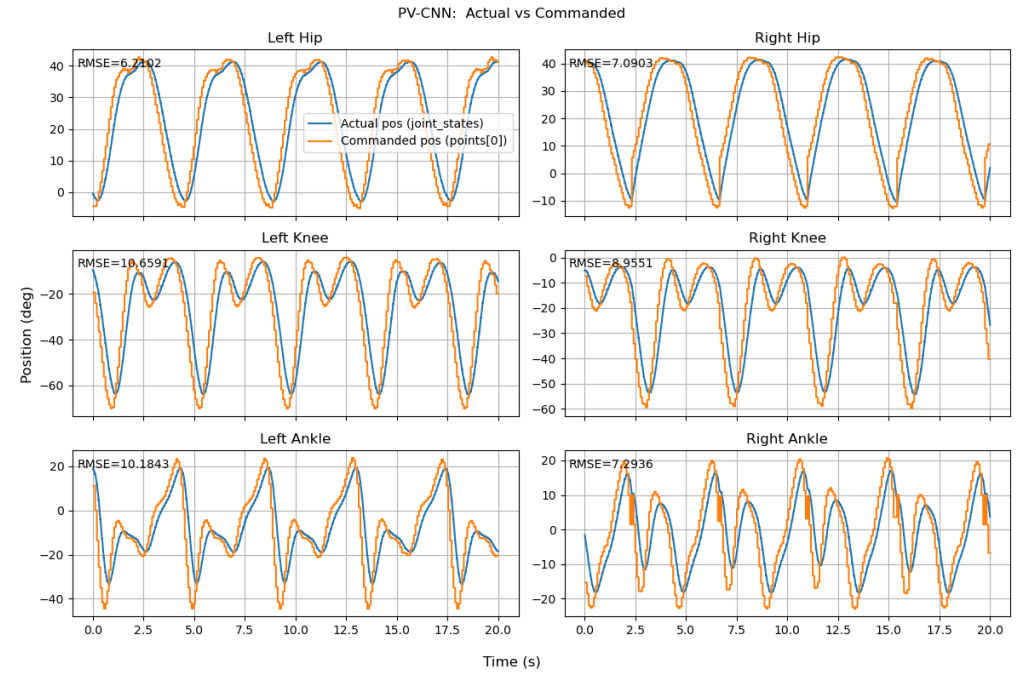


Figure 5.6. *joint_states vs commanded_joint_positions for PV CNN Model*

A consistent trend across all joints indicates that the phase variable models (PV-LSTM and PV-CNN) show better temporal alignment between the requested and actual trajectories than their timestamp equivalents (TS-LSTM and TS-CNN). Specifically, phase variable conditioning minimizes phase drift throughout consecutive gait cycles, leading to more seamless transitions at stride boundaries and enhanced consistency in joint patterns. This enhancement is particularly noticeable in the knee and ankle joints, where quick

shifts in speed happen during toe-off and the initial swing. Models based on timestamps often show minor yet accumulating phase discrepancies, resulting in perceptible misalignment between anticipated and performed trajectories as time progresses. In contrast, the phase-variable models preserve steady stride-to-stride alignment, even throughout extended simulation runs.

A clear distinction can be seen between models based on timestamps and those using phase variables. The timestamp models (TS-LSTM and TS-CNN) show significant variability from cycle to cycle, inconsistent amplitude, and gradual temporal misalignment, especially at the knee and ankle joints. These effects intensify as the simulation advances, showing the buildup of errors during the recursive execution of trajectories. Additionally, past the 30 seconds mark, the timestamp models will not generate accurate joint angles, causing the controller to halt due to the quick angle disparity between the last two values. This distinction emphasizes the importance of phase-variable conditioning in separating gait advancement from fixed time. By tying predictions to a standardized gait phase instead of the time passed, the phase variable models diminish responsiveness to minor timing inaccuracies and enhance joint synchronization throughout extended execution.

Overall, these figures demonstrate that while all models can generate plausible joint trajectories, phase variable conditioning improves temporal consistency and execution stability, which are critical properties for real-time gait generation and assistive device control.

5.5 Chapter Summary

This chapter described the ROS2 and Gazebo framework developed to execute the predicted gait trajectories on a simulated lower-limb exoskeleton. The simulated model, a simplified representation of the March VI exoskeleton, was driven through the `ros2_control` framework using a position-controlled architecture with a PD tracking loop, with the hip and ankle implemented as multi-degree-of-freedom joint stacks so that the full 18-channel, all-planes predictions of Section 3.4.5 could be executed rather than only their sagittal component. A Joint Publisher Node was developed to load the trained model and scaler, compute the phase variables where required, run inference, convert and validate the resulting joint angles against the kinematic limits of the model, and publish them as trajectory commands, while monitoring joint state feedback and returning the exoskeleton to a safe double-stance configuration on termination.

Executing the predictions in this way exposed behaviour that offline error metrics do not capture. The phase-variable models maintained closer alignment between commanded and executed trajectories across consecutive strides, while the timestamp-based models accumulated progressive temporal misalignment, most visibly at the knee and ankle where angular velocities change most rapidly. This supports the interpretation that anchoring predictions to gait progression rather than to elapsed time improves stability during extended execution.

It should be emphasized that the comparisons presented in this chapter are qualitative. The framework was developed to animate and visually assess neural network

generated strides, not to quantify closed-loop tracking performance, and the observations above are drawn from visual inspection of the commanded and measured trajectories rather than from tracking-error statistics. Quantifying execution accuracy, and evaluating the models under torque or impedance control rather than position control, remain directions for future work as discussed in Chapter 6.

Chapter 6

Epilogue

6.1 Conclusions

This thesis investigated the prediction and execution of human gait trajectories using deep learning models, with a particular focus on phase-variable conditioning and its impact on gait phase matching. The work combined data-driven modeling, biomechanical insight, and robotic simulation to bridge the gap between offline gait prediction and real-time execution within a control framework. A central challenge of this work was the inherently hybrid and non-linear nature of human gait. Rapid transitions, inter-joint coupling, and stride-to-stride variability make accurate and stable gait prediction a demanding problem, particularly when predictions are recursively fed back into a control loop. Addressing these challenges required careful consideration of both model architecture and the representation of time and phase within the learning process.

Multiple neural network architectures were developed and evaluated, including timestamp and phase variable LSTM and CNN models. While all four achieved comparable accuracy on a single fixed test segment, substantial differences emerged during extended execution and, more importantly, when the models were confronted with patients and movement planes they had not been trained on. Evaluated across sixteen held-out patients, the models separated primarily by architecture: both recurrent models held their mean absolute error near two degrees and reproduced each patient's own movement pattern on the majority of cases, whereas both convolutional models exceeded six degrees and, on most unseen patients, explained less of the measured motion than a constant posture would. Recurrence, which provides the network with short-term memory of a patient's own recent kinematics, is therefore the decisive factor for generalization.

Phase variable conditioning contributed a different and equally necessary property. Its effect on one-step-ahead accuracy was modest, comparable in size to the variation between individual patients, which is unsurprising given that this mode re-anchors the model to the measured trajectory at every step and therefore never allows a timing error to develop. Its value appeared instead once the models were required to run on their own output. Under recursive rollout, and in the closed-loop simulation of Chapter 5, both phase variable models preserved stride-to-stride alignment over extended runs while both timestamp models accumulated phase drift until their trajectories diverged. This held for both backbones, including the convolutional one, confirming that anchoring predictions

to gait progression rather than to elapsed time is what prevents drift, independently of the architecture carrying it.

Combining the two, the **PV-LSTM model is identified as the model recommended by this study**, being the only architecture strong on both axes: it attains the lowest prediction error across the test patients and the most stable extended execution. The two mechanisms are complementary and act on different properties: recurrence secures generalization accuracy to unseen patients, while phase-variable conditioning secures temporal stability under recursive execution. Neither alone is sufficient for a predictor intended to drive an assistive device, since a model may track accurately for a single step yet lose synchronization within a few strides, or hold its timing while reproducing the wrong joint angles. These findings highlight the importance of incorporating biomechanically meaningful representations when applying learning-based methods to locomotion control. To evaluate these behaviors in a realistic setting, a ROS2 and Gazebo visualization framework was developed. This framework enabled direct inspection of executed joint trajectories and revealed execution level problems that are not fully captured by traditional error metrics alone. The results underscore that prediction accuracy, while necessary, is not sufficient to guarantee stable long-term behavior in gait generation.

Beyond the technical outcomes, this work required extensive iteration across data preprocessing, model design, training, and system integration. Developing a pipeline that spans clinical gait data, neural network inference, and robotic simulation proved to be a demanding but valuable process. The challenges encountered throughout this work reinforced the importance of robustness, interpretability, and careful validation when applying machine learning techniques to biomechanically complex systems.

In conclusion, this thesis demonstrates that a recurrent architecture conditioned on a phase variable, as in the proposed PV-LSTM model, provides a strong foundation for gait trajectory generation and execution, with recurrence supplying the generalization to unseen patients and the phase variable supplying the temporal stability required for extended execution. The methods and insights presented here contribute toward the development of more reliable learning-based control strategies for assistive devices and lay the groundwork for future research involving adaptive control, subject-in-the-loop experiments, and physical hardware deployment.

6.2 Future Work

Several directions for future research emerge from this work. One immediate extension involves the development and evaluation of hybrid deep learning architectures, such as Fully Convolutional Networks (FCNs), which have demonstrated strong performance in time-series modeling. Architectures of this type can leverage the feature extraction capabilities of convolutional layers while incorporating recurrent mechanisms, such as LSTM units, to capture longer-term temporal dependencies.

A further direction concerns the choice of predicted channels. The models developed here predict the hip, knee and ankle in each of the three anatomical planes, but as discussed in Section 4.3.9, the three planes differ considerably in how reliably they can be

measured, and consequently in how much weight their predictions should carry clinically. Two refinements follow from this. The first is to include the foot progression angle, which is recorded in the ELEPAP data but is not currently among the predicted channels; it is one of the most clinically used descriptors of transverse-plane function, since in-toeing and out-toeing directly inform treatment decisions, and it is less sensitive to marker placement than the segmental axial rotations. The second is to weight the training objective by plane rather than treating all channels equally, so that the model allocates capacity in proportion to clinical relevance and to the confidence attached to each measurement, rather than expending it on channels whose ground truth is itself uncertain.

Another important direction concerns the modeling of stride-to-stride transitions. While the proposed networks perform well within individual gait cycles, discrepancies can arise when transitioning between consecutive strides. This behavior is consistent with the hybrid nature of human gait, where discrete events such as heel strike and toe-off introduce rapid velocity changes and impact dynamics. Explicitly modeling these events, or incorporating event-aware or hybrid control strategies, may improve continuity and stability across gait cycles.

Another promising direction for future research is the exploration of reinforcement learning (RL) techniques for gait generation and adaptation. Unlike supervised learning approaches, reinforcement learning frameworks allow control policies to be optimized directly through interaction with the environment, potentially enabling adaptive responses to changes in terrain, speed, or external disturbances. In the context of assistive locomotion, reinforcement learning could be employed to refine gait patterns generated by the proposed neural network models, for example by optimizing energy efficiency, smoothness, or stability-related objectives. Importantly, such approaches would need to be carefully constrained, potentially through hybrid or hierarchical control architectures, to ensure safety and interpretability, particularly in human-centered applications.

Finally, the development and further tuning of the proposed modeling and simulation framework could be pursued within the facilities of the National Technical University of Athens (NTUA), enabling extended experimentation, collaboration, and potential hardware-in-the-loop validation.

Bibliography

[1] Wikipedia, "Powered exoskeleton", [Online]. Available: https://en.wikipedia.org/wiki/Rehabilitation_robotics.

[2] Yuantao Ding and Suiran Yu, "Chmer: An exoskeleton robot with active body weight support walker based on compliant actuation for children with cerebral palsy", *Biomechanics, Sensing and Bio-inspired Control in Rehabilitation and Assistive Robotics, Volume II*, 2024. DOI: 10.3389/fbioe.2025.1551039.

[3] Chen Zhengquan and Jiang Haoyu, "Flexible lower limb exoskeleton robot for rehabilitation training of children with cerebral palsy", *Chinese Journal of Rehabilitation Medicine*, 2024. DOI: 10.1007/s12204-024-2760-x.

[4] Che Ani Adi Izhar, Z. Hussain, I. F. Maruzuki, and Mohd Suhaimi Sulaiman, "Gait cycle prediction model based on gait kinematic using machine learning technique for assistive rehabilitation device", *IAES International Journal of Artificial Intelligence*, 2021. DOI: 10.11591/ijai.v10.i3.pp752-763.

[5] Prajapati Nikita, Kaur Amandeep, and Sethi Dimple, "A review on clinical gait analysis", *Conference: 2021 5th International Conference on Trends in Electronics and Informatics p.967-974*, 2021. DOI: 10.1109/ICOEI51242.2021.9452951.

[6] Ling-Long Li, Guang-Zhong Cao, Hong-Jie Liang, Yue-Peng Zhang, and Fang Cui, "Human lower limb motion intention recognition for exoskeletons: A review", *IEEE Sensors Journal, vol. 23, no. 24, pp. 30007-30036*, 2013. DOI: 10.1109/JSEN.2023.3328615.

[7] M. Rana and V. Mittal, "Wearable sensors for real-time kinematics analysis in sports: A review", *Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, 2021. DOI: 10.1109/JSEN.2020.3019016.

[8] VICON, "Vicon motion capture", [Online]. Available: https://www.vicon.com/.

[9] Reed Ferber, Sean T. Osis, Jennifer L. Hicks, and Scott L. Delp, "Gait biomechanics in the era of data science", *Journal of Biomechanics 49 3759-3761*, 2016. DOI: 10.1016/j.jbiomech.2016.10.033.

[10] RANIA KOLAGHASSI and KONSTANTINOS SIRLANTZIS, "Deep learning models for stable gait prediction applied to exoskeleton reference trajectories for children with cerebral palsy", *IEEE-Access the Interreg 2 Seas Programme 2014-2020*, 2020. DOI: 10.1109/ACCESS.2023.3252916.

[11] Binbin Su and Elena M. Gutierrez-Farewik, "Gait trajectory and gait phase prediction based on an lstm network", *Journal of Biomedical Informatics*, 2023. DOI: 10.1016/j.jbi.2023.104524.

[12] ROBERT D. GREGG, SIAVASH REZAZADEH, DAVID QUINTERO, and NIKHIL DIVEKAR, "A phase variable approach for improved rhythmic and non-rhythmic control of a powered knee-ankle prosthesis", *IEEE Access*, 2019. DOI: 10.1109/ACCESS.2019.2933614.

[13] ROBERT D. GREGG, David Quintero, Daniel J. Lambert, and Susan Kapp, "Continuous-phase control of a powered knee–ankle prosthesis: Amputee experiments across speeds and inclines", *IEEE Trans Robot*, 2018. DOI: 10.1109/TRO.2018.2794536.

[14] ROBERT D. GREGG and Dario J. Villarreal, "Unified phase variables of relative degree two for human locomotion", *Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, 2016. DOI: 10.1109/EMBC.2016.7592160.

[15] JESSY W. GRIZZLE, KOUSHIL SREENATH, MATTHIEU MASSELIN, ALEXIS DUBURCQ, and M. EVA MUNGAI, "Feedback control of an exoskeleton for paraplegics", *IEEE Control Systems Magazine*, 2018. DOI: 10.1109/MCS.2018.2866604.

[16] M. EVA MUNGAI and JESSY W. GRIZZLE, "Feedback control design for robust comfortable sit-to-stand motions of 3d lower-limb exoskeletons", *IEEE Access, vol. 9, pp. 122-161*, 2021. DOI: 10.1109/ACCESS.2020.3046446.

[17] Patrick Slade, Christopher Atkeson, and J. Maxwell Donelan, "On human-in-the-loop optimization of human–robot interaction", *Nature 633, 779–788*, 2021. DOI: 10.1038/s41586-024-07697-2.

[18] Romain Baud, Ali Reza Manzoori, Auke Ijspeert, and Mohamed Bouri, "Review of control strategies for lower-limb exoskeletons to assist gait", *NeuroEngineering Rehabil 18, 119*, 2021. DOI: 10.1186/s12984-021-00906-3.

[19] ELEPAP, "Hellenic society for the protection and rehabilitation of disabled people", [Online]. Available: https://elepap.gr/.

[20] Scikit-learn Devs, "Standardscaler documentation", [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html.

[21] Milena Afeworki, "A comparison of machine learning algorithms: Knn vs decision treesr", [Online]. Available: https://milena-pa.medium.com/a-comparison-of-machine-learning-algorithms-knn-vs-decision-trees-d6110e08bfea.

[22] Valiev Koyiljon, "A gentle introduction to feed-forward neural networks with numpy", [Online]. Available: https://medium.com/@valievkoyiljon112/a-gentle-introduction-to-feed-forward-neural-networks-with-numpy-b13f2ea7ac73.

[23] Python Developers, "Python", [Online]. Available: https://www.python.org/.

[24] NumPy Developers, "Numpy documentation", [Online]. Available: https://numpy.org/.

[25] Pandas Developers, “Pandas documentation”, [Online]. Available: https://pandas.pydata.org/.

[26] Joblib Developers, “Joblib documentation”, [Online]. Available: https://joblib.readthedocs.io/en/stable/.

[27] Matplotlib Developers, “Matplotlib documentation”, [Online]. Available: https://matplotlib.org/.

[28] PyTorch Devs, “Adam pytorch”, [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html.

[29] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama, “Optuna: A next-generation hyperparameter optimization framework”, *25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019. DOI: 10.1145/3292500.3330701.

[30] Greg Van Houdt, Carlos Mosquera, and Gonzalo Nápoles, “A review on the long short-term memory model”, *IEEE Sensors Journal, vol. 21, no. 2, pp. 1187-1207*, 2021. DOI: 10.3390/s20247127.

[31] Tensorflow-Keras, “Keras, a high-level api for tensorflow”, [Online]. Available: https://www.tensorflow.org/guide/keras.

[32] Wikipedia, “Root mean square deviation”, [Online]. Available: https://en.wikipedia.org/wiki/Root_mean_square_deviation.

[33] Wikipedia, “Mean absolute error”, [Online]. Available: https://en.wikipedia.org/wiki/Mean_absolute_error.

[34] esri, “Probability plot of residual histograms”, [Online]. Available: https://pro.arcgis.com/en/pro-app/3.4/tool-reference/spatial-statistics/h-residual-histogram.htm.

[35] ROS Community, “Ros packages”, [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html.

[36] ROS Community, “Ros nodes”, [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html.

[37] ROS Community, “Ros topics”, [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html.

[38] ROS Community, “Ros messages”, [Online]. Available: https://docs.ros.org/en/humble/p/std_msg/.

[39] ROS Community, “Ros services”, [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html.

[40] ROS Community, “Ros launch files”, [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Intermediate/Launch/Launch-Main.html.

[41] ROS Community, “Ros urdf”, [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html.

[42] ROS Community, “Ros control”, [Online]. Available: https://control.ros.org/humble/index.html.

[43] ROS Community, “Ros bags”, [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html.

[44] Gazebo, “Gazebo simulation”, [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Gazebo/Gazebo.html.

[45] Project March, “Project march exoskeleton model vi”, [Online]. Available: https://www.projectmarch.nl/exoskeleton-m7-1-1.

[46] Open Source Robotics Foundation, “Contact sensor gazebo plugin”, [Online]. Available: https://classic.gazebosim.org/tutorials?tut=contact_sensor.

[47] ROS Community, “Ros2 control manager”, [Online]. Available: https://control.ros.org/rolling/doc/ros2_control/controller_manager/doc/userdoc.html.

[48] ROS Community, “Ros2 cascade control manager”, [Online]. Available: https://docs.ros.org/en/jazzy/p/controller_manager/doc/controller_chaining.html.

[49] Davide Faconti, “Plotjuggler”, [Online]. Available: https://github.com/facontidavide/PlotJuggler.

Appendix A

Source Code

All the codes created, or mentioned in the chapters, are **open source** and can be found on the following git repositories:

Neural Networks For Gait Analysis and Prediction:
https://github.com/Evangelos-Karavas/Neural-Networks-For-Gait-Analysis-and-Prediction

ROS2 Lower Limb Exoskeleton:
https://github.com/Evangelos-Karavas/ROS2-Lower-Limb-Exoskeleton-Control

A.1 Simulation Launch Instructions

This section describes the procedure to build the ROS 2 workspace and launch the simulation environment used throughout this thesis. All commands were executed on Ubuntu 22.04 with ROS 2 Humble.

A.1.1 Workspace Build

After installing ROS 2 Humble, Gazebo, and cloning the ROS 2 Lower Limb Exoskeleton repository into the *src* directory of a ROS 2 workspace, the workspace must be built using `colcon`:

```
colcon build
```

A.1.2 Environment Setup

Before running any ROS 2 nodes, the ROS 2 environment and the local workspace must be sourced:

```
source /opt/ros/humble/setup.bash
source install/setup.bash
```

These commands must be executed in every new terminal session or added to the *bashrc* file.

A.1.3 Launching the Simulation

The Gazebo simulation environment is launched using:

```
ros2 launch exo_gazebo gazebo.launch.py
```

Once the simulation is running, the joint command publisher can be started. The two nodes made for this work:

Timestamp neural network node:

```
ros2 launch exo_control joint_publisher.launch.py
```

Phase variable neural network node:

```
ros2 launch exo_control joint_publisher_pv.launch.py
```

For visualization of the robot model and joint states, PlotJuggler and RQT graph can be launched using:

```
ros2 run plotjuggler plotjuggler
```

```
ros2 run rqt_graph rqt_graph
```

In addition, inside the *misc/plotjuggler.xml* file, a prepared plotting structure has been created for quick visualisation. To record a ROS 2 topic to a CSV file for offline analysis, the following command can be used:

```
ros2 topic echo /joint_states --csv > topic.csv
```

A.2 Neural Network Models Scripts

A.2.1 CNN Model

Listing A.1: CNN model code using 6 joints as input

```
model = Sequential([
    # Block 1: 32 -> 48 -> pool
    ZeroPadding1D(padding=2, input_shape=(window, n_features)),
    Conv1D(filters=32, kernel_size=3, strides=2,
    dilation_rate=1, padding="same", activation="relu"),
    ZeroPadding1D(padding=2),
    Conv1D(filters=48, kernel_size=3, strides=2,
    dilation_rate=1, padding="same", activation="relu"),
    MaxPooling1D(pool_size=2, strides=2),
    Dropout(dropout),

    # Block 2: 256 -> 256 -> pool
    ZeroPadding1D(padding=2),
    Conv1D(filters=256, kernel_size=3, strides=2,
    dilation_rate=1, padding="same", activation="relu"),
```

```
        ZeroPadding1D(padding=2),
        Conv1D(filters=256, kernel_size=3, strides=2,
        dilation_rate=1, padding="same", activation="relu"),
        MaxPooling1D(pool_size=2, strides=2),
        Dropout(dropout),

        Flatten(),
        Dense(128, activation="relu"),
        Dropout(dropout),
        Dense(n_features, activation="linear"),
    ])
    model.compile(
        optimizer=Adam(learning_rate=0.0001),
        loss="mse",
        metrics=["mae", RootMeanSquaredError()],
    )
    return model
```

A.2.2 LSTM Model

Listing A.2: LSTM model code using 6 joints as input

```
    model = Sequential([
        LSTM(128, activation="tanh", return_sequences=True),
        Dropout(0.2),
        LSTM(128, activation="tanh", return_sequences=False),
        Dropout(0.2),
        Dense(128, activation="relu"),
        Dense(output_dim, activation="linear"),
    ])
    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss="mse",
        metrics=["mae", RootMeanSquaredError()],
    )
    return model
```

A.2.3 Phase Variable Computation

Listing A.3: Phase Variable Computation for Neural Networks Input

```
def compute_pv_stride(q: np.ndarray, c: float = 0.53):
```

```
    """
    q: hip angle over one stride, shape (N,)
    c: fixed empirical scaling constant (c = 0.53), applied identically
       to every subject and every stride
    Returns Phase Variable in [0,1].
    """
    q = q.astype(np.float64)
    N = q.shape[0]

    c = float(np.clip(c, 0.05, 0.95))
    q0 = float(q[0])
    idx_min = int(np.argmin(q))
    qmin = float(q[idx_min])

    denom = (q0 - qmin)
    s = np.zeros(N, dtype=np.float64)

    # stance portion (down to min)
    s[:idx_min + 1] = ((q0 - q[:idx_min + 1]) / denom) * c

    # swing portion
    s[idx_min:] = 1.0 + ((1.0 - c) / denom) * (q[idx_min:] - q0)

    s = np.clip(s, 0.0, 1.0)
    return s
```

A.3 ROS2 - Gazebo Scripts

A.3.1 Joint Publisher

Listing A.4: Generic Joint Publisher Node for moving the joints in ROS2-Gazebo Simulation using predefined Excel file containing joint positions.

```
class JointPublisherFromExcel(Node):
    def __init__(self):
        super().__init__('joint_publisher_from_excel')

        self.traj_pub = self.create_publisher(JointTrajectory,
        '/trajectory_controller/joint_trajectory', 10)

        self.joint_names = [
            'left_hip_revolute_joint',
```

```
            'right_hip_revolute_joint',
            'left_knee_revolute_joint',
            'right_knee_revolute_joint',
            'left_ankle_revolute_joint',
            'right_ankle_revolute_joint'
        ]
        pub_rate = 0.1
        pkg_dir = get_package_share_directory('exo_control')
        excel_path = os.path.join(pkg_dir,
        'neural_network_parameters/excel', 'data_cp.xlsx')
        columns = ['LHipAngles (1)', 'RHipAngles (1)',
        'LKneeAngles (1)', 'RKneeAngles (1)',
        'LAnkleAngles (1)', 'RAnkleAngles (1)']
        df = pd.read_excel(excel_path, sheet_name="Data",
        usecols=columns, skiprows=[1, 2])
        self.joint_data = df.values.astype(np.float32)
        self.traj_index = 0
        self.timer = self.create_timer(pub_rate, self.timer_callback)

    def timer_callback(self):
        num_samples = len(self.joint_data)
        if self.traj_index >= num_samples:
            self.get_logger().info("All joint positions sent.")
            self.traj_index = 0
            return
        joint_positions = np.radians(self.joint_data[self.traj_index])

        # publish trajectory message
        msg = JointTrajectory()
        msg.joint_names = self.joint_names
        point = JointTrajectoryPoint()
        point.positions = joint_positions.tolist()
        point.time_from_start = Duration(sec=0, nanosec=100_000_000)
        msg.points.append(point)
        self.traj_pub.publish(msg)
        self.traj_index += 1

def main(args=None):
    rclpy.init(args=args)
    node = JointPublisherFromExcel()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```

A.3.2 ROS2 Launch file

Listing A.5: ROS2 Launch file for controller manager, and joint_publisher node.

```
def generate_launch_description():
    exo_description_pkg = FindPackageShare('exo_description')
    exo_control_pkg = FindPackageShare('exo_control')
    urdf_path = PathJoinSubstitution([exo_description_pkg, 'urdf',
    'lleap_exo.urdf.xacro'])
    controllers_config = PathJoinSubstitution([exo_control_pkg,
    'config', 'ros2_controller.yaml'])
    # Load robot description
    robot_description = ParameterValue(Command(['xacro ',
    urdf_path]), value_type=str)

    return LaunchDescription([
        # Start joint state broadcaster and trajectory controller
        Node(
            package='controller_manager',
            executable='spawner',
            arguments=['joint_state_broadcaster'],
            output='screen'
        ),
        Node(
            package='controller_manager',
            executable='spawner',
            arguments=['trajectory_controller'],
            output='screen'
        ),
        # Start the joint publisher node
        TimerAction(
            period=2.0,
            actions=[
                Node(
                    package='exo_control',
                    executable='joint_publisher',
                    name='joint_trajectory_publisher_node',
```

```
                output='screen'
            )
        ]
    )
])
```

A.3.3 ROS2-Gazebo PD controller gains and constraints

Listing A.6: ROS2 controller used for visualization

```
controller_manager:
  ros__parameters:
    update_rate: 1000

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    trajectory_controller:
      type: joint_trajectory_controller/JointTrajectoryController

trajectory_controller:
  ros__parameters:
    joints:
      - left_hip_revolute_joint
      - left_knee_revolute_joint
      - left_ankle_revolute_joint
      - right_hip_revolute_joint
      - right_knee_revolute_joint
      - right_ankle_revolute_joint

    command_interfaces: [position]
    state_interfaces: [position]

    update_rate: 500
    action_monitor_rate: 100.0

    interpolate_from_desired_state: true
    allow_partial_joints_goal: false
    allow_nonzero_velocity_at_trajectory_end: false

    gains:
      left_hip_revolute_joint: {p: 5000.0, i: 0.0, d: 100.0}
```

```
    left_knee_revolute_joint: {p: 5000.0, i: 0.0, d: 100.0}
    left_ankle_revolute_joint: {p: 500.0, i: 0.0, d: 10.0}
    right_hip_revolute_joint: {p: 5000.0, i: 0.0, d: 100.0}
    right_knee_revolute_joint: {p: 5000.0, i: 0.0, d: 100.0}
    right_ankle_revolute_joint: {p: 500.0, i: 0.0, d: 10.0}

  constraints:
    stopped_velocity_tolerance: 0.01
    goal_time: 0.5
    left_hip_revolute_joint: {trajectory: 2.0, goal: 2.0}
    left_knee_revolute_joint: {trajectory: 2.0, goal: 2.0}
    left_ankle_revolute_joint: {trajectory: 2.0, goal: 2.0}
    right_hip_revolute_joint: {trajectory: 2.0, goal: 2.0}
    right_knee_revolute_joint: {trajectory: 2.0, goal: 2.0}
    right_ankle_revolute_joint: {trajectory: 2.0, goal: 2.0}
```